

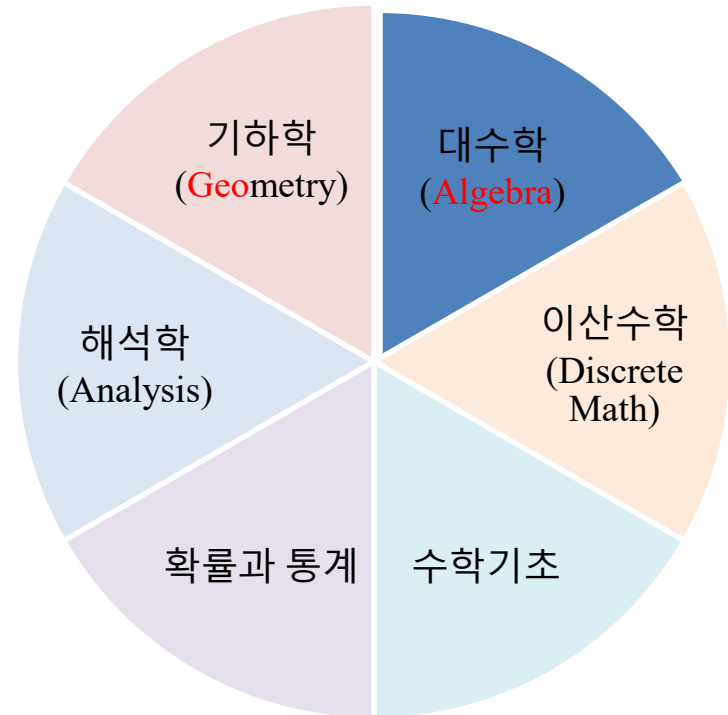
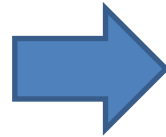
AI .ML.DL 꼭 알아야 할 수학 (Linear Algebra)

* 목차

1. 수학 역사 살펴보기
2. 머신러닝을 위해 꼭 알아야 할 수학
3. 선형대수학
4. 확률 분포
5. 확률 process & 최적화

Mathematics

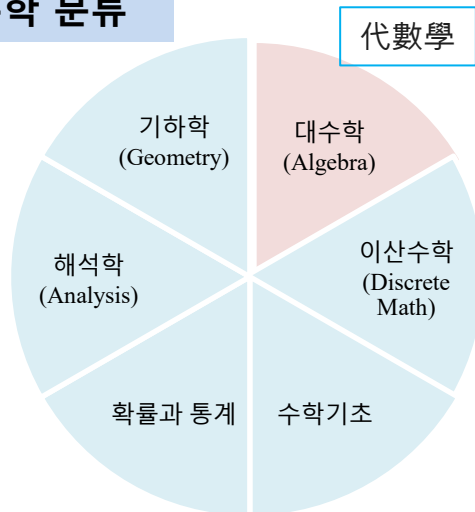
수를 바탕으로 하는 논리 체계를 연구하는 학문



$$R + G + B \rightarrow ?$$

Mathematics

수학 분류



- 수와 수학적 구조 사이의 연산 및 관계를 연구
- 새로운 대상과 새로운 연산을 활용 : 벡터 행렬 등
- 숫자뿐만 아니라, 미지수를 포함하는 방정식과 이러한 미지수를 포함하는 다양한 수학적 구조를 다루면서, 수학적 문제를 일반화하고 추상화하는 데 중점을 둔다.

- 추상대수학 (Abstract Algebra) : Group(군론) Ring , Field theory
- 선형대수학 (Linear Algebra) : 선 형태의 공간, 선형공간에서의 대수
→ Vector , Matrix
- 정수론 : 페르마의 마지막 정리

1. 기초 대수학 (Elementary Algebra)

주로 변수를 포함하는 방정식과 이들을 해결하는 기법을 다룬다.
예를 들어, 선형 방정식 $ax+b=c$ 를 해결하여 x 의 값을 찾는 과정이 포함된다.

2. 추상 대수학 (Abstract Algebra)

'구조'에 초점을 맞춘다. 여기에는 그룹, 환, 체와 같은 대수적 구조들이 포함되며, 이들 구조 내에서의 연산 법칙과 속성들을 체계적으로 연구 → 이러한 대수적 구조들은 다양한 수학적 및 실제 응용 문제를 모델링하는 데 사용

3. 선형 대수학 (Linear Algebra)

벡터 공간과 이들 공간에서의 선형 변환을 다룬다. 선형 대수학은 행렬 이론을 포함하며, 컴퓨터 그래픽스, 공학, 통계학, 자연과학, 경제학 등 광범위한 분야에서 중요한 도구로 사용

4. 대수 기하학 (Algebraic Geometry)

대수적 구조(특히, 다항식 방정식)를 이용해 기하학적 구조를 연구하는 학문
이 분야는 곡선, 곡면, 다양체 등의 기하학적 형상을 대수적 방정식을 통해 설명하고, 그 성질을 연구

5. 컴퓨터 대수학 (Computational Algebra)

대수적 문제를 해결하기 위해 컴퓨터 알고리즘을 설계하고 구현하는 분야
이는 큰 수의 인수분해, 대수적 방정식의 해 찾기, 기호적 계산 등에 사용

* 쉬어가기



알과리즈미 (780~ ?)

- 페르시아의 수학자. 페르시아 최초의 수학책을 만들었는데,
- 인도에서 도입된 아라비아 숫자를 이용하여 최초로 사칙연산(덧셈, 뺄셈, 곱셈, 나눗셈)을 만들고
- 0과 위치값을 사용한 수학자
- 10진법 보급자
- '대수학의 아버지'로 불리기도 한다.
- 알고리즘이라는 말은 그의 이름에서 나왔고, 대수학을 뜻하는 영어 단어 앨지브라(Algebra)는 그의 저서 <al-jabr wa al-muqabala>로부터 기원한다

1983년 9월 6일 소련에서 알과리즈미 출생 1200주년을 기념하기 위해 만든 우표

<https://ko.wikipedia.org/wiki/%EC%BD%B0%EB%A6%AC%EC%A6%88%EB%AF%B8>

✓ 아라비아 숫자

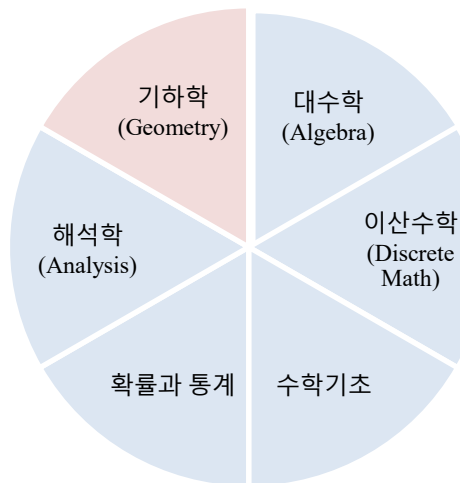
현재 전 세계에서 가장 널리 사용되는 숫자 체계 → 0에서 9까지의 10개 숫자로 구성

✓ 로마 숫자

고대 로마에서 사용되기 시작한 숫자 체계 → I, V, X, L, C, D, M의 일곱 기호를 사용하여 수를 표현

- 제곱은 근과 같다 ($ax^2 = bx$)
- 제곱은 숫자와 같다 ($ax^2 = c$)
- 근은 숫자와 같다 ($bx = c$)
- 제곱과 근의 합은 숫자와 같다 ($ax^2 + bx = c$)
- 제곱과 숫자의 합은 근과 같다 ($ax^2 + c = bx$)
- 근과 숫자의 합은 제곱과 같다 ($bx + c = ax^2$)

수학 분류



- 공간의 속성과 그 안에 있는 점, 선, 면, 입체 등의 관계를 연구하는 수학의 한 분야
- 기하학은 고대 그리스 시대부터 발전해 왔으며 (이집트), 오늘날에도 과학, 공학, 건축, 예술 등 다양한 분야에서 매우 중요한 역할
- (구분) 유클리드 기하학, 비유클리드 기하학, 현대 기하학

유클리드 기하학(Euclidean geometry) 기하학 원본 제13권

고대 그리스 수학자 유클리드가 "원론"에서 체계화한 기하학 평면(2차원)과 공간(3차원)에서의 기하학적 개념과 명제를 다룬다.

점, 선, 평면:

- 점: 위치
- 선: 두 점을 연결
- 평면: 무한한 두 차원 공간을 의미

각과 삼각형: 두 선의 만나는 점에서 형성되는 각과, 세 점이 연결되어 만들어지는 삼각형의 성질을 연구

원과 다각형: 중심과 반지름을 가지는 원과, 세 개 이상의 각과 변을 가진 다각형의 특성을 다룬다.

- 원론을 명제나 정리부터 시작하지 않고 **정의(定義, definition)**로 시작
- 수학에서 정의는 개인적인 사유 혹은 학자간 대화의 견고한 출발점

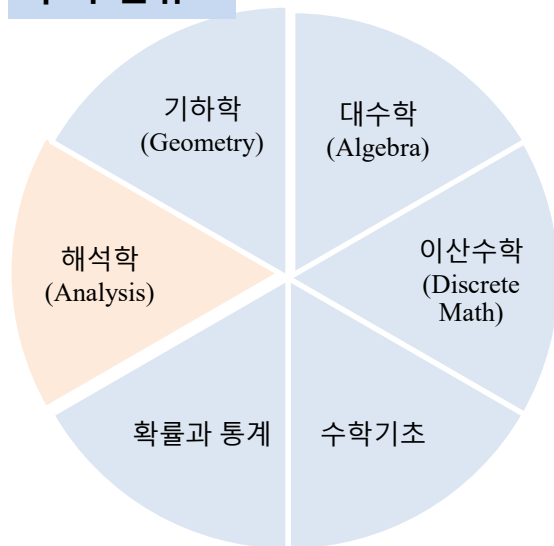
비유클리드 기하학 (Non-Euclidean Geometry) : 지구는 평면인가?

19세기 초에 발견된 비유클리드 기하학은 유클리드 기하학의 기본 가정 중 하나인 평행선 공준을 수정하거나 제외함으로써 개발

쌍곡 기하학 (Hyperbolic Geometry): 평행선 공준이 두 평행선이 무한히 멀어질 수 있다고 주장하는 기하학으로, 특이한 성질을 가지며 주로 고곡률 공간에서 발견된다.

구면 기하학 (Spherical Geometry): 공간이 구면과 같이 곡률을 가지는 경우의 기하학으로, 평행선이 존재하지 않는다.

수학 분류



해석학 (Analysis)

- 대수학과 기하학에 대하여, 미분과 적분의 개념을 기초로 함수의 연속성에 관한 성질을 연구하는 수학의 분야
- 데카르트 (좌표계 탄생) → 뉴턴 과 라이프니츠 미분 탄생
- 함수, 극한, 미분, 적분, 수열, 급수 등을 다루며, 이러한 개념들을 통해 함수의 연속성, 변화율, 누적 변화량과 같은 현상을 정밀하게 분석
- 기본적인 대수학과 기하학적 인사이트를 수학적 엄밀성과 함께 깊이 있게 탐구하며, 자연과학과 공학의 여러 분야에 필수적인 수학적 도구를 제공
- 수학, 종교, 철학 등 사용

1. 미적분학

2. 미분방정식 (Differential Equations),

변수들과 그들의 미분(도함수) 사이의 관계를 설명하는 방정식 → 물리학, 공학, 생물학 등 다양한 과학 분야에서 자연 현상을 모델링하는 데 사용

3. 실수 해석학 (Real Analysis) : 실수체계

실수 선상에서의 함수와 수열의 극한, 연속성, 미분 가능성, 적분 가능성 등을 다룬다.
극한의 개념을 바탕으로 미분과 적분을 정의하고, 이를 활용해 실수 함수의 성질을 연구

4. 복소수 해석학 (Complex Analysis) : 복소수 체계

복소수 변수에 대한 함수를 다루며, 이 분야에서는 복소수 평면에서의 함수의 연속성, 미분 가능성(해석성), 적분을 연구

5. 함수해석학 (Functional Analysis) : 벡터공간 연산자

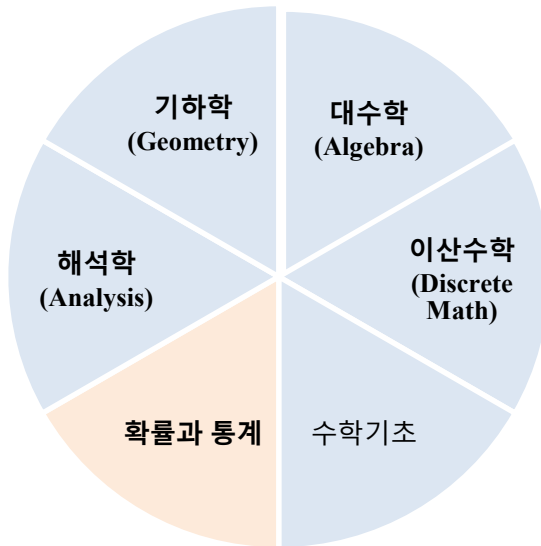
함수공간과 그 위의 연산자를 다루는 해석학의 분야 → 이는 힐베르트 공간이나 바나흐 공간 같은 무한차원 공간에서의 문제를 분석.

6. 수치해석학 (Numerical Analysis)

수학적 문제를 수치적으로 근사하는 방법을 연구 → 이는 컴퓨터를 사용하여 미분, 적분, 미분방정식의 수치적 해를 구하는 방법 등을 포함

Mathematics

수학 분류



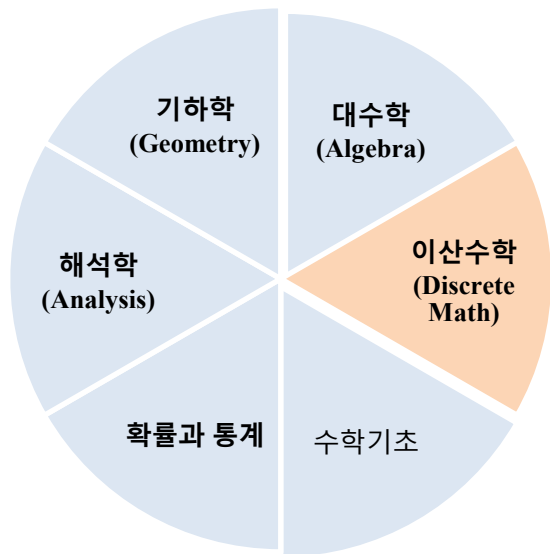
확률 (Probability) & Statistics 통계학

- 표본 공간, 사건, 순열
- 확률, 조건부 확률, 베이즈 정리
- 확률변수 : 누적분포함수, 확률밀도함수, 기대값, 분산
- 확률분포 : 이산확률분포(베르누이, 이항.) 연속확률분포(정규, 표준정규, t분포, F분포 등)
- 공분산, 상관관계
- 모수 추정 : 확률분포, 추정량, 편향(biased), 불편추정량 (unbiased estimator)
BLUE(Best Linear Unbiased Estimator)
- 게임이론
- 결정이론

게임이론(game theory)

- 둘 이상의 의사 결정자(플레이어)가 상호 작용하는 상황에서 각 플레이어가 자신의 이익을 최대화하기 위해 선택할 수 있는 전략을 연구
- 게임이론의 주요 목표는 각 플레이어의 최적의 전략을 찾고, 이들이 선택하는 전략들이 시스템에 어떤 결과를 초래하는지 이해하는 것
- 게임(game)이란 효용 극대화를 추구하는 행위자들이 일정한 전략을 가지고 최고의 보상을 얻기 위해 벌이는 행위
- 경제학: 시장 가격 결정, 경매 설계, 계약 이론 등 경제학의 다양한 분야에서 게임이론이 사용
- 정치학: 선거 전략, 투표 행위, 국제 관계와 갈등 해결 등에서 게임이론적 접근이 적용
- 컴퓨터 과학: 알고리즘 설계, 네트워크 보안, 인공지능 개발 등에 게임이론이 활용
- 생물학: 동물의 진화적 전략, 생태계 내의 상호 작용 등을 분석하는 데 게임이론이 사용
- 제로섬 게임 (Zero-sum Game), 협력 게임과 비협력 게임, **Nash Equilibrium**

수학 분류



이산수학(Discrete Mathematics)

연속적이지 않은, 즉 불연속적인 구조를 다루는 수학의 한 분야로 컴퓨터 과학, 정보 이론, 암호학, 로직, 그래프 이론, 알고리즘 등과 같은 다양한 분야에서 중요

1. 집합론

집합과 집합 간의 연산 및 관계를 연구합니다. 집합은 정의된 요소의 모임으로, 수학적 연구의 기본적인 구성 요소

2. 논리와 부울 대수

제와 명제 간의 연산을 다루며, 컴퓨터 과학에서 프로그램을 이해하고 검증하는 데 사용
부울 대수는 0과 1의 값을 가지는 이진 변수에 대한 연산을 포함하며, 디지털 회로 설계의 기본을 형성

3. 관계와 함수

이산수학에서의 관계와 함수는 입력과 출력 간의 매핑을 정의
이는 데이터베이스, 검색 알고리즘, 그리고 함수형 프로그래밍 언어 등에 적용

4. 그래프 이론

노드(정점)와 그 노드를 연결하는 엣지(간선)로 이루어진 그래프를 연구
이는 네트워크 설계, 사회 네트워크 분석, 운송 경로 최적화 등 다양한 응용 분야에 활용

5. 조합론

유한 집합의 조합 가능성을 연구하며, 확률 계산, 복잡한 구조의 카운팅, 그리고 알고리즘의 효율성 분석에 적용

6. 알고리즘 이론

문제 해결을 위한 절차나 방법을 체계적으로 연구
이는 컴퓨터 프로그래밍, 데이터 처리, 자동화된 추론에서 중요한 역할

7. 암호학

정보를 안전하게 전송하고 저장하기 위한 방법을 연구 → 암호화페

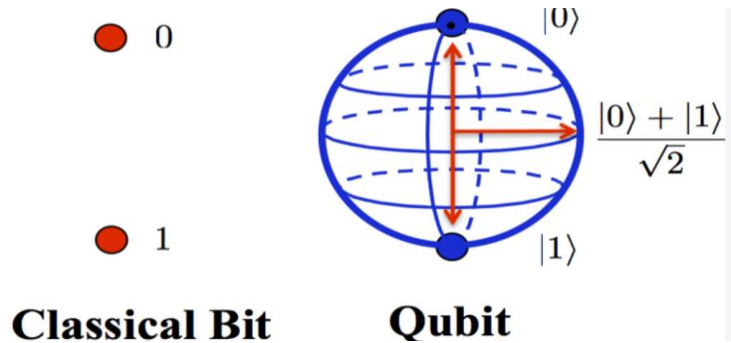
- 이산수학의 원리는 암호 알고리즘과 프로토콜의 개발에 광범위하게 적용
- 이산수학은 컴퓨터 과학과 밀접하게 연결되어 있으며, 효과적인 프로그래밍, 시스템 설계, 보안 문제 해결 등에 필수적인 기초 지식을 제공

수학 체계 구성 용어

명칭	설명	사례
공리 (axiom, 公理)	- 증명할 필요가 없이 자명한 진리 – ‘기하학원본’에서는 ‘공준’이라고 함. - 증명이 불가능 - 참 이라고 가정하는 수학적 서술 - 수학정리의 기반	평행선 공리 (유클리드 공리 중) $A=B, B=C \rightarrow A=C$ $A=B$ 이면 $A+C=B+C$
정의 (definition, 定義))	- 수학적 특성을 모호하지 않고 정확하게 표현하는 것 - 정확한 사고의 출발점 - 수학적 증명을 기술하는 기본 용어	삼각형 정의 점의 정의, 선의 정의, 평행선의 정의
정리 (theorem, 定理)	수학에서 정의나 공리에 의해 가정(assumption)으로부터 이미 진리(참)로서 증명된 명제	피타고라스의 정리
명제 (proposition, 命題)	- 논리적이고 수학적 추론으로 증명한 수학적 서술 - 그 내용이 참인지 거짓인지를 명확하게 판단할 수 있는 식이나 문장을 의미	두 직선의 교차 여부 두 명제의 진릿값이 동일할 때, 두 명제식을 논리적 동치(Logically equivalent)
보조정리 (lemma)	중요한 정리 증명을 위해 사용하는 보조적인 수학적 서술	유클리드 보조 정리 Ito's Lemma
따름정리 (corollary)	정리, 명제 보조정리를 이용해 쉽게 증명할 수 있는 수학적 서술	산술-기하 평균 부등식
가설 (hypothesis, 假說)	과학적 자료들에 근거하여 논리적으로 유추하여 설정한 것	

컴퓨터 체계로 보는 수학

구분	현재	양자컴퓨터
작동 원리	- 이진 숫자 체계를 기반으로 작동 - 모든 데이터가 비트(bit)로 표현 - 각 비트는 0 또는 1의 상태만을 가질 수 있다.	- 양자역학의 원리를 이용 - 데이터는 큐비트(qubit)에 저장되며, 큐비트는 0과 1의 상태를 동시에 가질 수 있는 양자 중첩 상태를 활용할 수 있다. - 트랜지스터로 만들어진 게이트 대신 양자(量子) 게이트를 연산도구로 사용
처리 방식	- 데이터 처리는 논리 게이트(AND, OR, NOT 등)를 통해 이루어진다. - 이 게이트들은 비트의 조합을 변환하여 계산을 수행한다.	- 양자 중첩, 양자 얽힘, 양자 간섭 같은 현상을 이용해 데이터를 처리한다. - 이를 통해 여러 가능성을 동시에 탐색하고, 복잡한 문제를 보다 빠르게 해결할 수 있는 잠재력을 가진다.
구조	폰 노이만 아키텍처가 일반적으로 사용되며, 이는 메모리에서 명령어를 가져와서 프로세서가 순차적으로 실행하는 구조	- 큐비트 간의 얽힘을 이용하여 정보를 전송하고 처리한다. - 전통적인 논리 게이트와는 다른, 양자 게이트를 사용하여 연산을 수행한다.



인물로 보는 수학 역사



Euclid of Alexandria

- 기원전 4세기 중반
- 고대 그리스의 수학자
- 기하학계에서 가장 유명한 저서인 '원론' Elements of Geometry
- 유클리드 기하학은 좌표를 사용하지 않고 공리에서 명제로 논리적으로 진행된다는 점에서 순수 기하학,
- 공리 기하학, 논증 기하학, 합성 기하학 등으로 불리기도 하며,^[2] 좌표를 사용하는 해석기하학과 대조적이다.
- 유클리드 기하학이 적용되는 위상 공간을 **유클리드 공간(Euclidean space)**이라고 한다.

<https://namu.wiki/w/%EC%9C%A0%ED%81%B4%EB%A6%AC%EB%93%9C>

<https://namu.wiki/w/%EC%9C%A0%ED%81%B4%EB%A6%AC%EB%93%9C%20%EA%B8%B0%ED%95%98%ED%95%99>



René Descartes (1596년 - 1650년)

- 프랑스의 철학자, 수학자, 과학자,
- 근대 철학의 아버지
- 해석기하학의 창시자

https://ko.wikipedia.org/wiki/%EB%A5%B4%EB%84%A4_%EB%8D%B0%EC%B9%B4%EB%A5%B4%ED%8A%B8

인물로 보는 수학 (고대)

인물	주요 기여
유클리드(Euclid)	- 고대 그리스의 수학자. 본명인 '에우클레이데스'(고전 그리스어)보다 영어 발음 표기인 '유클리드(Euclid)'로 알려짐 - 알렉산드리아에서 활동 기하학에는 왕도가 없다라는 말을 남긴 사람으로 유명하다. 《원론》(Στοιχεῖα, Elements of Geometry) https://ko.wikipedia.org/wiki/%EC%97%90%EC%9A%B0%ED%81%B4%EB%A0%88%EC%9D%B4%EB%8D%B0%EC%8A%A4
피타고라스(Pythagoras)	- 고대 그리스의 수학자, 철학자. 기원전 570년 ~ 기원전 495년), - 피타고라스는 만물의 근원이 수라고 주장했다. - 피타고라스의 정리
아르키메데스	- 기원전 287년 경 ~ 기원전 212년 경)는 고대 그리스 마그나 그라이키아의 일부였던 시라쿠사 출신의 철학자, 수학자, 천문학자, 물리학자 겸 공학자 - 아르키메데스는 고전 고대 시기의 가장 뛰어난 수학자 가운데 한 명으로도 평가받고 있다. - 십진법의 도입, - 포물선으로 둘러싸인 도형의 넓이 계산, - 원주율의 계산
헤론(Heron)	- 고대 이집트에서 태어나 알렉산드리아에서 활약한 고대 그리스인 발명가이자 수학자, 10년경 ~ 70년 헤론 공식(Heron's formula): 삼각형의 세 변의 길이를 통해 넓이를 구하는 공식이다
디오판토스(Diophantos)	200년 또는 214년 ~ 298년 또는 330년 - 알렉산드리아에서 활동한 고대 그리스의 수학자 - 정수를 계수로 가지는 방정식에 대한 연구로 유명하다. 1차·2차 방정식 또는 연립 방정식에 대한 문제와 해법이 다루어져 있다 - 이와 같은 연구 내용을 《산학》이라는 책에 정리

인물로 보는 수학 (중세 이후)

인물	주요 기여
레오나르도 피보나치 (Leonardo Fibonacci)	1170년 ~ 1240년-1250년) 또는 레오나르도 피사노 (Leonardo da Pisa, Leonardo Pisano)는 이탈리아의 수학자 피보나치 수열: 피보나치는 1202년에 발표한 《Liber Abaci》에서 유명한 피보나치 수열을 제시 → 현대 수학에서 이를 단순한 수열보다는 귀납적 정의 및 황금 비율과의 연관성 등으로 분석되고 있음. - 피사 대성당의 탑 완공: 피사의 사탑 완공에 기여 - 아라비아 숫자 도입: 피보나치는 아라비아 숫자(0부터 9까지의 숫자)와 유리수, 제곱근 등의 개념을 이탈리아와 유럽에 소개 → 서유럽 수학 발전에 큰 기여
비에트 (Viète)	- Francois Viète 1540년~1603년 프랑스 수학자 대수학에 기여, 1591년 해석학 입문 발표 미지수를 알파벳 문자로 나타낸 최초의 수학자 - 수학에 문자를 도입했고 수학적인 내용을 문자를 사용해 간략하고 정확하게 표현하는데 기여 - 문자의 사용은 수학의 기본인 다항식과 방정식, 함수의 발전으로 이어졌고, 수학이 발전하는 계기
머키스톤의 존 네이피어 (John Napier of Merchiston)	1550년 2월 1일 ~ 1617년 4월 4일)는 스코틀랜드의 수학자, 물리학자, 천문학자 로그의 발견자로 가장 잘 알려져 있다. 또한 "네이피어의 막대"(Napier's bone)라는 도구를 발명했고 산술과 수학에서 소수점의 사용을 일반화 《로그의 놀라운 규범 설명》(Mirifici Logarithmorum Canonis Descriptio, 1614년)은 57쪽에 달하는 설명 자료와 90쪽에 달하는 삼각함수의 자연로그를 나열한 표를 포함
블레즈 파스칼 (Blaise Pascal)	1623년 6월 19일~1662년 8월 19일)은 프랑스의 심리학자, 수학자, 과학자, 신학자, 발명가, 작가, 철학자 12세 때 삼각형의 내각의 합이 180도라는 사실을 오직 스스로 힘으로 발견
르네 데카르트 (René Descartes)	1596년 3월 31일 - 1650년 2월 11일)는 프랑스의 철학자, 수학자, 과학자, 근대 철학의 아버지, 해석기하학의 창시자 좌표평면
피에르 드 페르마 (Pierre de Fermat)	1607년 ~ 1665년 프랑스의 변호사이자 수학자이며 현대 정수론의 창시자 - 미적분학에서 이용되는 여러 방법을 창안하는 등 많은 연구 성과를 남겼다. 페르마의 마지막 정리: 정수론에서 n이 3 이상의 정수일 때, $a^n + b^n = c^n$을 만족하는 양의 정수 a, b, c가 존재하지 않는다는 정리
라이프니츠 (Leibniz)	1646년 7월 1일 ~ 1716년 11월 14일)는 독일의 수학자이자 과학자, 철학자 - 아이작 뉴턴과 같이 함수를 사용한 계산법(미분과 적분)을 발명 적분기호 integral 제시, 미분을 곡선의 기울기로 (뉴턴은 미분을 극한의 개념으로)

인물로 보는 수학 : 최근

인물	주요 기여
Isaac Newton	1642년 12월 25일 출생 영국의 수학자, 물리학자, 천문학자, 철학자 미적분학 : 미적분학의 기초를 독립적으로 개발→ 그의 방식을 "플럭스(Fluxions)"라고 명명: 미분과 적분의 개념을 통해 순간적인 변화율과 면적 계산 문제를 수학적으로 해석 광학과 색채 이론 : 광학 분야에서도 혁신을 이루었으며, 백색광이 여러 색으로 구성된 스펙트럼으로 분해될 수 있음을 발견 만유인력의 법칙
브룩 테일러 (Brook Taylor)	1685~1731)는 영국의 수학자 → 뉴턴의 직접 후계자. 테일러 급수(級數)는 1715년에 저술한《증분법(増分法; Methodus Incrementorum Directa et Inversa)》에서 처음으로 발표 테일러 급수(Taylor series) : 미적분학에서 도함수들의 한 점에서의 값으로 계산된 항의 무한함으로 해석할 수를 나타내는 방법
Carl Friedrich Gauss	1777년 4월 30일, 독일 출생으로 수학, 물리학, 천문학, 통계학 분야에서 뛰어난 업적을 남긴 수학자
Leonhard Euler	스위스 출신으로, 해석학, 그래프 이론, 기하학 등 수학의 많은 분야에서 광범위한 연구를 수행한 수학자
J. L. Lagrange	1736년 1월 25일 ~ 1813년 4월 10일, 토리노, 피에몬테에서 태어난 이탈리아 태생, 프랑스와 프로이센에서 활동한 프랑스 수학자이자 천문학자 라그랑주 승수법 Lagrange Multiplier : 제약 조건이 있는 다변수 함수의 최적화 문제를 제약 조건이 없는 형태로 바꾸어 푸는 방법
Georg Friedrich Bernhard Riemann	1826년 9월 17일~1866년 7월 20일 독일 수학자 해석학, 미분기하학에 혁신적인 업적을 남겼으며, 리만기하학은 일반상대성이론의 기술에 사용되고 있다. 리만 적분, 코시-리만 방정식, 리만 제타 함수, 리만 다양체 등의 수학 용어에 남아 있다. 리만 가설을 최초로 고안한 수학자로도 유명하다
Felix Christian Klein	1849년 4월 25일 ~ 1925년 6월 22일 독일 수학자 함수론, 방정식론, 군론, 비유클리드 기하학, 역학 등 여러 분야에 지대한 공헌
다비트 힐베르트 (David Hilbert)	1862년 1월 23일 ~ 1943년 2월 14일 독일 수학자, 도형을 연구하는 수학의 한 분야인 기하학을 공리화하였으며, 힐베르트 공간을 정의하여 함수해석학의 기초 마련 일반 상대성 이론을 수학적으로 정의하는 데 핵심적인 역할
Andrew Wiles	페르마의 마지막 정리를 증명하여 수세기 동안의 수학적 난제를 해결한 영국의 수학자

인물로 보는 천재 : 20c

John von Neumann

- 1903년 12월 28일 **헝가리** 부다페스트에서 출생 유대인- 1957년 2월 8일 미국 워싱턴 D.C.에서 사망
- 저명한 수학자이자 컴퓨터 과학자, 물리학자, 화학자로 **프린스턴 대학교** 천재 of 천재
- 그의 업적은 수학, 과학, 공학, 경제학, 컴퓨터 과학 분야에 걸쳐 있으며, 그는 특히 게임 이론의 발전과 현대 전자식 컴퓨터의 구조 개발에 지대한 영향



- 1930년대 초에 미국으로 이주하여 프린스턴 대학교에서 교수로 재직
- 게임 이론의 개념을 정립하고, 이론적 컴퓨터 과학의 초석을 놓았다.
- 그의 연구는 수학적 모델링, 양자역학, 유체역학, 통계학 등 다양한 분야에 걸쳐 있다. **게임 이론 (심리학 + 수학+ 경제학)**
- 폰 노이만은 오스카 모르겐슈테른과 함께 작업하여 "**게임 이론과 경제 행동(Theory of Games and Economic Behavior)**"을 출판
- 전략적 의사 결정 상황에서의 게임 이론을 체계적으로 소개했으며, 이는 경제학, 정치학, 심리학 등 다양한 분야에 큰 영향을 미쳤다.

컴퓨터 과학에의 기여

- 폰 노이만은 전자식 디지털 컴퓨터의 아키텍처인 "폰 노이만 아키텍처"를 제안 : ENIAC (30톤)→ EDVAC 보고서

노이만 아키텍처: CPU - 메모리 - 프로그램 → 현대 컴퓨터의 기본 설계 원리

양자역학: 양자역학의 수학적 기초를 정립하는 데 중요한 역할 저서 "양자역학의 수학적 기초(Mathematical Foundations of Quantum Mechanics)"는 이 분야의 클래식으로 여겨진다.

후반 생애

- 미국 원자력 위원회의 자문역으로 활동하면서 미국의 핵무기 개발 프로그램 '**Manhattan Project**'에 참여
- 호 OO, 패O OOO
- 그의 이론적 통찰력은 많은 과학적, 군사적 프로젝트에 중요한 영향을 미쳤다. 1957년에 **췌장암**으로 인해 사망

(예시) 두 기차 A와 B가 서로를 향해 같은 속도로 120킬로미터 떨어진 곳에서 출발한다. 각 기차의 속도는 시속 30킬로미터이다. 이때, 기차 A에서 출발한 파리가 시속 45킬로미터의 속도로 기차 B를 향해 날아간다. 파리가 기차 B에 도달하면 방향을 바꿔 기차 A를 향해 다시 날아가고, 이를 기차들이 충돌할 때까지 반복한다.
두 기차 A, B가 충돌할 때 까지 파리가 총 얼마나 이동했을까?

In mathematics you don't understand things. You just get used to them.

Exponential (function)

지수 (指數)

어떤 수의 몇 제곱하는지를 나타내는 것

쉬어가기

종이접기 : 1번 접으면? 100번 접으면?

$$x^2$$

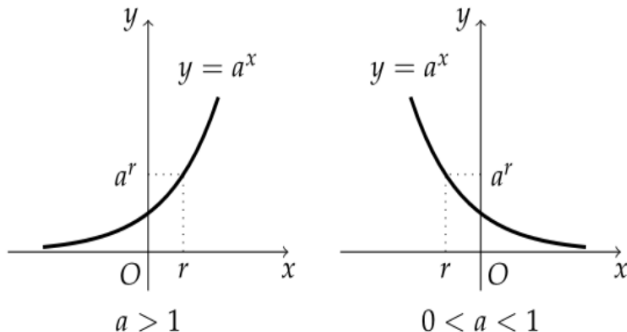
$$x^{-2} = \frac{1}{x^2}$$

지수 함수 (Exponential function)

- 거듭제곱의 지수를 변수로 하고, 정의역을 실수 전체로 정의하는 초월함수
- 로그 함수의 역함수

$a \times a \times \dots \times a$: a 를 x 번 곱한 경우

$$y = a^x$$



지수 (指數)의 성질

지수법칙

- $x^a \times x^b = x^{a+b}$
- $x^a \div x^b = \frac{x^a}{x^b} = x^{a-b}$
- $(x^a)^b = x^{ab}$

- 함수 : 대수함수와 초월함수
- 대수함수(Algebraic Function)** : 다항식의 연산으로 나타낼 수 있는 함수(+, -, $\times$) 유.무리함수
- 초월함수(Transcendental Function)** : 대수적으로 표현할 수 없는 함수 $\rightarrow$ 다항식이나 그 연산만으로는 나타낼 수 없는 함수
- 지수/로그함수, 삼각함수

그래프 특징:

- $a > 1$ 인 경우 :
 a 가 1보다 크면 지수함수는 급격히 증가하는 형태 그래프는 오른쪽으로 갈수록 급격히 위로 증가한다.
- $0 < a < 1$ 인 경우
그래프는 오른쪽으로 갈수록 점점 0에 가까워지면서 감소
- $a = 1$ 인 경우
그래프는 상수 함수가 된다 ($f(x) = 1$).

x 의 변화에 따른 큰 변화: 지수함수는 다른 함수에 비해 변수가 작을 때에는 천천히 변화하다가, 변수가 커질수록 값이 급격히 커진다.

Logarithm (log)

로그(log)

logarithm의 줄임말

대수(代數, algebra)

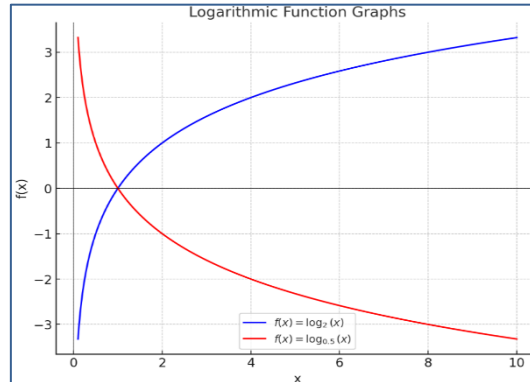
- 로그(Logarithm)는 17세기 초에 존 네이피어(John Napier)에 의해 처음 도입
- 로그는 복잡한 곱셈과 나눗셈을 덧셈과 뺄셈으로 단순화할 수 있는 강력한 도구로 수학계에 혁신

지수함수 : $y = a^x$

$a \times a \times \dots \times a$: a 를 x 번 곱한 경우 $\rightarrow a^x = y$ ($a > 0, y > 0$)
 $\rightarrow x = \log_a y$

로그 함수 $\ln(x)$

로그함수 : $y = f(x) = \log_a(x)$
 단, $x > 0, a > 0, a \neq 1$

 $a > 1$ $0 < a < 1$

지수에 대비된다는 의미에서 '대수(對數)'

$a^x = b$ 만족할 때 $x = \log_a b$ 로 정의 한다.

- 1) 밑이 2인 경우 : **이진 binary 로그**
- 2) 밑이 10인 경우 : **상용로그 (common log)**
 - 10을 생략하는 건 우리가 쓰는 수체계가 십진법이기 때문
 - 대중적인 분야의 로그에서는 대부분 상용로그이다.
- 3) 밑이 e 인 경우 : **자연(natural) 로그** $\ln x = \log_e x$

상용로그 : $f(x) = \log_{10}(x)$

자연로그 : $f(x) = \log_e(x) = \ln x$

Logarithm (log) : Common logarithm

상용로그 : $f(x) = \log_{10}(x)$

$$\log_{10}(x) = \log(x)$$

상용로그의 값의 부호

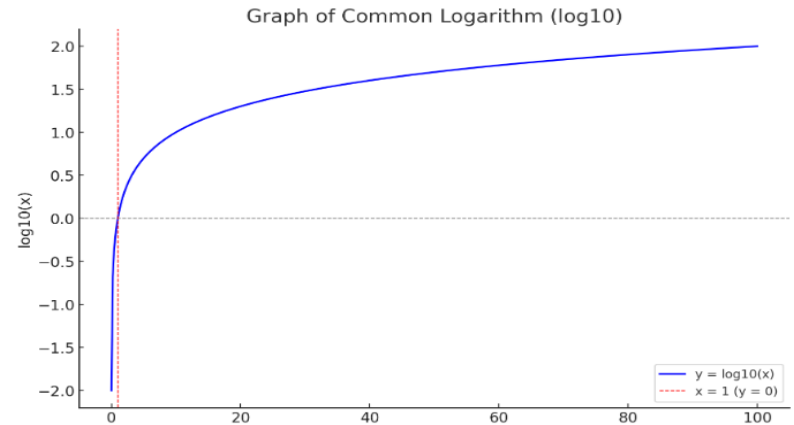
- 상용로그의 값이 양수인지, 음수인지, 또는 0인지 여부는 입력 값 x 의 크기에 따라 달라진다:
- x 가 0이거나, 음수이면 로그 값이 정의되지 않는다.

- $x=0$ 인 경우 : 0 , $\log_{10}(1)=0$
- $0 < x < 1$ 인 경우 : 음수값을 갖는다, $\log_{10}(0.1) = -1$
- $x > 1$ 인 경우 : 양수값을 갖는다. $\log_{10}(100)=2$

상용로그의 특징

- 숫자를 단순화: 상용로그는 큰 숫자를 다룰 때 매우 유용
예시) 1,000,000의 상용로그는 6이므로, 매우 큰 숫자도 상대적으로 작은 숫자로 표현할 수 있다.
- 과학적 표기법과의 관계: 상용로그는 과학적 표기법과 밀접한 관계가 있으며, 큰 수나 작은 수를 다룰 때 유용
예시) 상용로그는 주로 기계적인 계산이나 과학적 표기에서 유용하며, 특히 물리학, 화학, 공학에서 자주 사용

Graph Of Common Logarithm (Log10)



- $x=1$ 에서 $\log_{10}(x)=0$ (빨간 점선)
- $x > 1$ 인 경우, 로그 값은 양수(+)
- $0 < x < 1$ 인 경우, 로그 값은 음수.(-)
- x 가 0에 가까워질수록 $\log_{10}(x)$ 는 음(-)의 무한대로 발산

Logarithm (ln): Natural logarithm

자연로그 : $f(x) = \log_e(x) = \ln x$

$\ln(x)$: x 는 양수여야 함.. 즉, $x > 0$.

입력 값이 음수이거나 0인 경우에는 자연로그가 정의되지 않는다.

자연로그 값의 부호

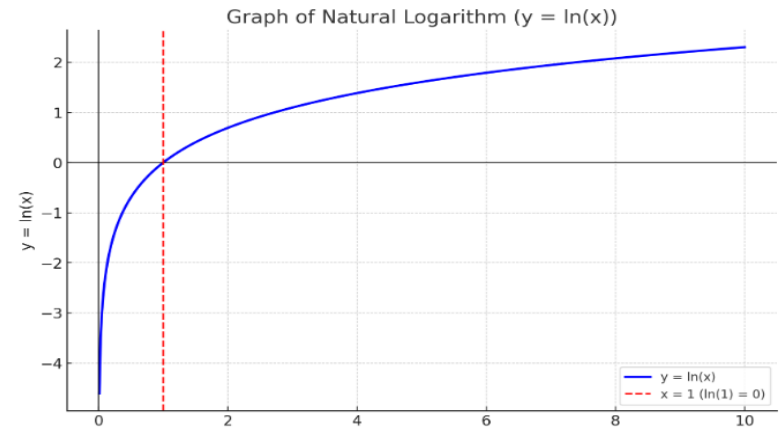
자연로그의 값이 양수인지 음수인지는 입력 값 x 의 크기에 따라 달라진다.

- $x=1$ 인 경우: $\ln(1)=0$
- $0 < x < 1$ 인 경우: $\ln(x) < 0$ (음수).
- $x > 1$ 인 경우: $\ln(x) > 0$ (양수)

자연로그의 특징

- 자연로그는 지수 함수와 밀접한 관계가 있으며, 성장과 감소를 설명하는 데 많이 사용
- 자연로그는 주로 미적분학, 물리학, 경제학, 확률 이론에서 많이 사용되며, 지수적 증가와 감소를 다루는 문제에서 유용

Graph Of Natural Logarithm (Y = Ln(X))



- $x=1$ 에서 $y=0$: $\ln(1)=0$ 이므로, 그래프가 x 축과 교차한다.
- $0 < x < 1$ 구간에서는 y 값이 음수(-)이다.
- $x > 1$ 구간에서는 y 값이 양수로 증가한다.
- x 가 0에 가까워질수록, y 는 음(-)의 무한대로 발산한다.

<https://github.com/freejyb/Deep-learning>
딥러닝(수학1)

Logarithmic functions

로그함수 특성(성질)

덧셈법칙	$\log_a xy = \log_a x + \log_a y$
뺄셈법칙	$\log_a \frac{x}{y} = \log_a x - \log_a y$
상수법칙	$\log_a 1 = 0, \log_a a = 1$
지수법칙	$\log_a x^n = n \cdot \log_a(x)$
밑 변환 법칙	$\log_b x = \frac{\log_k a}{\log_k b}$ 단, $k > 0, k \neq 1$
역수 법칙	$\log_b x = \frac{1}{\log_x b}$ 단, $b \neq 1$

로그 거듭제곱의 곱셈 법칙

$$\log_b(x^n) = n \cdot \log_b(x)$$

1. x^n 을 풀어쓰면 $x \cdot x \cdot x \dots x$ (n번 곱한다)
2. 이를 log취하면 $\log_b x^n = \log_b(x \cdot x \cdot x \dots x)$
3. 로그 곱셈법칙에 의해
 $\log_b x^n = \log_b(x) + \log_b(x) \dots + \log_b(x)$
4. 결국
 $\log_b x^n = n \cdot \log_b(x)$ 된다.

$$\log_a xy = \log_a x + \log_a y$$

곱셈이 log에 의해 덧셈이 되는 이유!!

- x 를 $b^{\log_b x}$ 로 표시 할 수 있고, y 를 $b^{\log_b y}$ 로 표시 할 수 있다.
- $x \cdot y = b^{\log_b x} \cdot b^{\log_b y}$ 이므로 거듭제곱의 곱셈법칙에 의해 $x \cdot y = b^{\log_b x + \log_b y}$ 가 된다.
- 결국 $\log_b(x \cdot y) = \log_b(x) + \log_b(y)$ 가 된다.

$$\log_a \frac{x}{y} = \log_a x - \log_a y$$

나눗셈이 log에 의해 뺄셈이 되는 이유!!

- x 를 $b^{\log_b x}$ 로 표시 할 수 있고, y 를 $b^{\log_b y}$ 로 표시 할 수 있다.
- $\frac{x}{y} = \frac{b^{\log_b x}}{b^{\log_b y}}$ 이므로 거듭제곱의 나눗셈법칙에 의해 $\frac{x}{y} = b^{\log_b x - \log_b y}$ 가 된다.
- 결국 $\log_b\left(\frac{x}{y}\right) = \log_b(x) - \log_b(y)$ 가 된다.

Logarithm (log) 활용

자연로그 (자연 대수)의 활용 사례: 머신러닝 및 데이터 과학

1. 정규화 및 데이터 스케일링

- **로그 변환(Log Transformation):** 데이터가 큰 스케일이나 비대칭인 경우, 자연로그를 사용해 데이터를 정규화할 수 있다
- **예시:** 금융 데이터(주가, 소득 등)나 지수형 데이터(코로나 감염자 수 증가 등)의 변동성을 줄이는 데 사용됩니다.

2. 로지스틱 회귀(Logistic Regression)

- 로지스틱 회귀에서 자연로그는 **시그모이드 함수**의 계산에 사용
- 자연대수의 역수 형태로 계산되며, 이는 이진 분류 문제에서 확률을 예측하는 데 사용

3. 로그 우도 함수(Log-Likelihood Function)

- 머신러닝 모델에서 우도 함수(Likelihood Function)는 데이터를 설명하는 확률을 나타낸다.
- 자연대수를 취해 로그 우도 함수(Log-Likelihood Function)를 사용하면 계산이 더 간단해지고 수렴 속도가 빨라진다.
- 로그 우도 함수는 모델의 손실 함수로 사용되며, 특히 최대우도추정법(MLE, Maximum Likelihood Estimation)에서 사용.

4. 정규화된 지수 함수(Softmax)

다중 분류 문제에서 사용되는 **소프트맥스 함수**는 자연지수를 사용하여 각 클래스의 확률을 계산한다:

5. KL 다이버전스(Kullback-Leibler Divergence)

- KL 다이버전스는 두 확률분포 간의 차이를 측정하는 지표로, 자연로그를 사용하여 계산
- 머신러닝에서 분포 간의 차이를 최소화할 때, 특히 변분 오토인코더(VAE)나 **GAN**에서 사용

6. 정보 이론 (Information Theory)

자연대수는 정보 이론에서 엔트로피(Entropy)나 교차 엔트로피(Cross-Entropy)의 계산에 사용
교차 엔트로피는 분류 문제의 손실 함수로 자주 사용

7. 그레디언트 부스팅(Gradient Boosting)

- 그레디언트 부스팅 알고리즘에서 손실 함수로 로그 손실(Log Loss)를 사용할 때 자연대수가 사용
- 로그 손실은 모델의 예측 확률이 실제 레이블과 얼마나 일치하는지 측정한다.

$$\text{Sigmoid}(z) = \frac{1}{1 + e^{-z}}$$

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

$$D_{KL}(P \parallel Q) = \sum P(x) \log \left(\frac{P(x)}{Q(x)} \right)$$

$$H(p, q) = - \sum p(x) \ln q(x)$$

$$\text{Log Loss} = - \frac{1}{N} \sum_{i=1}^N (y_i \ln(\hat{y}_i) + (1 - y_i) \ln(1 - \hat{y}_i))$$

Logarithm (log) 활용

상용로그의 활용 사례: 머신러닝 및 데이터 과학에서

1) 로그 변환 (Log Transformation) 로그 스케일링

로그 변환은 비대칭적 데이터나 스케일 차이가 큰 데이터의 분포를 정규 분포에 가깝게 만들기 위해 자주 사용. 로그를 적용하면 데이터의 범위를 줄이고, 비정규 분포를 더 다루기 쉽게 만들 수 있다.

- 예시 1: 재무 데이터 분석
 - 주가, 소득, 부동산 가격 등은 분포가 비대칭(오른쪽으로 꼬리가 긴 분포)일 수 있다. 이때 $\log_{10}$ 변환을 통해 분포를 더 대칭적으로 만들 수 있다.
 - 예시 2: 자연지수형 증가 데이터
- 바이러스 확산 데이터나 인구 성장 데이터처럼 지수적으로 증가하는 데이터에 로그 변환을 적용하면, 선형적인 패턴으로 변환되어 모델링이 더 쉬워진다.

2) 손실 함수 (Loss Function)

- 로그 손실 함수는 머신러닝 분류 문제에서 자주 사용되며, 특히 로지스틱 회귀(Logistic Regression)와 같은 확률 기반 모델의 손실 함수로 사용
- 일반적으로 자연로그가 사용되지만, 상용로그도 사용될 수 있다.

3) 이상치(outlier) 처리 및 감지

로그 변환은 데이터에서 큰 값의 영향을 줄여준다. 따라서 이상치가 많은 경우 로그 변환을 통해 데이터를 더 안정적으로 만들 수 있다.

예시: 소득 데이터에서 높은 소득자의 값이 이상치로 간주될 수 있다. 로그 변환을 하면 소득 값의 스케일이 줄어들어 이상치의 영향이 감소한다.

상용로그는 실생활 계산과 과학적 표기법에서 자주 활용되고, 자연로그는 미적분학과 지수 함수와 관련된 문제에서 핵심적인 역할을 한다.

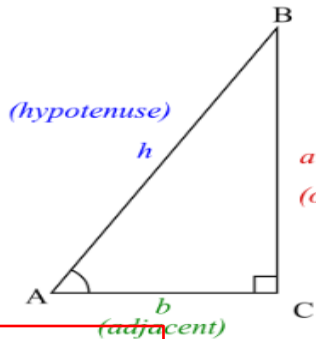
머신러닝에서도 로그는 손실 함수, 데이터 변환, 확률 계산 등 다양한 용도로 사용

밑이 2인 로그는 정보량(Information)**을 측정할 때 사용된다.
엔트로피는 확률 분포에서 정보의 불확실성을 나타낸다.

$$H(X) = - \sum_i P(x_i) \log_2 P(x_i)$$

삼각 함수(trigonometric functions)

- 삼각 함수(trigonometric functions)는 각의 크기를 삼각비로 나타내는 함수이다.
- 즉, 삼각형의 각도와 변의 길이의 관계를 나타낸 것이다.
- 가장 기본적인 삼각함수는 사인(sine), 코사인(cosine), 탄젠트(tangent)로, 이 세 가지 함수로부터 다른 삼각함수들이 정의된다.

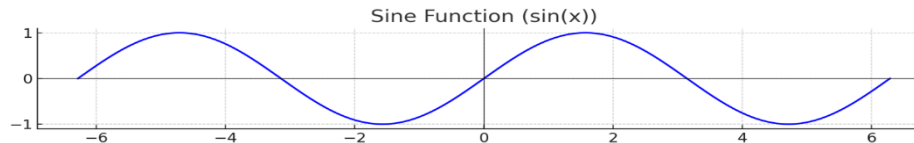


$$\cos A = \frac{b}{h}$$

1. 사인 함수 (sine, sin)

정의: 직각삼각형에서 각도의 빗변과 높이 사이의 비율

- 주기적 함수이며, 주기는 2π
- 그래프는 파형(sinusoidal) 형태
- 값의 범위는 -1에서 1 사이



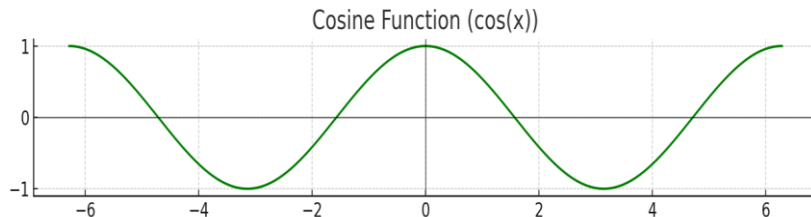
$$\sin A = \frac{a}{h}$$

$$\tan A = \frac{a}{b}$$

2. 코사인 함수 (cosine, cos)

정의: 직각삼각형에서 각도의 빗변(hyp)과 밑변 사이의 비율

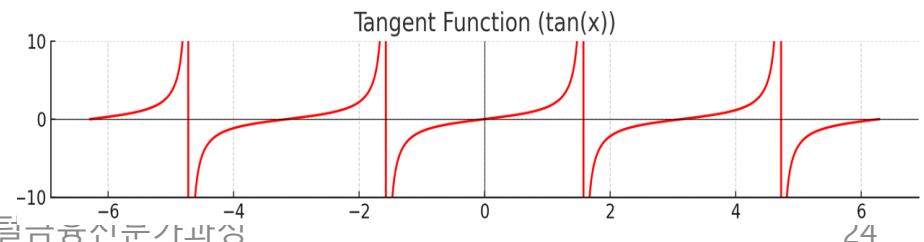
- 사인 함수와 유사하지만 위상이 다릅니다.
- 주기도 2π 로 동일하지만, $x=0$ 에서 시작할 때 값이 1
- 값의 범위는 사인 함수와 같이 -1에서 1 사이



3. 탄젠트 함수 (tangent, tan)

정의: 직각삼각형에서 각도의 대변(opp)과 인접변(adj) 사이의 비율

- 무한대(∞)로 발산하는 지점들이 있으며, 주기적으로 급격한 변화(asymptote)를 보인다.
- 탄젠트 그래프는 기울기가 매우 커지는 지점에서 수직 비슷한 선 형성



삼각 함수(trigonometric functions)

사인(sine) 함수와 코사인(cosine) 함수 머신러닝에서 활용

1. 주파수 분석 및 신호 처리 (Fourier Transform)

Fourier Transform과 같은 변환 기법은 신호 처리나 이미지 분석에서 필수적이며, 사인과 코사인 함수는 Fourier Transform의 핵심 요소

이를 통해 복잡한 신호를 여러 주파수 성분으로 분해하여 분석할 수 있다.

딥러닝에서는 CNN(Convolutional Neural Network)과 같은 모델이 주파수 성분을 분석하는 데 도움

2. 포지셔널 인코딩 (Positional Encoding)

Transformer 모델에서 사인과 코사인 함수는 입력 시퀀스의 위치 정보를 인코딩하는 데 사용된다.

BERT, GPT 같은 모델에서 포지셔널 인코딩이 사용되며, 각 위치를 사인과 코사인 함수를 기반으로 다양한 주파수로 표현해 모델이 시퀀스 내의 단어 순서를 인식할 수 있게 한다.

3. 주기적 데이터 모델링

시간 시계열 데이터 중 주기적인 패턴을 가진 데이터(예: 기온 변화, 주식 가격 변동, 에너지 소비량 등)를 모델링할 때 사인과 코사인 함수가 자주 사용

예를 들어, Recurrent Neural Network (RNN)이나 Long Short-Term Memory (LSTM) 같은 모델에서 주기적인 패턴을 처리하기 위해 입력 데이터에 사인과 코사인 값을 포함시켜, 주기적인 변화를 학습할 수 있다.

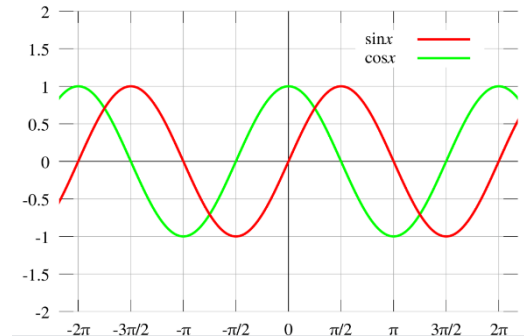
4. 주기적인 패턴 학습

딥러닝에서 주기적인 현상이나 반복되는 패턴을 학습할 때, 사인과 코사인 함수가 입력 데이터로 사용될 수 있다.

5. 주기적 활성화 함수

인공지능 모델에서 활성화 함수(Activation Function)로 사인 함수를 사용하는 경우도 있다. 이는 ReLU나 시그모이드 같은 일반적인 활성화 함수 대신에 주기적 특성을 활용하는 경우이다. 사인 활성화 함수는 특정 상황에서 유리할 수 있다.

6. 기타 : 전기공학, 컴퓨터 그래픽, 천문학, 통신



$$\sin^2 x + \cos^2 x = 1$$

* Fourier Series/ Fourier Transform

푸리에 급수(Fourier Series)

- 주기적인 함수(혹은 신호)를 여러 개의 sine과 cosine 으로 분해하여 표현하는 방법이다.
- 이 과정은 특히 복잡한 주기 함수의 주파수 성분을 분석할 때 유용하다. 푸리에 급수를 이용하면 주기적인 데이터를 다양한 주파수 성분으로 나눌 수 있어 신호 처리, 물리학, 공학, 금융 등 다양한 분야에서 많이 활용된다.
- 프랑스의 과학자이자 수학자인 조제프 푸리에가 열 방정식을 풀기 위하여 도입하였다.
- 푸리에 급수(Fourier series)는 주기 함수를 삼각함수의 가중치로 분해한 급수이다.

어떤 주기 함수 $f(x)$ 가 주기 T 를 가질 때, 이 함수를 사인과 코사인으로 나타내면 다음과 같은 형태가 된다:

$$f(x) = a_0 + \sum_{n=1}^{\infty} \left(a_n \cos \left(\frac{2\pi n}{T} x \right) + b_n \sin \left(\frac{2\pi n}{T} x \right) \right)$$

a_0 : 함수의 평균값(DC 성분).

a_n, b_n : 각각의 사인과 코사인 성분의 계수. 이 계수들은 주기 함수의 성분을 결정한다.

푸리에 급수는 주기 함수를 무한히 많은 삼각함수의 합으로 근사한다. 주어진 함수가 특정한 성질을 만족한다면(예를 들어 연속적이거나 구간적으로 연속적이라면), 이러한 근사는 정확하게 함수 자체와 같아질 수 있다.

푸리에 계수 계산

a_0 (함수의 평균값):

$$a_0 = \frac{1}{T} \int_0^T f(x) dx$$

a_n (코사인 계수):

$$a_n = \frac{2}{T} \int_0^T f(x) \cos \left(\frac{2\pi n}{T} x \right) dx$$

b_n (사인 계수):

$$b_n = \frac{2}{T} \int_0^T f(x) \sin \left(\frac{2\pi n}{T} x \right) dx$$

이러한 계수들을 통해 주어진 함수의 각 주파수 성분이 얼마나 강하게 나타나는지를 알 수 있다.

* Fourier Series/ Fourier Transform

푸리에 변환 / Fourier Transform

- 푸리에(Fourier, J. ; 1768 ~ 1830)가 처음 제안한 것으로 신호(signal)를 진동수(frequency)의 성분으로 분해(decomposition)하는 수학적 기법
- 파동의 특성을 결정짓는 기본 요소는 주파수와 진폭이다.
- 소리굽쇠를 치면 일정한 음높이의 소리를 나는데, 큰 굽쇠는 낮은 음을, 작은 굽쇠는 높은 음을 내며, 세게 치면 소리는 커지지만, 음높이는 변하지 않는다.
- 크고 작은 여러 개의 소리굽쇠를 동시에 치면 음높이가 다른 소리와 합쳐지면서 복잡한 파형을 만들어낸다.
- 여기서, 푸리에 급수(Fourier Series)는 아무리 복잡한 신호라 할지라도 기본적인 주기함수인 사인과 코사인 함수의 조합으로 전개하는 것을 말한다.
- 주파수 : 원래 1초 동안의 파동 개수인데, 공간으로 생각하면 1미터당 들어있는 파동의 개수 즉, 파수(wave number)에 해당한다.
- 주기 : 동일한 상태가 반복되는 현상에 있어서 그 반복의 최소간격을 말한다. 공간에서는 파장(wave length)에 해당하지만, 흔히 주파수나 주기라는 말을 그대로 사용한다.

푸리에 변환 (Fourier Transform)

: 주기적, 비주기적인 함수를 포함한, 연속적인 주파수로 변환하여 신호를 주파수 영역으로 표현하는 방법이다.

푸리에 변환은 비주기적인 신호에 대한 모든 주파수 성분을 연속적으로 다루기 때문에, 비주기적인 데이터에서 주파수 분포를 파악할 때 유용. → 주기적 신호를 문한대로 증가시키면 비주기적 신호

Fourier Transform은 어떤 신호(함수)를 다양한 주파수를 가진 사인파와 코사인파의 합으로 표현하는 수학적 도구

: 시간 영역(Time domain)의 신호 → 주파수 영역(Frequency domain)으로 변환

* Fourier Transform / Fourier Series

Fourier Series 활용

- 주파수 분석: 특정 신호의 주파수 성분을 분석하여 어떤 주파수가 포함되어 있는지 파악할 수 있다.
- **시계열 예측**: 주기적인 시계열 데이터를 푸리에 급수로 변환하여 계절성을 분석하고, 이를 기반으로 미래 값을 예측할 수 있다.
- 이미지 및 음성 처리: 푸리에 급수를 이용해 이미지를 주파수 성분으로 변환하면 압축이나 잡음 제거에 유용하다.

Fourier Transform 활용

1. 신호 처리 및 필터링

- 오디오 신호 분석: 음성이나 음악과 같은 오디오 신호를 주파수 성분으로 분해하여, 특정 주파수 성분을 증폭하거나 제거 가능
- 이미지 처리: 이미지의 주파수 성분을 분석하여 이미지 압축(JPEG 알고리즘)이나 엣지 검출 등에서 활용

2. 통신

- 신호 변조 및 복조: Fourier Transform은 AM/FM 라디오, TV 방송, 휴대폰 신호 등의 변조 방식을 이해하고 구현하는 데 필수적이다.
- 대역폭 분석: 통신 시스템에서 신호가 차지하는 주파수 범위를 분석하고, 이를 통해 효율적인 주파수 할당 및 주파수 간섭 제거를 수행할 수 있다.

3. 의료 영상 처리

MRI 및 CT 스캔: MRI(자기공명영상)와 CT(컴퓨터 단층촬영)와 같은 의료 영상 장치는 Fourier Transform을 사용하여 신체의 2D, 3D 이미지를 생성

신호를 주파수 도메인에서 처리하여, 인간의 신체 구조를 정밀하게 보여주는 이미지를 만든다.

4. 진동 분석 및 고장 진단

기계 장치의 진동 분석: Fourier Transform을 사용하여 회전하는 기계 장치나 구조물의 진동 신호를 분석할 수 있

구조물 안정성 평가: 건물이나 교량과 같은 구조물에서 지진파나 바람에 의한 진동을 분석하여 구조의 안전성을 평가

5. 데이터 압축

JPEG 이미지 압축: Fourier Transform의 일종인 이산 코사인 변환(DCT)은 JPEG 이미지 압축의 핵심 요소입니다. 이미지를 주파수 성분으로 변환한 후, 불필요한 고주파 성분을 제거하여 데이터를 효율적으로 압축한다.

오디오 및 비디오 압축: 오디오 신호는 MP3와 같은 포맷으로 압축될 때 Fourier Transform을 사용하여 신호를 주파수 도메인에서 처리합니다. 이를 통해 불필요한 주파수 성분을 제거하고, 데이터를 손실 압축한다.

6. 날씨 및 기후 데이터 분석

Fourier Transform은 기후 변화 분석과 같은 장기적인 데이터에서 주기적인 패턴을 찾는 데 유용

7. 천문학 활용

<https://github.com/freejyb/Deep-learning>
딥러닝(수학1)

* Fast Fourier Transform (FFT)

- Fast Fourier Transform (FFT)은 시계열 데이터를 구성하는 주파수 성분을 빠르게 계산하는 알고리즘
- 주기적 패턴이나 주파수 기반 특성을 분석할 때 필수적인 도구
- 기본적으로 이산 푸리에 변환(DFT, Discrete Fourier Transform)을 효율적으로 계산하는 방법

Discrete Fourier Transform (DFT)의 수식

$$X_k = \sum_{n=0}^{N-1} x_n \cdot e^{-2\pi i k n / N}$$

x_n : 시간 영역의 입력 신호

$e^{-2\pi i k n / N}$: 복소수 형태의 회전 인자: (삼각함수 기반 파형)

N: 샘플 수

X_k : 주파수 성분 (k번째 성분)

→ FFT는 이 계산을 빠르게 처리하는 방식이다.

입력을 짝수/홀수 인덱스로 분할

xn 을 짝수와 홀수 성분으로 나눔

$$X_k = \sum_{n=0}^{N/2-1} x_{2n} e^{-2\pi i \frac{2nk}{N}} + \sum_{n=0}^{N/2-1} x_{2n+1} e^{-2\pi i \frac{(2n+1)k}{N}}$$

작은 DFT를 재귀적으로 반복 → 큰 문제를 빠르게 해결

Fast Fourier Transform (FFT)의 정의

FFT는 Discrete Fourier Transform (DFT)를 계산하는 빠른 알고리즘이다.

- DFT 계산 복잡도: $O(N^2)$

- FFT 계산 복잡도: $O(N \log N)$ → FFT는 DFT를 $O(N \log N)$ 로 줄임

→ 즉, 계산 속도가 획기적으로 빠름 (대규모 신호 분석 가능)

→ 주로 사용되는 FFT 알고리즘은 Cooley-Tukey 방식입니다 (입력 크기가 2^n 일 때 최적화됨).

* Fast Fourier Transform (FFT)

FFT 결과 해석 :

: FFT를 수행하면 다음과 같은 정보들을 얻을 수 있다

항목	의미
주파수(f)	주기성(얼마나 자주 반복되는가)
진폭(A)	해당 주파수의 강도
위상(ϕ)	신호의 시작 위치, 변환된 위상 정보

→ 이를 통해 데이터에 숨어 있는 주기적 구조를 정량적으로 분석할 수 있다.

FFT 활용 예시

분야	활용 내용
시계열 분석	주기 탐지 (전력 수요, 날씨, 생체신호 등)
오디오 처리	음성 주파수 분해 (음성 인식, 음악 분석 등)
이미지 처리	고주파/저주파 필터링
딥러닝	TimesNet, Autoformer 등에서 주기성 추출용 사전 처리 단계로 활용

시계열 분석에서의 FFT

왜 중요한가?

현실의 많은 데이터는 숨겨진 주기성을 가짐 (예: 하루 단위, 주 단위, 계절 등)

어떻게 쓰나?

FFT로 주기 성분 추출 → 모델 입력으로 사용하거나 데이터 전처리에 반영

대표적 예

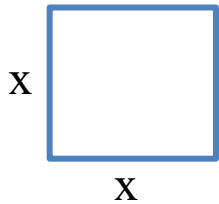
TimesNet 모델은 FFT로 Top-K 주기를 추출하여 시계열을 2D 텐서 구조로 재편성함

differential

미분 (differential) 개념 잡기

미분(微分)

$$S = \int dx$$



$$S = x^2$$

S를 x로 미분(d)

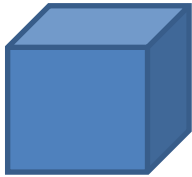


$$d(x^2) = 2x dx$$

$$\frac{d(x^2)}{dx} = 2x$$

미분 전 원래 차원

x의 변화



$$S = x^3$$



$$d(x^3) = 3x^2 dx$$

$$\frac{d(x^3)}{dx} = 3x^2$$

$$S = x^n$$



$$d(x^n) = nx^{n-1} dx$$

$$\frac{d(x^n)}{dx} = nx^{n-1}$$

미분 differential

미분 결과 (Derivative)

미분 differential

1. 기하학적 관점: 접선의 기울기

- 미분은 함수 $y=f(x)$ 의 특정 점에서의 접선의 기울기를 구하는 과정
- 함수가 특정 점에서 얼마나 빠르게 변하는지를 나타낸다.
- 수학적으로는 극한을 통해 정의

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

여기서 $f'(x)$ 의 미분계수로, 변화율 또는 기울기를 나타낸다.

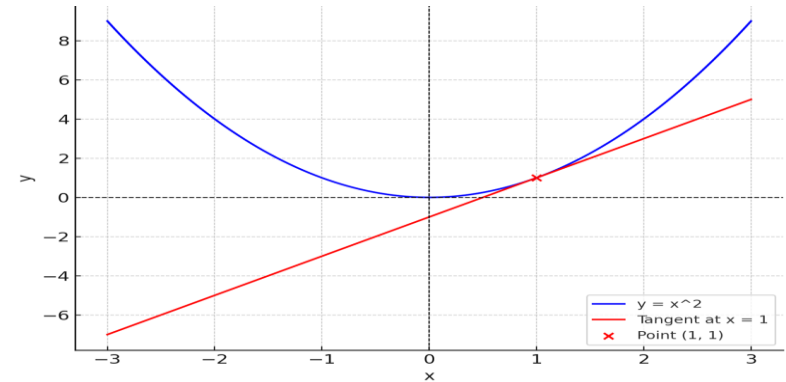
2. 물리적 관점: 순간 변화율

- 미분은 시간, 거리, 온도 등 물리적 변수의 순간 변화율을 나타낸다.
- 예: 물체의 위치를 나타내는 함수 $s(t)$ 가 주어질 때, 미분 $s'(t)$ 는 물체의 순간 속도를 나타낸다.
- 위치의 변화율 $\rightarrow$ 속도
- 속도의 변화율 $\rightarrow$ 가속도

미분 : 함수 $\rightarrow$ (구간) 평균변화율 $\rightarrow$ (극한) 순간 변화율(instantaneous rate of change), 미분 계수(differential coefficient) $\rightarrow$ (일반화) 도함수

3. 응용 관점: 최적화와 변화

- 최적화 문제: 미분은 함수의 극대값과 극소값(최댓값, 최솟값)을 찾는 데 사용
- 예: $f'(x)=0$ 인 점에서 함수의 변화가 없는 정지점을 분석.
- 변화의 해석: 함수가 증가하는지, 감소하는지, 또는 변화가 없는지를 파악한다.
 $f'(x) > 0$ (증가), $f'(x) < 0$ (감소)



파란색 $S = x^2$

빨간색 : $\frac{d(x^2)}{dx} = 2x$

미분 differential



Newton

'움직이는 물체의 순간 속도
위치를 한 번 미분하면 속도,
두 번 미분하면 가속도'와 같은 이야기

Leibniz

기하적 Geometric 관점 : 곡선에서는 아주 작은 단위를 주어 접선의 기울기

$$y = f(x) \rightarrow \frac{dy}{dx} = \frac{df(x)}{dx} = \frac{d}{dx} f(x)$$

$$\frac{\Delta y}{\Delta x} \approx \frac{dy}{dx}$$

분수가 아니다.
 dx : x 로 미분하라.(주체)
 dy : 미분할 대상

미분 Notation

- **Leibniz Notation** : $\frac{dy}{dx}, \frac{\partial y}{\partial x}, \frac{d^2y}{dx^2}$
→ 물리학이나 공학에서 많이 사용되며, 특히 비율과 변화량을 나타낼 때 유용
- **Lagrange Notation** : $f'(x), f''(x)$
→ 수학적 분석에서는 라그랑주 표기법이 더 간결하고 편리하게 사용
- Euler Notation : 미분 연산자 D 를 사용 Dy
- Newton Notation : $\dot{y}$
→ 시간에 따른 변화율을 나타낼 때 주로 사용

$y = f(x)$
 $y = x^2 + 1$ 미분하면
→ 도함수 $y = 2x$
2차함수 → 1차함수

벡터 미적분학, 행렬 미분학, 미분 방정식에서는 행렬 연산이 필수적입니다.
특히 기계 학습, 물리학, 공학 분야에서 이들의 결합이 중요한 역할

미분 differential

$$y = f(x) \rightarrow y' = f'(x)$$

$$\Delta y = f(x + \Delta x) - f(x)$$

$$\Delta y = \Delta Y + \varepsilon \Delta x$$

$$\Delta y = f'(x)\Delta x + \varepsilon \Delta x$$

$$\lim_{\Delta x \rightarrow 0} \varepsilon$$

$$\rightarrow 0$$

$$\rightarrow \Delta y = \Delta Y$$

$$\Delta x \rightarrow 0$$

$$\Delta x \rightarrow dx$$

$$\Delta Y = dy$$

$$\lim_{\Delta x \rightarrow 0} \frac{f(x + \Delta x) - f(x)}{\Delta x} = f'(x) = \frac{dy}{dx}$$

$$dy = f'(x)dx$$

$$dy = \frac{dy}{dx} dx$$

3차원 $z = f(x, y)$

$$\Delta z = f(x + \Delta x, y + \Delta y) - f(x, y)$$

$$\Delta z = f(x + \Delta x, y + \Delta y) - f(x, y + \Delta y) + f(x, y + \Delta y) - f(x, y)$$

$$\Delta z = \frac{\partial f}{\partial x} \Delta x + \frac{\partial f}{\partial y} \Delta y$$

$$\text{만일 } \Delta x \rightarrow 0, \Delta y \rightarrow 0$$

$$dz = \frac{\partial f}{\partial x} dx + \frac{\partial f}{\partial y} dy$$

$$dz = \frac{\partial z}{\partial x} dx + \frac{\partial z}{\partial y} dy$$

* 기억 할 미분 성질

1. $f(x) = x^n$ 경우

$$\cdot \frac{y}{dx} f(x) = nx^{n-1}$$

2. 함수 $f(x), g(x)$, $a = \text{상수}$

1) 선형성

$$\frac{y}{dx} (f(x) + g(x)) = \frac{y}{dx} f(x) + \frac{y}{dx} g(x)$$

$$\frac{y}{dx} (af(x)) = a \frac{y}{dx} (f(x))$$

$$2) \frac{y}{dx} a = 0$$

$$3) \cdot \frac{y}{dx} \sum_{i=0}^n x^n = \sum_{i=0}^n \frac{y}{dx} x^n$$

Partial derivative, Total derivative

편미분 (Partial derivative)

다변수 함수의 특정 변수를 제외한 나머지 변수를 상수로 간주하여 미분하는 것

표기: $\partial \rightarrow$ 역사적으로도 d 를 굴려 쓴 [컬리-디]Curly Dee로 유래

$$f'(x,y,z,\dots) = \frac{\partial f}{\partial x} = \frac{\partial}{\partial x} f(x,y,z,\dots) \quad x \text{에 대한 } z \text{의 편미분하면?}$$

y 에 대한 z 의 편미분하면?

$$f = x^2 + xy + y^2$$

$$\frac{\partial f}{\partial x}(x,y) = 2x + y$$

$$\frac{\partial f}{\partial y}(x,y) = x + 2y$$

전미분 (Total derivative)

- 다변수 함수를 모든 변수에 대해 미분하고 싶은 경우 $\rightarrow$ 전미분 사용
- 전미분은 각각의 변수에 대한 편미분을 구한 뒤 모두 합한것. !!!
- 이때 $dx, dy, dz, \dots$ 는 증분, 즉 변화량

$$df(x,y) = \frac{\partial f}{\partial x} dx + \frac{\partial f}{\partial y} dy$$

$$df(x,y,z,\dots) = \frac{\partial f}{\partial x} dx + \frac{\partial f}{\partial y} dy + \frac{\partial f}{\partial z} dz + \dots$$

Composite function & Derivative

합성함수와 미분

$$f(x) = x^2 + 1$$

$$g(x) = x + 2$$

$$\text{합성함수 : } f(g(x)) = (x + 2)^2 + 1$$

합성함수 미분 $y=f(u)$, $u=g(x)$

$$\frac{dy}{dx} = \frac{dy}{du} \times \frac{du}{dx}$$

$$\frac{dy}{du} = \frac{d}{du}f(u) = \frac{d}{du}(1 + u^2) = 2u$$

$$\frac{du}{dx} = \frac{d}{dx}g(x) = \frac{dy}{du} = \frac{d}{dx}(x+2) = 1$$

$$\frac{dy}{dx} = \frac{dy}{du} \times \frac{du}{dx} = 2u \times 1 = 2u = 2(g(x)) = 2(x + 2) = 2x + 4$$

연쇄 법칙(Chain Rule)
시각적 해석

- 내부 함수 $u=g(x)$ 는 x -축에서 u -축으로의 매핑을 나타낸다.
- 외부 함수 $y=f(u)$ 는 u -축에서 y -축으로의 매핑을 나타낸다.
- 연쇄 법칙은 이 두 매핑의 변화율을 결합하여 x 에서 y 로의 변화율을 계산한다.

Chain rule

다변수 함수일 경우 편미분과 행렬을 사용한 체인 룰이 자주 사용

역전파(Backpropagation)

1. y 가 u 의 함수이고, u 가 x 의 함수라면
 $y=f(u)$, $u=g(x)$

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

2. $z=f(x,y)$ 가 있고, x 와 y 가 t 의 함수라면
 $z=f(x,y)$, $x=g(t)$, $y=h(t)$

$$\frac{dz}{dt} = \frac{\partial z}{\partial x} \cdot \frac{dx}{dt} + \frac{\partial z}{\partial y} \cdot \frac{dy}{dt}$$

Natural Logarithm (ln) & Derivative

자연로그(natural logarithm)

$$\log_e x = \ln x$$

$$e = \lim_{n \rightarrow \infty} \left(1 + \frac{1}{n}\right)^n$$

$$\lim_{\Delta x \rightarrow 0} \frac{f(x+\Delta x) - f(x)}{\Delta x} = f'(x) = \frac{dy}{dx}$$

지수함수 a^x 를 미분하는 경우

$$\begin{aligned} & \lim_{\Delta x \rightarrow 0} \frac{a^{x+\Delta x} - a^x}{\Delta x} \\ & \lim_{h \rightarrow 0} \frac{a^{x+h} - a^x}{h} \\ & = \lim_{\Delta x \rightarrow 0} a^x \frac{(a^h - 1)}{h} \rightarrow a^x \lim_{\Delta x \rightarrow 0} \frac{(a^h - 1)}{h} \\ & = a^x \log_e a = a^x \ln a \end{aligned}$$

$$y = a^x \rightarrow y' = a^x \ln a$$

$$y = e^x, y' = e^x$$

극한값을 구하는 과정에서 필요!!!

$$\begin{aligned} & \lim_{\Delta x \rightarrow 0} \frac{(a^h - 1)}{h} \\ & = \lim_{x \rightarrow 0} \frac{x}{\log_a(1+x)} \\ & = \lim_{x \rightarrow 0} \frac{1}{\frac{1}{x} \log_a(1+x)} \\ & = \lim_{x \rightarrow 0} \frac{1}{\log_a(1+x)^{\frac{1}{x}}} \\ & = \frac{1}{\log_a e} = \log_e a = \ln a \end{aligned}$$

(분자) $a^h - 1 = x$

$a^h = x + 1$

(분모) $h =$

$\log_a(1+x)$

$h \rightarrow 0$ 일 때, $x \rightarrow 0$

Derivative

미분의 활용 사례

(1) 경사하강법 (Gradient Descent)

목적: 모델이 학습하는 과정에서 손실 함수(loss function)를 최소화하기 위해 사용된다.

원리: 손실 함수 $L(\theta)$ 를 미분하여 기울기 $\partial L / \partial \theta$ 를 계산한다.

기울기를 따라 파라미터 θ 를 업데이트:

(2) 활성화 함수의 기울기 계산

역전파(backpropagation) 과정에서 활성화 함수(예: ReLU, sigmoid)의 미분값이 사용된다.

활성화 함수의 변화율은 네트워크의 출력에 대한 학습 신호를 조절하는 데 필수적이다.

$$\theta \leftarrow \theta - \alpha \frac{\partial L}{\partial \theta}$$

편미분의 활용 사례

(1) 다변수 손실 함수 최적화

- 딥러닝 모델은 수많은 가중치 $w_1, w_2, \dots, w_n$ 를 가지고 있다.
- 편미분은 각 가중치에 대해 손실 함수의 변화율을 계산:
- 모든 가중치에 대한 변화율을 계산하여 네트워크를 업데이트한다.

(2) 합성곱 신경망(CNN)

- CNN에서 필터가 학습될 때 각 필터의 파라미터에 대해 손실 함수의 편미분을 계산하여 업데이트.

(3) 순환 신경망(RNN)

- 시간 축에 따른 손실 함수의 편미분을 통해 시퀀스 데이터(예: 텍스트, 음성)의 학습에 활용.

Taylor series, Taylor expansion

테일러 급수(Taylor series) 또는 테일러 전개(Taylor expansion)

미적분학에서 **테일러 급수** (Taylor series)는 **도함수들의 한 점에서의 값으로 계산된 항의 무한합으로 해석함수를 나타내는 방법**
해석 함수(analytic function)란 국소적으로(locally) 수렴하는 **멱급수**로 나타낼 수 있는 함수-특정한 값에서 테일러 급수를 구할 때 원래함수와 **근사치**를 보이는 함수

- 테일러 급수는 **수학에서 어떤 함수가 특정 점에서의 값을 근사하기 위해 사용하는 급수 전개 방식이다.** (아래 예시 참조)
- 주로 **함수가 미분 가능한 경우**, 이 함수를 **다항식의 형태로 근사할 수 있게 해 준다.**
- 테일러 급수는 주어진 점에서의 함수 값과 그 점에서의 미분 계수를 이용하여 점점 더 정밀하게 근사한다.

테일러 급수의 정의

어떤 함수 $f(x)$ 가 무한히 미분 가능하고 특정점 a 에서의 모든 도함수가 존재할 때, 테일러 급수는 다음과 같이 정의 (다음장 참조)

만약 $x=a$ 이면 $f(x) = f(a)$, 여기서 a 는 중심(center)

$$f(x) = f(a) + f'(a)(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \frac{f'''(a)}{3!}(x-a)^3 + \dots$$

$$y = f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!} (x-a)^n$$

함수 $f(x) = x^3 - 3x^2 + 2x$

예시) 테일러 급수를 사용해서 3차 함수 $f(x) = x^3 - 3x^2 + 2x$ 인 경우 이를 근사하게 하는 과정
 이 함수를 특정 점 a 에서 테일러 급수를 사용하여 근사할 수 있다. 예를 들어, $a=1$ 에서 이 함수를 테일러 급수로 근사해보면

1. 테일러 급수 전개를 위해 $f(x)$ 와 2. 각 도함수의 값을 $x=1$ 에서 계산
 그 도함수를 계산

1. $f(x) = x^3 - 3x^2 + 2x$

2. 1차 도함수 $f'(x) = 3x^2 - 6x + 2$

3. 2차 도함수 $f''(x) = 6x - 6$

4. 3차 도함수 $f'''(x) = 6$

1. $f(1) = 1^3 - 3 \cdot 1^2 + 2 \cdot 1 = 0$

2. $f'(1) = 3 \cdot 1^2 - 6 \cdot 1 + 2 = -1$

3. $f''(1) = 6 \cdot 1 - 6 = 0$

4. $f'''(1) = 6$

3. 테일러 급수 전개 (3차까지)

$$f(x) \approx f(1) + f'(1)(x-1) + \frac{f''(1)}{2!}(x-1)^2 + \frac{f'''(1)}{3!}(x-1)^3$$

정리하면 이 함수는 $x=1$ 주변에서 다음과 같은 형태로 근사하게 정리 가능하다.

$$f(x) \approx (x-1)^3 - (x-1)$$

테일러 근사는 3차 함수의 값을 특정 점 주변에서 빠르게 근사하고자 할 때 유용하다.

복잡한 3차 방정식을 풀지 않고도, 근사식을 통해 $x=1$ 근처에서 함수 값을 빠르게 예측할 수 있다.

*멱급수 (power series) & Taylor Series

멱급수와 테일러 급수

멱급수 (power series)

- 급수: 수열을 합한 것. 즉, 수열의 합 → 유한급수, 무한급수
- 멱급수: 멱수열의 합. 멱: 반복해서 곱해지는 것,

멱급수(power series)의 일반적인 형태

$$\sum_{n=0}^{\infty} a_n x^n = a_0 + a_1 x + a_2 x^2 + a_3 x^3 + \dots$$

멱급수는 흔히 테일러 급수(Taylor Series)의 형태로 나타낼 수 있다.
테일러 급수는 함수 $f(x)$ 가 $x=a$ 에서 다음과 같이 멱급수로 표현

$$y = f(x) = \sum_{n=0}^{\infty} c_n (x - a)^n$$

$$f(x) = C_0 + C_1(x - a)^1 + C_2(x - a)^2 + C_3(x - a)^3 + \dots$$

$x = a$ (중심 center)

$$x = a \text{ 이면, } f(a) = C_0$$

1차 미분 하면

$$f'(x) = C_1 + 2C_2(x - a)^1 + 3C_3(x - a)^2 + \dots$$

$$x = a \text{ 이면, } f'(a) = C_1$$

2차 미분 하면

$$f''(x) = 1 \cdot 2C_2 + 2 \cdot 3C_3(x - a)^1 + \dots$$

$$x = a \text{ 이면, } f''(a) = 1 \cdot 2C_2 \rightarrow C_2 = \frac{f''(a)}{2!} \quad \text{일반화하면 } C_n = \frac{f^n(a)}{n!}$$

$$y = f(x) = \sum_{n=0}^{\infty} \frac{f^n(a)}{n!} (x - a)^n$$

Taylor series, Taylor expansion

테일러 급수(Taylor series) 또는 테일러 전개(Taylor expansion)

테일러 정리(Taylor's theorem)

테일러 급수의 응용 예시

테일러 급수는 복잡한 함수를 단순한 다항함수 형식으로 근사하는 데 매우 유용하게 활용된다.

맥클로린 급수 (Maclaurin series)

특히 a=0인 경우를 맥클로린 급수라고 부르며, 이는 테일러 급수의 특별한 경우이다.
맥클로린 급수를 사용하면 복잡한 함수를 단순한 다항식으로 근사하여 계산을 용이하게 할 수 있다.

$$f(x) = f(0) + f'(0)x + \frac{f''(0)}{2!}x^2 + \frac{f'''(0)}{3!}x^3 + \dots$$

1. 지수함수 e^x

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots$$

2. 사인함수 $\sin(x)$:

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

3. 코사인 함수 $\cos(x)$

$$\cos(x) = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots$$

맥클로린 급수 (Maclaurin series)

- X=0 인 경우 $\sin(0) = 0$
- X=1 인 경우 $\sin(1) = 1 - \frac{1}{3!} + \frac{1}{5!} - \frac{1}{7!} + \dots \approx 0.84147$

Taylor series, Taylor expansion

금융 시장 분석에서의 테일러 급수 활용

- **옵션 가격 결정:** 블랙-숄즈 모델과 같은 옵션 가격 결정 모델에서는 테일러 급수를 통해 복잡한 확률 함수나 가격 함수를 근사시킨다. 이 근사는 옵션의 "그릭스(Greeks)" 계산에 활용되며, 이를 통해 옵션의 민감도를 분석하고 헤지 전략을 수립할 수 있다.
- 위험 관리에서의 **포트폴리오 민감도 분석:** 포트폴리오의 가치 변화에 따른 민감도 분석(감마, 델타 등)에서 테일러 급수를 사용하여 포트폴리오 가치를 근사한다. 이러한 분석을 통해 포트폴리오 가치가 기초 자산의 가격 변동에 어떻게 반응하는지를 알 수 있다.
- 금리 모델링: 금리 변동에 따른 채권 가격을 근사하거나 금리 옵션을 평가할 때 테일러 급수를 사용한다. 금리의 시간에 따른 변동성을 반영하기 위해 금리 함수의 테일러 전개를 통해 리스크 요소를 정밀하게 추정한다.
- **리스크 분석과 VaR (Value at Risk):** 테일러 급수는 VaR 계산 시 특정 자산이나 포트폴리오의 가치 변동을 근사하여 리스크를 추정하는 데도 사용된다. 이는 자산 가격의 작은 변화에 대한 포트폴리오 민감도를 평가하여 손실 가능성을 계산하는 데 유용하다.

머신러닝에서의 테일러 급수 활용 테일러 급수는 머신러닝 모델을 최적화하거나 모델을 해석할 때 유용

- **로지스틱 회귀와 손실 함수 근사:** 로지스틱 회귀 모델에서 손실 함수를 최적화할 때, 테일러 급수를 사용하여 손실 함수를 근사해 쉽게 해석하거나 최적화할 수 있다.
- (예시) 손실 함수가 복잡한 비선형일 경우 테일러 급수를 사용해 2차 또는 3차 다항식으로 근사함으로써 최적화 과정을 간소화할 수 있다.
- **그래디언트 기반 최적화:** 테일러 급수는 함수의 국소적인 기울기를 나타내므로, 그래디언트 기반 최적화(예: 경사 하강법)에서 사용된다. 최적화 문제에서 함수의 2차 도함수를 포함한 2차 근사 테일러 급수를 통해 헤스를 계산하여 뉴턴 방법 등의 최적화 알고리즘에 적용할 수 있다.
- **딥러닝에서의 활성화 함수 근사:** 복잡한 활성화 함수(예: **하이퍼볼릭 탄젠트** 등)를 테일러 급수로 근사하여 계산 효율을 높이는 방법도 존재한다. 이는 주로 모델 경량화나 빠른 예측이 필요한 경우에 유용하게 쓰인다.

Linear Algebra : Scalar

선형대수학(Linear Algebra)

- 새로운 대상과 새로운 연산을 활용 :
- 벡터 공간, 벡터, 선형 변환, 행렬, 연립 선형 방정식 등을 연구하는 대수학 한 분야

스칼라(Scalar)

: 길이, 넓이, 질량, 온도 - 크기만 주어지만 완전히 표시되는 양

- : 크기 값을 가지는 양

$$x \in \mathbb{R}$$

→ 하나의 숫자만으로 이루어진 데이터

→ 0차원 (0D) 단일 값, 위치가 없는 단순한 숫자

→ x와 같이 소문자로 표기하며 실수(Real number)인 숫자 중의 하나

스칼라 a는 행렬 [a]로 표현될 수 있다.

→ 스칼라는 0차원이며 단일 값으로 표현되지만,

→ [a]는 스칼라 a를 포함하는 1차원 벡터를 나타내어 벡터와 스칼라 사이의 연산을 수행할 때 사용된다.

스칼라(scalar) 연산법칙

스칼라(scalar) : a , b

- 덧셈의 교환법칙 : $a + b = b + a$
- 덧셈의 결합법칙 : $(a+b) + c = a + (b+c)$
- 곱셈의 교환법칙 : $ab = ba$
- 곱셈의 결합법칙 : $a(bc) = (ab)c$
- 분배법칙 : $c(a+b) = ac + ab$

◆ 차원 (Dimension) 개념

- 차원(Dimension)은 데이터나 벡터의 복잡성과 형태를 설명하는 개념
- 차원은 데이터에서 독립적인 특징이나 축(axis)의 수를 나타내며, 일반적으로 공간에서의 위치나 데이터를 나타내는 데 사용

차원의 직관적 이해

- 0차원: 점 (위치만), 정지상태
- 1차원: 선 (길이만), 한쪽 방향
- 2차원: 평면 (길이와 너비), 두 방향
- 3차원: 공간 (길이, 너비, 높이), 세 방향

Linear Algebra : Vector

벡터(Vector)의 특징

- 크기(Magnitude) : 벡터의 길이로, 방향과 상관없이 벡터가 얼마나 큰지를 나타낸다.
 - 방향(Direction) : 벡터가 공간에서 가리키는 방향이다.
- 수학적으로는 벡터는 순서쌍이다. 숫자들을 한 줄로 늘어 놓은 것에 불과하다.

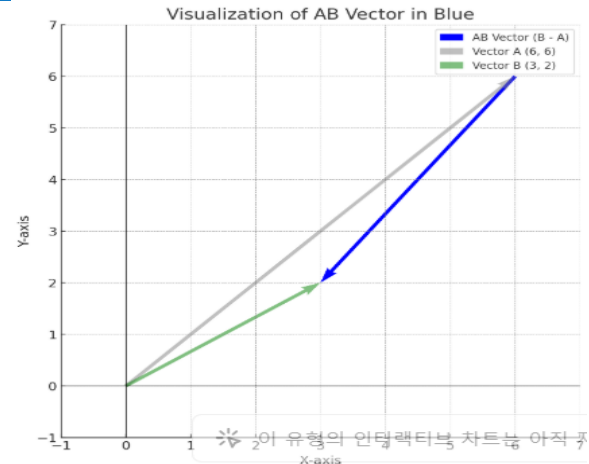
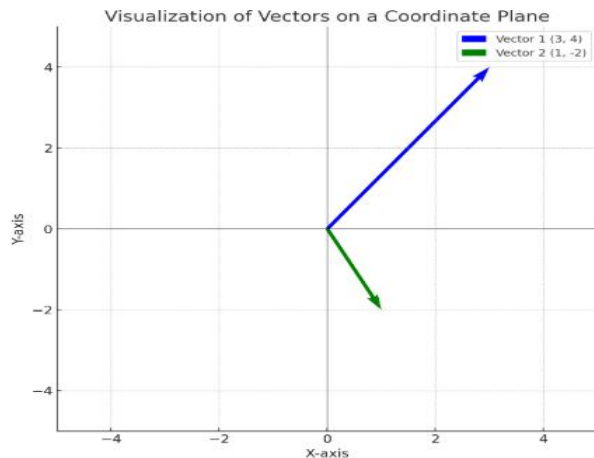
벡터의 표현 방법

- 기하학적 표현: 공간에서 화살표로 나타낸다.
 - 예: 화살표의 길이는 크기를 나타내고, 화살표의 방향은 방향을 나타낸다.
 - 좌표 표현: 벡터는 좌표로 표현된다.
 - 예: 2차원 공간에서 벡터는 $v=(x,y)$ 형태로 나타내고, 3차원에서는 $v=(x,y,z)$ 로 나타낸다.
 - 벡터는 공간에서 한 점을 나타 남.
 - 벡터는 원점에서부터 상대적 위치를 표현 : 벡터의 동일성 이해
- 1 차원 배열(array)

표현

Vector 1 : $\vec{V}_1 = (3, 4) \rightarrow \begin{pmatrix} 3 \\ 4 \end{pmatrix}$

Vector 1 : $\vec{V}_1 = (3, 4)$
 $\vec{A} = (6, 6)$, $\vec{B} = (3, 2)$
 $\vec{AB} = (3, 4)$
 $\vec{AB}$ 와 $\vec{V}_1$ 는 동일하다.



- Vector 1 (3, 4): 파란색 벡터로, 원점에서 (3, 4) 방향으로 가리킨다.
- Vector 2 (1, -2): 녹색 벡터로, 원점에서 (1, -2) 방향으로 가리킨다.

Linear Algebra : Vector

벡터(Vector)의 추상성

계산 사례

$$\begin{aligned} x + y &= 5 \\ 2x - 4y &= 2 \end{aligned}$$



$$\begin{bmatrix} 1 & 1 \\ 2 & -4 \end{bmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{bmatrix} 5 \\ 2 \end{bmatrix}$$

추상화

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 &= b_1 \\ a_{21}x_1 + a_{22}x_2 + a_{23}x_3 &= b_2 \\ a_{31}x_1 + a_{32}x_2 + a_{33}x_3 &= b_3 \end{aligned}$$



$$\begin{bmatrix} \vdots & \ddots & \vdots \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

Linear Algebra : Vector

벡터(Vector)의 추상성

- 벡터의 추상성(Abtractness of Vectors)은 수학적 개념에서 벡터가 단순히 물리적인 방향과 크기를 가지는 화살표로만 이해되는 것이 아니라, 더 일반화된 형태로 해석될 수 있다는 아이디어를 말한다.
- 이 추상성 덕분에 벡터는 수학, 물리학, 공학 등 다양한 분야에서 매우 유용하게 사용된다.

1. 벡터의 물리적 직관

- 가장 기본적으로 벡터는 물리학에서 방향과 크기를 가지는 양(예: 힘, 속도, 가속도)으로 시작한다.
- 2차원 또는 3차원 공간에서 화살표로 표현되며, 크기(길이)와 방향으로 정의된다.

2. 벡터의 수학적 일반화

벡터는 수학적으로 n 차원 공간에서 하나의 점으로 이해할 수 있다.

예를 들어:

- 2차원 벡터: $v=(v_1, v_2)$
- 3차원 벡터: $v=(v_1, v_2, v_3)$
- n 차원 벡터: $v=(v_1, v_2, \dots, v_n)$

벡터 공간은 단순히 숫자들로 이루어진 집합이 아니라, 선형 연산(덧셈, 스칼라 곱)이 정의되고 닫혀 있는 구조이다.

3. 벡터 공간의 추상적 성질

벡터의 추상성은 벡터 공간의 공리에서 드러납니다. 벡터 공간은 다음 조건들을 만족하는 집합으로 정의된다.

- 벡터 덧셈: 두 벡터를 더해도 여전히 같은 공간의 벡터.
- 스칼라 곱셈: 스칼라(숫자)를 벡터에 곱하면 여전히 같은 공간의 벡터.
- 영벡터의 존재: 0라는 특별한 벡터가 존재하며, 어떤 벡터와 더해도 그 벡터 자체를 반환.
- 벡터 덧셈의 교환 법칙 및 결합 법칙.

4. 추상성과 선형대수

- 선형 변환: 벡터 공간 간의 변환.
- 기저와 차원: 벡터 공간의 구조를 정의하는 기본 벡터 집합.
- 내적 공간: 벡터 간의 "각도"와 "길이"를 정의할 수 있는 공간.

Linear Algebra : Vector

벡터(Vector)의 추상성 사례

사례 1: 데이터 분석에서의 벡터

문제: 고객 데이터를 분석하는 경우

각 고객의 정보는 나이, 연봉, 소비 점수로 구성

벡터 표현: 고객 데이터를 (나이, 연봉, 소비 점수)로 나타내면, 각 고객은 3차원 공간의 벡터로 표현된다.

예: 고객 A = (35, 5000, 75), 고객 B = (27, 3000, 50)

벡터 간 연산: 두 고객 간의 유사성을 측정하기 위해 벡터 간의 거리를 계산하거나(유클리드 거리), 군집 분석을 통해 데이터를 분류한다.

벡터의 추상성 덕분에 고객 데이터와 같은 "숫자 목록"이 물리적인 공간과는 무관하지만, 여전히 벡터 공간의 수학적 연산을 적용할 수 있다.

사례 2: 자연어 처리(NLP)

문제: 두 문장의 의미가 얼마나 유사한지 측정하고 싶은 경우

벡터 표현:

각 단어나 문장을 벡터로 변환하는 "단어 임베딩" 기법을 사용한다.

예: "apple" → [0.5, 0.1, 0.3] "orange" → [0.4, 0.2, 0.4]

벡터 공간의 활용: 두 벡터의 코사인 유사도를 계산하여 단어 간 의미적 유사성을 측정한다.

$\cos(\theta) = \frac{u \cdot v}{\|u\| \|v\|}$

이 경우, "apple"과 "orange"는 물리적 위치나 방향을 가지지 않지만, 추상적인 벡터 공간에서 연산 될 수 있다.

벡터의 추상성은 데이터 분석, 컴퓨터 그래픽스, 함수 공간, 자연어 처리, 양자역학 등에서 벡터는 추상적인 구조를 나타내며, 이를 통해 현실 세계의 다양한 문제를 모델링하고 해결할 수 있다.

Linear Algebra : Vector

벡터의 종류

(공간)벡터(vector in space) $\mathbf{x} = (x_1, x_2, x_3)$ 또는 $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}_{3 \times 1}$

x_1, x_2, x_3 를 (공간)벡터 $\mathbf{x}$ 의 성분(component)이라고 함.

n차원 벡터(-dimensional vector)

$$\mathbf{x} = (x_1, x_2, \dots, x_n) = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}_{n \times 1}$$

행 벡터(row vector) 1 by n 행렬

$$(a_1 \ a_2 \ \cdots \ a_n)$$

열 벡터(column vector) n by 1 행렬

$$\begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{pmatrix}$$

$\mathbb{R}^n$

n by 1 벡터 전체로 이루어지는 집합
을 $\mathbb{R}$ 에 이중선을 추가하여 표시

단위벡터(unit vector) 벡터의 힘의 크기가 1인 벡터

$$|\overrightarrow{AB}| = |\vec{a}| = \|\vec{a}\| = 1$$

Linear Algebra : Vector

벡터의 연산

1. 벡터와 벡터의 덧셈

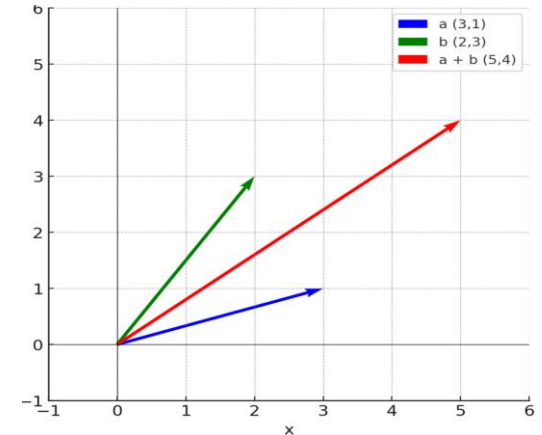
$$\vec{a} = (a_1, a_2), \quad \vec{b} = (b_1, b_2)$$

$$\rightarrow \vec{a} + \vec{b} = (a_1 + b_1, a_2 + b_2)$$

$$\rightarrow (a_1 + b_1, a_2 + b_2 + \dots, a_n + b_n)$$

$$\vec{a} = \begin{pmatrix} 3 \\ 1 \end{pmatrix}, \quad \vec{b} = \begin{pmatrix} 2 \\ 3 \end{pmatrix}$$

$$\vec{a} + \vec{b} = \begin{pmatrix} 5 \\ 4 \end{pmatrix}$$



2. 벡터와 벡터의 뺄셈

$$\vec{a} = (a_1, a_2), \quad \vec{b} = (b_1, b_2)$$

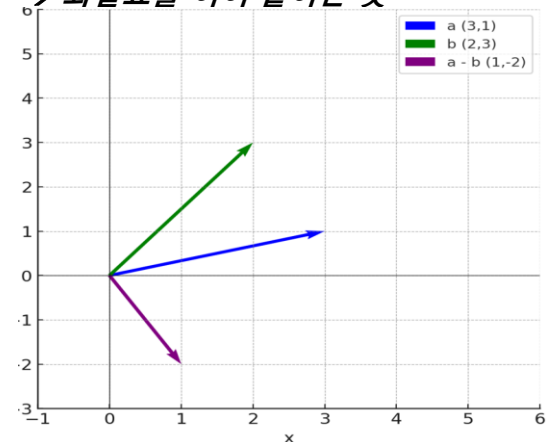
$$\rightarrow \vec{a} - \vec{b} = (a_1 - b_1, a_2 - b_2)$$

$$\rightarrow (a_1 - b_1, a_2 - b_2 + \dots, a_n - b_n)$$

$$\vec{a} = \begin{pmatrix} 3 \\ 1 \end{pmatrix}, \quad \vec{b} = \begin{pmatrix} 2 \\ 3 \end{pmatrix}$$

$$\vec{a} - \vec{b} = \begin{pmatrix} 1 \\ -2 \end{pmatrix}$$

벡터간 덧셈의 기하적 의미
→ 화살표를 이어 붙이는 것



벡터간 뺄셈의 기하적 의미

→ 벡터의 방향을 거꾸로 해서 화살표를 이어 붙이는 것

x = np.array([[1], [3], [5], [7]])

깃허브 참조: 행렬 및 벡터 사칙연산

Linear Algebra : Vector

벡터의 연산

3. 벡터와 벡터의 곱셈 내적內積 inner product

$$\vec{a} = (a_1, a_2), \vec{b} = (b_1, b_2)$$

$$\rightarrow \vec{a} \cdot \vec{b} = a_1 b_1 + a_2 b_2$$

$\rightarrow$ 결과 값은 스칼라로!! (외적 : 벡터로)

= **dot product** = scalar product

벡터를 마치 수치럼 곱하는 개념 (표시 : $\cdot$)

- 대응하는 성분끼리 곱한 다음 모두 더한 값이 내적 !!
- 벡터에는 방향이 있으므로, 방향이 일치하는 만큼만 곱한다.
- 두 벡터의 방향이 같으면, 두 벡터의 크기를 그냥 곱한다.
- 두 벡터가 이루는 각이 90도일 땐, 일치하는 정도가 전혀 없기 때문에 내적의 값은 0

$$\vec{a} = \begin{pmatrix} 3 \\ 1 \end{pmatrix}, \vec{b} = \begin{pmatrix} 2 \\ 3 \end{pmatrix}$$

$$\vec{a} \cdot \vec{b} = 9$$

$$\vec{a} \cdot \vec{b} = \langle \vec{a}, \vec{b} \rangle = \vec{a}^T \vec{b}$$

$$\vec{a} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \vec{b} = \begin{pmatrix} 0 \\ 3 \end{pmatrix}$$

$$\vec{a} \cdot \vec{b} = 0$$

두 열 벡터의 내적을 구하려고 할 경우 둘 중 하나의 벡터를 transpose 시켜 행 벡터의 형태로 바꾼 후,
나머지 벡터와 벡터곱을 시키는 것

$$\vec{a} \cdot \vec{b} = \sum_{i=1}^n a_i b_i = a_1 b_1 + a_2 b_2 + a_3 b_3 + \dots + a_n b_n$$

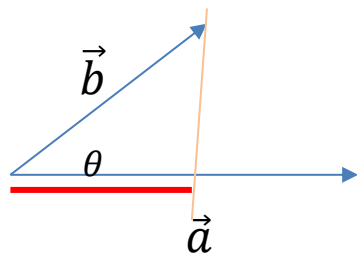
예시) 1x3 행렬(row vector), 3x1 행렬(column vector) $\rightarrow$ 내적 : 1x1행렬

$$(1 \quad -3 \quad 5) \begin{pmatrix} 2 \\ 1 \\ 2 \end{pmatrix} = 2 - 3 + 10 = 9$$

Linear Algebra : Vector

벡터의 연산

3. 벡터와 벡터의 곱셈

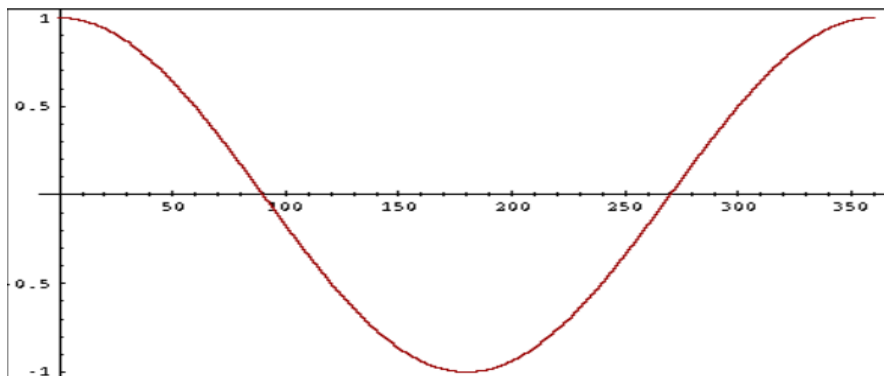


내적 : 두 벡터의 각 성분끼리의 곱의 합산

$$\vec{a} \cdot \vec{b} = |\vec{a}| \times |\vec{b}| \times \cos\theta$$

$$\cos\theta = \frac{\text{밑변}}{\text{빗변}}$$

- 두 벡터의 크기 (Norm)와 두 벡터 사이의 각(θ)의 코사인 값($\cos\theta$)을 곱한 것
- 내적으로 두 벡터 간에 이루는 각도(θ)를 알 수 있다는 것
- 두 벡터의 크기 (Norm)는 항상 양수(+)
- **내적은 한 벡터를 다른 벡터로 projection 시켜서 $|\vec{b}| \times \cos\theta$**
- **동일 방향 길이를 서로 곱한다.**
- **내적이 0 된다는 것은 $\cos\theta=0$ 을 의미하므로 각도(θ)=90도 또는 270도** → 이것은 직각 법선 벡터 : 직선이나 평면의 기울기나 경사각을 표현할 때, 해당 직선이나 평면에 수직인 벡터를 사용하는 데를 **법선 벡터(normal vector)**라고 함..



$$\vec{a} = \begin{pmatrix} 3 \\ 1 \end{pmatrix}, \vec{b} = \begin{pmatrix} 2 \\ 3 \end{pmatrix}$$

$$\vec{a} \cdot \vec{b} = 9$$

$$|\vec{a}| = \sqrt{3^2 + 1^2} = \sqrt{10}, \quad |\vec{b}| = \sqrt{2^2 + 3^2} = \sqrt{13}$$

$$9 = \sqrt{10} \sqrt{13} \cos\theta$$

https://www.chip1stop.com/sp/knowledge/016_trigonometric-table_ko

Linear Algebra : Vector

벡터의 연산

3. 벡터와 벡터의 곱셈 내적(dot product) 활용

두 벡터의 크기 (Norm)와 두 벡터 사이의 각의 코사인 값($\cos \theta$)을 곱한 것

→ 내적으로 두 벡터 간에 이루는 각도를 알 수 있다는 것

→ 두 벡터를 내적을 하고 나면 스칼라 값 : **두 벡터의 유사도 정도를 알 수 있다**. → 유사도가 높을수록 큰값을 가진다.

• **내적의 값이 양수(+)**: 두 벡터는 **비슷한 방향**을 가진다.

• **내적의 값이 음수(-)**: 두 벡터는 **반대 방향**을 가진다.

▪ 내적값 = 0 → 각도(θ) = 90도

▪ 내적값 > 0 → 각도(θ) < 90도

▪ 내적값 < 0 → 각도(θ) > 90도

→ 위 관계를 이용하여 **두 벡터간의 위치관계를 파악 가능**

크기가 1인 두 벡터가 있다고 한다면

(1) 두 벡터가 수직이라면 두 벡터가 이루는 **각도가 90도** → **내적값은 0**

(2) 두 벡터가 평행하면서 동일 방향이면 두 벡터가 이루는 각도가 0도 → **내적값은 1**

(3) 두 벡터가 평행하면서 반대 방향이면 두 벡터가 이루는 각도가 180도 인 경우 → **내적값은 -1**

(4) 두 벡터가 이루는 각도가 0도보다 크고 90도보다 작다면 → **내적값은 0 ~ 1사이값**.

→ **두 벡터의 크기가 1이라면, 두 벡터의 방향이 유사할수록 내적값은 1에 가까워 진다.**

두 벡터 간의 유사도(similarity)를 계산하는 경우 사용

: 유사도(similarity) → 두 벡터가 닮은 정도를 정량적으로 나타낸 값

- **두 벡터 사이의 거리를 측정하는데 이용**
- **한 벡터의 norm을 구하는 경우 활용**

벡터의 연산

3. 벡터와 벡터의 곱셈

$$\vec{a} \cdot \vec{b} = \|\vec{a}\| \|\vec{b}\| \cos\theta$$

- 벡터 내적 : 벡터의 norm을 구하는 경우 활용

$$\vec{a} = (1 \ 0 \ 1) \quad \vec{b} = \begin{pmatrix} 0 \\ -1 \\ 1 \end{pmatrix}$$

$$\vec{a} \cdot \vec{b} = 0 + 0 + 1 = 1$$

$$\|\vec{a}\| \|\vec{b}\| \cos\theta = \sqrt{2} \times \sqrt{2} \times \cos\theta \rightarrow \cos\theta = 0.5$$

$$\|\vec{a}\| = \sqrt{2}$$

$$\|\vec{b}\| = \sqrt{2}$$

두 벡터를 내적하기 위한 조건

1. 두 벡터의 차원이 같아야 한다.
2. 앞의 벡터가 행 벡터이고 뒤의 벡터가 열 벡터여야 한다.

내적은 교환법칙이 성립, 분배법칙 성립

Linear Algebra : Vector

벡터의 연산

3. 벡터와 벡터의 곱셈

내적(dot product) 활용

- 임의의 벡터의 특정 방향을 가진 성분의 크기를 알아내는데 유용
- 내적은 벡터의 특정 방향, 성분, 투영(정사영)의 크기, 일의 크기, 전류 밀도에 대한 전류의 크기 등을 구할 때 필요

1. 기계 학습 및 데이터 분석

1) 코사인 유사도 (Cosine Similarity)

- 코사인 유사도는 두 벡터 간의 각도를 사용하여 유사도를 측정하는 방법
- 내적을 사용하여 벡터 간의 각도를 계산하고, 이를 통해 두 벡터(데이터 포인트)의 유사성을 평가한다.
- 특히 문서 간의 유사도 분석에서 자주 사용된다.
- → 해석: 유사도가 1에 가까울수록 두 벡터는 비슷한 방향이다.

2) 선형 회귀 (Linear Regression)

내적은 선형 회귀 모델에서 입력 벡터와 가중치 벡터 간의 곱을 계산할 때 사용된다. 모델의 예측값은 입력 벡터와 가중치 벡터의 내적 합으로 계산된다.

벡터 유사도 : cosine Similarity

$$= \cos \theta = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \|\vec{b}\|}$$

2. 컴퓨터 그래픽스: 조명, 음영 계산, 벡터 투영.
3. 물리학: 힘과 이동 벡터를 통한 일 계산.
4. 신호 처리: 신호 유사성 분석, 직교 변환.

벡터의 연산

4. 벡터와 벡터의 곱셈

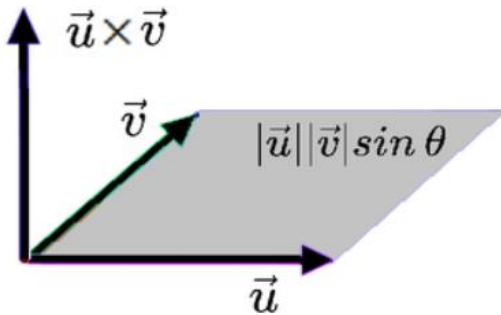
외적 (外積, outer product)

= vector product 또는 cross product

(표시 : $\times$)

→ 외적의 크기는 면적

내적의 결과값은 스칼라, 외적의 결과값은 벡터



두 벡터의 크기와 두 벡터 사이의 각의 사인값($\sin \theta$) 그리고 수직인 벡터의 곱으로 정의

외적은 교환법칙이 성립하지 않는다

Linear Algebra : Vector

Norm

- Norm는 벡터나 행렬의 크기 (또는 길이)를 측정하는 수학적 함수이다.
- 주로 선형대수학과 머신러닝에서 사용되며, 데이터의 크기를 표준화하거나 벡터의 길이를 계산하는 데 활용된다.

하나의 벡터를 이루는 데이터의 개수가 n 개 인 경우 $\rightarrow$ n -dimensional vector

Norm의 정의

- Norm은 $\|x\|$ 로 표기
- 다음과 같은 조건을 만족하는 함수
- ✓ $\|x\| \geq 0$: Norm은 0 이상의 값을 가진다. (0은 벡터가 영벡터일 때만 가능하다.
- ✓ $\|x\|=0 \Leftrightarrow x=0$: Norm이 0이면, 벡터도 영벡터이다.
- ✓ $\|ax\|=|a|\|x\|$ 스칼라 곱에 대해 선형성(축척성)을 가진다.
- ✓ $\|x+y\| \leq \|x\| + \|y\|$: 삼각 부등식이 성립한다.

Norm의 활용

- 머신러닝과 데이터 과학:
 - 1) 데이터 표준화 및 정규화에 사용 : L1 정규화는 희소성을 강조하고, L2 정규화는 일반적인 규제를 위해 사용된다.
 - 2) 최적화 문제: Norm은 손실 함수에서 자주 활용
예시) 손실 함수에 L2 규제를 추가하면 과적합을 방지할 수 있다.
- 컴퓨터 그래픽스: 벡터를 단위벡터로 변환시 L2 Norm을 사용
- 신경망 및 딥러닝: 가중치나 입력값의 크기를 제어하여 안정적인 학습을 도와준다. $\rightarrow$ deep learning 대부분

주요 Norm의 종류

- Norm을 이용하면 두 점 사이의 거리(distance)를 쉽게 계산
- 벡터의 노름은 원점에서부터 벡터까지의 거리

- 1) L_1 Norm (Manhattan Distance, 택시거리) : 각 성분의 변화량의 절대값을 모두 합 한 것 $\rightarrow$ 주로 희소성을 측정할 때 사용
- 2) L_2 Norm (Euclidean Distance) : 벡터의 크기를 계산하는 가장 일반적인 방식이다.
직선 거리(유클리드 거리)를 의미한다. $\rightarrow$ 피타고라스 정리를 이용해 유클리드 거리를 계산
- 3) Frobenius Norm : 행렬의 각 성분의 제곱합의 제곱근이다. 행렬의 크기를 나타내는 데 사용된다.

$$\|x\|_1 = \sum_{i=1}^n |x_i|$$

$$\|x\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$$

$$\|A\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2}$$

벡터의 힘의 크기 계산 $\|\vec{a}\| = \sqrt{a_x^2 + a_y^2}$

* Euclidean Distance

유클리드 거리 (Euclidean Distance)

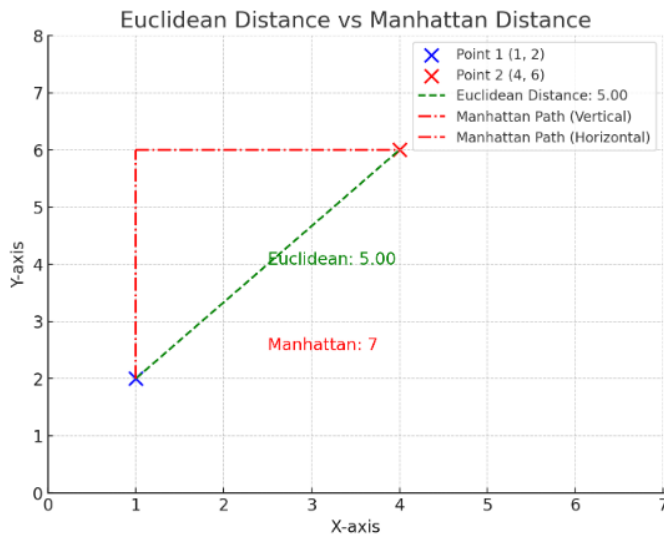
- 유클리드 거리는 두 점 사이의 직선 거리이다. L_2 거리라고 불림
- 벡터 $\mathbf{p}=(p_1,p_2,...,p_n)$ 와 $\mathbf{q}=(q_1,q_2,...,q_n)$ 사이의 유클리드 거리는 다음과 같이 계산된다.

$$d(\mathbf{p}, \mathbf{q}) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$

활용 사례

- K-최근접 이웃 알고리즘 (K-Nearest Neighbors, KNN):**
 - KNN 알고리즘에서는 새로운 데이터 포인트가 주어졌을 때, 가장 가까운 K개의 이웃을 찾는다.
 - 이때 거리를 계산할 때 주로 유클리드 거리를 사용하여 가장 가까운 이웃을 선택한다.
 - 유클리드 거리는 연속형 데이터에서 잘 작동하지만, 스케일링이 다르면 잘못된 결과를 초래할 수 있어 표준화나 정규화가 필요하다.
- 군집화 (Clustering):**
 - K-평균 군집화 (K-Means Clustering)에서 군집의 중심(centroid)과 각 데이터 포인트 간의 거리를 계산할 때 유클리드 거리를 사용한다.
 - 군집화는 데이터를 그룹으로 묶는 작업이며, 유클리드 거리를 사용하면 직관적인 군집 결과를 얻을 수 있다..

자연어처리 유사도 활용: 뒷장 별도



•유클리드 거리 (녹색 점선)

• 두 점 (1,2)와 (4,6) 사이의 직선 거리 → 계산된 거리는 5.00이다.

•맨하탄 거리 (빨간 점선)

• 두 점 사이의 수직 및 수평 거리의 합 → 계산된 거리는 7이다.

* Manhattan Distance

맨하탄 거리 (Manhattan Distance)

맨하탄 거리는 두 점 사이의 **축에 평행한 거리**이다. L_1 거리라고 불림

$$d(\mathbf{p}, \mathbf{q}) = \sum_{i=1}^n |p_i - q_i|$$

활용 사례

1. L_1 정규화 (L1 Regularization):

1. Lasso 회귀 (Lasso Regression)**에서 사용되는 L_1 정규화는 맨하탄 거리를 기반으로 한 패널티를 추가하여 모델의 과적합 (overfitting)을 방지하고, 일부 가중치를 0으로 만든다.
2. 이로 인해 모델이 **특성 선택 (feature selection)** 기능을 갖게 되어, 불필요한 특성을 제거할 수 있다. (과적합 통제)

2. K-최근접 이웃 알고리즘 (KNN):

1. 데이터의 분포가 **직교 축**을 따라 넓게 퍼져 있거나, 데이터가 고차원인 경우 맨하탄 거리가 유클리드 거리보다 더 적합할 수 있다.
2. 특히, 특성이 독립적이고 중요도가 동일한 경우 맨하탄 거리가 잘 작동한다.

* Cosine Similarity

코사인 유사도 (Cosine Similarity)

- 코사인 유사도는 두 벡터 사이의 각도(cosine of the angle)를 사용하여 유사성을 측정한다.
- 두 벡터 **a**와 **b**에 대해 코사인 유사도는 다음과 같이 정의된다:

$$\text{Cosine Similarity} = \cos(\theta) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$$

$\mathbf{a} \cdot \mathbf{b}$ 는 두 벡터의 내적(dot product)

$\|\mathbf{a}\|$ 와 $\|\mathbf{b}\|$ 는 각각 벡터의 크기(노름, norm)

값의 범위

- 코사인 유사도의 값 범위: **-1에서 1 사이**
- 1: 완전히 같은 방향
- 0: 직교(관련 없음)
- 1: 완전히 반대 방향

활용 사례

1. 텍스트 분석 (Text Analysis) 및 자연어 처리 (NLP):

- TF-IDF 벡터화된 텍스트에서 코사인 유사도는 문서 간의 유사도를 측정하는 데 자주 사용

2. 추천 시스템 (Recommendation Systems):

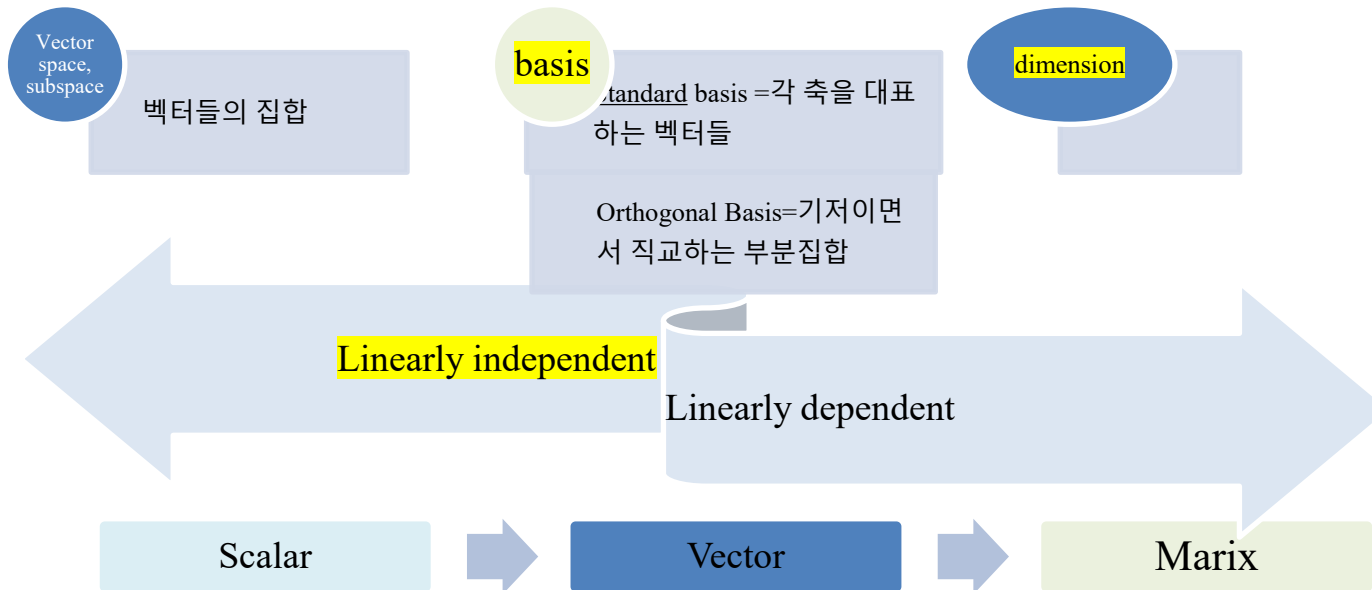
- 사용자 간 또는 아이템 간의 유사도를 측정할 때 코사인 유사도가 사용
- 예를 들어, 사용자 A와 사용자 B의 평가 벡터 간의 코사인 유사도를 계산하여 유사한 취향의 사용자를 찾고, 추천을 생성할 수 있다.

거리/유사도 지표	정의	활용 분야
유클리드 거리	두 점 사이의 직선 거리	KNN, K-평균 군집화
맨하탄 거리	축에 평행한 거리의 합	L1 정규화, Lasso 회귀
코사인 유사도	벡터 간의 각도 기반 유사도	텍스트 분석, 추천 시스템

거리와 유사도 지표는 데이터를 비교하고 분류하며, 패턴을 찾는 데 필수적인 도구이다.

머신러닝 모델에서 데이터를 해석하고 최적화하는 데 중요한 역할을 한다.

선형연립방적식 해 존재성, 유일성	Gauss-Jordan Elimination 최적화 선형계획법	Eigen value Eigen vector
Rank : 행렬에 포함된 선형적으로 독립적인 행(또는 열)의 최대 개수를 말한다		



* linear independence

선형 독립성 (linear independence)

- 벡터들이 선형 독립적이라는 것은, 각 벡터가 다른 벡터들의 선형 결합으로 표현될 수 없음을 의미한다.
- 즉, 주어진 벡터 $v_1, v_2, \dots, v_n$ 에 대해, 아래의 방정식이 성립할 때: $c_1 v_1 + c_2 v_2 + \dots + c_n v_n = 0$
- 모든 계수 $c_1, c_2, \dots, c_n$ 가 0이어야 한다면, 이 벡터들은 선형 독립적이다.

$$R^2 \text{ 두 벡터 } v_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad v_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

$$c_1 \begin{pmatrix} 1 \\ 0 \end{pmatrix} + c_2 \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

$c_1 = 0$ 이고 $c_2 = 0$ 이어야만 성립한다. → 따라서 벡터 v_1 과 v_2 는 선형 독립이다.

$$R^3 \text{의 세 벡터 } v_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \quad v_2 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} \quad v_3 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

$$c_1 \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} + c_2 \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} + c_3 \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

유일하게 $c_1 = c_2 = c_3 = 0$ 이어야만 성립한다. → 따라서 벡터 v_1, v_2, v_3 는 선형 독립이다.

선형 종속적인 경우

벡터 v_1, v_2 중 하나가 다른 벡터의 스칼라 배수인 경우를 말한다.

$$v_1 = \begin{pmatrix} 1 \\ 2 \end{pmatrix} \quad v_2 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}$$

$$c_1 \begin{pmatrix} 1 \\ 2 \end{pmatrix} + c_2 \begin{pmatrix} 2 \\ 4 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

c_1 과 c_2 가 동시에 0일 필요가 없으므로, v_1 과 v_2 는 선형 종속이다.
예를 들어, $c_1 = -2$,와 $c_2 = 1$ 이라면 이 방정식이 성립하기 때문이다.

* basis vector

벡터의 종류

기저 (Basis)

- 수학적 공간을 생성하는 최소의 집합 : 가장 적은 수의 벡터로써, 선형 독립을 이루며, 벡터 공간을 생성 가능
- 벡터공간 내에서 기저 벡터의 집합은, ① 선형 독립이고, ② 전체 공간을 생성(Span) 하게 됨

기저 벡터(basis vector)

- 주어진 벡터공간(벡터 부분공간)을 생성할 수 있는 기저 요소를 이루는 벡터
 - 기저 벡터는 벡터 공간을 정의하는 중요한 요소로, 공간 내에서 위치와 방향을 정의하는 기준 역할을 한다.
 - 기저 벡터는 공간을 "형성"하는 벡터들이라고 할 수 있다.
- 벡터 공간 V 의 기저(basis)는 그 공간의 벡터들을 **선형 결합**으로 표현할 수 있는 **독립적인 벡터들의 집합**이다.
 - 이 기저 벡터들은 다음과 같은 성질을 가져야 한다.

1) 선형 독립성(linearly independent):

• 기저벡터는 **선형 독립**이어야 한다. 즉, 기저의 각 벡터는 다른 벡터들의 조합으로 표현될 수 없다.

• $V = c_1 v_1 + c_2 v_2 + \dots + c_n v_n$ 일때 모든 스칼라계수 가 0 이어야 한다. ($c_1, c_2 = \dots =, v_n = 0$)

2) 공간을 생성(span):→ 생성성

기저 벡터들의 선형 결합으로 벡터 공간의 모든 벡터를 표현할 수 있어야 한다.

즉, 공간 V 의 임의의 벡터 v 는 기저 벡터들 $b_1, b_2, \dots, b_n$ 의 선형 결합으로 다음과 같이 나타낼 수 있다.

기저 벡터(Basis Vector)

- 주어진 벡터공간(벡터 부분공간)을 생성할 수 있는 기저 요소를 이루는 벡터

• 벡터공간 V 의 생성 : $V = c_1 v_1 + c_2 v_2 + \dots + c_n v_n$. 기저 벡터의 집합 : $\{v_1, v_2, \dots, v_n\}$. 여기서, 기저 벡터의 수 = 차원 = n

기저 벡터의 개수를 차원(dimension)이라고 하며, 이는 벡터 공간의 크기를 의미한다.

예를 들어, 3차원 공간에서는 세 개의 기저 벡터가 필요하다.

기저 벡터의 직관적 이해

- 2차원 공간 : 표준 기저 벡터: $e_1=(1,0)$, $e_2=(0,1)$
- 벡터 $v=(3,2)$ 는 $3 \cdot e_1 + 2 \cdot e_2$ 로 표현.
- 3차원 공간 : 표준 기저 벡터: $e_1=(1,0,0)$, $e_2=(0,1,0)$, $e_3=(0,0,1)$
- 벡터 $v=(4,5,6)$ 는 $4 \cdot e_1 + 5 \cdot e_2 + 6 \cdot e_3$ 로 표현

* basis vector

벡터의 종류

기저 벡터(basis vector)

기저 벡터의 활용

(1) 좌표계 변환

- 벡터 공간 변환: 임의의 기저 벡터로 좌표계를 변환하여 문제를 간단히 만들.
- 예: 선형 변환에서 표준 기저 벡터를 고유 기저(eigen basis)로 변환하여 다루기 쉽게 만들.
- 응용: 컴퓨터 그래픽스에서 회전/이동을 처리할 때 새로운 기저로 변환.

(2) 선형 대수학에서의 역할

- 기저 벡터는 행렬의 대각화나 고유값/고유벡터 계산의 기초가 됨.
 - 특정 문제를 해결하기 위한 최소 단위.
- 예: 데이터 분석에서 주성분 분석(PCA)은 데이터를 고유 기저로 변환.

(3) 머신러닝 및 딥러닝

- 고유값 분해와 PCA: 데이터를 저차원으로 투영할 때, 기저 벡터를 이용해 주성분(Principal Component)을 찾음.
- 임베딩 공간: 단어 임베딩에서 단어를 벡터 공간으로 표현할 때, 각 차원이 기저 벡터 역할을 함.

(4) 물리학

좌표계 표현: 물리 공간에서 기저 벡터로 위치나 힘, 속도 등의 벡터를 표현.

예: 3D 공간에서 i, j, k 는 x, y, z 축의 기저 벡터

(5) 로봇 공학

로봇의 관절 움직임이나 위치를 표현할 때, 특정 기저 벡터를 사용하여 복잡한 운동을 분해.

예: 로봇 팔의 각 관절은 특정 기저 벡터를 기준으로 움직임을 나타냄.

기저 벡터는 선형 공간의 "기본 구성 요소"로, 데이터 표현, 변환, 압축, 분석 등 다양한 분야에서 필수적이다. 머신러닝, 물리학, 그래픽스 등에서 활용되는 중요한 개념.

* Rank

* Rank

- 행렬에서 중요한 속성 중 하나로, 행렬에 포함된 선형적으로 독립적인 행(또는 열)의 최대 개수를 말한다.
- 간단히 말해, 행렬이 얼마나 많은 독립적인 정보를 가지고 있는지를 측정한다.

1. 랭크의 정의

1. **행 벡터 랭크(row rank)**: 행 공간(Row Space)의 차원, 행렬의 행 벡터들 중 선형 독립적인 벡터의 개수.
 2. **열 벡터 랭크(column rank)**: 열 공간(Column Space)의 차원(dimension), 행렬의 열 벡터들 중 선형 독립적인 벡터의 개수.
→ 열 랭크와 행 랭크는 항상 같다.
- 랭크의 의미**: 행렬이 나타내는 선형 변환에서 결과 공간의 차원이다. 즉, 행렬이 변환하는 공간의 차원을 나타낸다.

2. 랭크 계산 방법

(1) 가우스 소거법 (Gaussian Elimination)

- 행렬을 행의 기본 연산(row operations)을 통해 행사다리꼴(row echelon form)로 변환
- 행렬의 비영 벡터 (0이 아닌 행의 개수)가 랭크가 된다.

(2) 특이값 분해(SVD)

- 행렬을 분해하여 선형 독립적인 성분을 구한다.
- 이 방법은 수치적으로 안정적이고 고차원 데이터에서 유용하다.

(3) 행렬식 이용 (정사각 행렬)

- 정사각 행렬에서는 부분 행렬의 행렬식이 0이 되지 않는 가장 큰 크기의 부분 행렬을 찾아 그 크기를 랭크로 사용한다.

3. 랭크의 특징

- 랭크는 항상 행렬의 행과 열의 수 중 작은 값 이하: $\text{rank}(A) \leq \min(\text{행의 개수}, \text{열의 개수})$
- **풀 랭크(Full Rank)**: 행렬의 랭크가 행이나 열의 개수와 같으면, 해당 행렬은 풀 랭크(Full Rank)이다.
- 예: $n \times n$ 정사각 행렬에서 $\text{rank}(A)=n$ 이면 풀 랭크이다.
- 행렬 A 와 그 전치 행렬에 대해 동일하다. → $\text{rank}(A)=\text{rank}(A^T)$
- 특이 행렬(Singular Matrix): 정사각 행렬 A 의 랭크가 n 보다 작으면, A 는 역행렬이 없고, 특이 행렬이라고 한다..

풀랭크 (Full Rank)

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

2 × 2 경우
 $\text{rank}(A)=2$

* Rank

랭크 계산 예시

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \text{ 경우}$$

- 행 벡터는 $[1 \ 2]$, $[3 \ 4]$ 이다.
두 벡터는 선형 독립적이다. → 따라서 **행 벡터 랭크 = 2**.
- 열 벡터는 $\begin{pmatrix} 1 \\ 3 \end{pmatrix}$, $\begin{pmatrix} 2 \\ 4 \end{pmatrix}$ 이다.
이 두 벡터도 선형 독립적이다. → 따라서 **열 벡터 랭크 = 2**.
- **결론: rank(A)=2**

$$B = \begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix} \text{ 경우}$$

- 행 벡터는 $[1 \ 2]$, $[2 \ 4]$ 이다.
여기서 두 번째 벡터는 첫 번째 벡터의 2배이므로 **선형 종속**이다. → 따라서 **행 벡터 랭크 = 1**.
- 열 벡터는 $\begin{pmatrix} 1 \\ 2 \end{pmatrix}$, $\begin{pmatrix} 2 \\ 4 \end{pmatrix}$ 이다.
두 번째 벡터는 첫 번째 벡터의 2배이므로 **선형 종속**이다. → 따라서 **열 벡터 랭크 = 1**.
- **결론: rank(B)=1**

* Rank

* Rank

랭크 계산 예시

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

가우스 소거법(Gaussian Elimination)적용:

```
Final Row Echelon Form:
[[ 1.  2.  3.]
 [-0.  1.  2.]
 [ 0.  0.  1.]]
```

비영 벡터(0이 아닌 행)의 개수는 3이므로
 $\text{rank}(A)=3 \rightarrow A$ 는 Full rank

가우스 소거법은 행렬을 행 간의 기본 연산(row operations)을 통해 **행 사다리꼴 (row echelon form)** 또는 축약 행 사다리꼴(reduced row echelon form)로 변환하는 방법이다.

이를 통해:

- 선형 방정식의 해를 구하거나,
- 행렬의 랭크를 계산하거나,
- 행렬의 특성을 분석할 수 있다.

가우스 소거법은 세 가지 연산을 사용하여 행렬을 변환한다:

- 두 행을 서로 교환 ($R_i \leftrightarrow R_j$)
- 한 행에 0이 아닌 상수를 곱함 ($R_i \rightarrow c \cdot R_i$, $c \neq 0$).
- 한 행에 다른 행의 상수배를 더하거나 또는 뺌 ($R_i \rightarrow R_i + c \cdot R_j$)

<https://github.com/freejyb/Deep-learning>

딥러닝(수학3)

*Gauss-Jordan Elimination

- 가우스-조던 소거법(Gauss-Jordan Elimination)은 선형 방정식의 해를 구하거나 행렬을 역행렬 형태로 변환하는 데 사용하는 알고리즘이다.
- 이 방법은 행렬을 행사다리꼴(Row Echelon Form) 또는 기약 행사다리꼴(Reduced Row Echelon Form, RREF)로 변환하는 데 기반한다.

가우스-조던 소거법의 목적

1. 선형 방정식 풀이: 여러 개의 선형 방정식으로 이루어진 시스템의 해를 구함.
2. 역행렬 계산: 정사각 행렬의 역행렬을 구함.
3. 랭크 계산: 행렬의 랭크를 계산하여 선형 독립성을 판단.

가우스-조던 소거법의 단계

1. 주어진 행렬 구성

선형 방정식 시스템은 행렬 형태로 나타낼 수 있다.

방정식 $Ax=b$ 는 다음과 같은 증강 행렬(Augmented Matrix)로 표현된다. $[A|b]$

2. 기약 행사다리꼴로 변환

다음 기본 행 연산(Elementary Row Operations)을 사용하여 행렬을 변환한다.

- 행 교환(Row Swap): 두 행을 교환한다.
- 스칼라 곱(Scaling): 한 행을 0이 아닌 상수로 곱한다.
- 행 덧셈(Addition/Subtraction): 한 행의 상수 배를 다른 행에 더하거나 뺀다.

3. 피벗(Pivot) 설정

- 각 열에서 첫 번째로 0이 아닌 값을 피벗(Pivot)으로 선택한다.
- 피벗이 1이 되도록 스칼라 곱 연산을 수행한다.
- 해당 열의 나머지 값들을 0으로 만들기 위해 다른 행과 연산을 수행한다.

4. 최종 형태

기약 행사다리꼴은 다음 조건을 만족한다.

- 피벗은 1이어야 합니다.
- 피벗이 포함된 열의 다른 모든 값은 0이어야 한다.
- 피벗은 왼쪽에서 오른쪽으로 순차적으로 나타낸다.

*Gauss-Jordan Elimination

선형 방정식 시스템

$$2x + y - z = 8$$

$$-3x - y + 2z = -11$$

$$-2x + y + 2z = -3$$

1. 증강 행렬로 표현:

$$\begin{bmatrix} 2 & 1 & -1 & 8 \\ -3 & -1 & 2 & -11 \\ -2 & 1 & 2 & -3 \end{bmatrix}$$

2. 단계별 변환:

1) 첫 번째 행의 첫 번째 요소를 1로 만들기 위해 첫 번째 행을 1/2로 나눔:

$$\begin{bmatrix} 1 & 0.5 & -0.5 & 4 \\ -3 & -1 & 2 & -11 \\ -2 & 1 & 2 & -3 \end{bmatrix}$$

2) 첫 번째 열의 나머지 값을 0으로 만들:

두 번째 행: $R2 \rightarrow R2 + 3 \cdot R1$

$$R2 = [-3, -1, 2, -11] + 3 \cdot [1, 0.5, -0.5, 4]$$

계산: $R2 = [0, 0.5, 0.5, 1]$ 세 번째 행: $R3 \rightarrow R3 + 2 \cdot R1$

$$R3 = [-2, 1, 2, -3] + 2 \cdot [1, 0.5, -0.5, 4]$$

계산: $R3 = [0, 2, 1, 5]$

결과:

$$\begin{bmatrix} 1 & 0.5 & -0.5 & 4 \\ 0 & 0.5 & 0.5 & 1 \\ 0 & 2 & 1 & 5 \end{bmatrix}$$

3) 두 번째 행의 두 번째 값을 1로 만들:

두 번째 행의 두 번째 요소(0.5)를 1로 만들기
위해 다음 변환: $R2 \rightarrow R2 \div 0.5$

$$\begin{bmatrix} 1 & 0.5 & -0.5 & 4 \\ 0 & 1 & 1 & 2 \\ 0 & 2 & 1 & 5 \end{bmatrix}$$

4) 두 번째 열의 나머지 값을 0으로 만들:

두 번째 행의 두 번째 요소(0.5)를 1로 만들기
위해 다음 변환:

$$R2 \rightarrow R2 \div 0.5 \quad R_2 \rightarrow R_2 \div 0.5 \quad R2 \rightarrow R2 \div 0.5$$

$$\begin{bmatrix} 1 & 0 & -1 & 3 \\ 0 & 1 & 1 & 2 \\ 0 & 0 & -1 & 1 \end{bmatrix}$$

세 번째 행: $R3 \rightarrow R3 - 2 \cdot R2$

$$R3 = [0, 2, 1, 5] - 2 \cdot [0, 1, 1, 2]$$

계산: $R3 = [0, 0, -1, 1]$

$$\begin{bmatrix} 1 & 0 & -1 & 3 \\ 0 & 1 & 1 & 2 \\ 0 & 0 & 1 & -1 \end{bmatrix}$$

5) 세 번째 행의 세 번째 값을 1로 만들:

6) 세 번째 열의 나머지 값을 0으로 만들:

세 번째 행의 세 번째 요소(-1)를 1로 만들
기 위해 다음 변환:

$$R3 \rightarrow R3 \div -1$$

두 번째 행: $R2 \rightarrow R2 - R3$

$$R2 = [0, 1, 1, 2] - [0, 0, 1, -1]$$

$$R2 = [0, 1, 0, 3]$$

첫 번째 행: $R1 \rightarrow R1 + 0.5 \cdot R3$

$$R1 = [1, 0.5, -0.5, 4] + 0.5 \cdot [0, 0, 1, -1]$$

계산:

$$R1 = [1, 0, 0, 2]$$

$$\begin{bmatrix} 1 & 0 & 0 & 2 \\ 0 & 1 & 0 & 3 \\ 0 & 0 & 1 & -1 \end{bmatrix}$$

3. 결과:

$$x=2, y=3, z=-1$$

* Rank

$$\text{rank}(A) = \text{rank}(A^T)$$

이유 1: 선형 독립성은 변하지 않는다

- 랭크는 행렬의 선형 독립적인 행 또는 열 벡터의 최대 개수로 정의된다.
- A 의 열 벡터들의 선형 독립성은 A^T 의 행 벡터들의 선형 독립성과 같다.
- 이는 단순히 열과 행의 위치만 바뀌는 것이기 때문에, 선형 독립성에는 영향을 미치지 않는다.

이유 2: 행렬의 기본 행 연산

- 랭크는 행렬을 행 사다리꼴(Echelon form)로 변환하거나 기약 행 사다리꼴(RREF)로 변환하는 과정을 통해 계산할 수 있다.
- 이 과정에서 사용되는 행 연산은 전치 행렬에도 동일하게 적용된다.
- 따라서 A 를 변형할 때와 A^T 를 변형할 때의 결과는 동일하다.

이유 3: 행 공간과 열 공간의 차원

- A 의 열 공간(column space)과 A^T 의 행 공간(row space)은 동일한 차원을 가진다.
- 따라서 A 의 열 랭크(열 공간의 차원)와 A^T 의 행 랭크(행 공간의 차원)는 동일하다.
- 선형 대수학에서 열 랭크와 행 랭크는 항상 같으므로, 이 성질은 A^T 에도 적용된다.

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \quad A^T = \begin{bmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{bmatrix}$$

- 행렬 A
행 벡터: $[1 \ 2], [3 \ 4], [5 \ 6]$ 이 세 벡터는 선형 종속이므로 $\text{rank}(A)=2$
- 전치 행렬 A^T
행 벡터: $[1 \ 3 \ 5], [2 \ 4 \ 6]$ 이 두 벡터도 선형 종속이므로 $\text{rank}(A^T)=2$
- 결론: $\text{rank}(A)=\text{rank}(A^T)=2$

- 행렬 A : 3×2 행렬로, 열 벡터가 2개뿐이다. $\rightarrow A$ 의 최대 rank는 열 벡터의 개수인 2이다.
- A 의 세 행 벡터 중 3개가 모두 선형 독립일 수 없다(랭크가 2이므로).
- 실제 선형 종속 확인: 행 벡터: $r_1=[1 \ 2], r_2=[3 \ 4], r_3=[5 \ 6]$
 $\rightarrow$ 세 번째 행 벡터 r_3 는 첫 번째와 두 번째 행 벡터의 선형 결합으로 표현된다.
 $r_3=2r_1+(-1)r_2 \rightarrow [5 \ 6]=2[1 \ 2]-1[3 \ 4]$
- 따라서 r_3 는 r_1 과 r_2 로 표현될 수 있으므로, 세 행 벡터는 선형 종속이다.

* Vector space

벡터 공간(vector space)과 부분 공간(Subspace)

- 벡터 공간(Vector Space): 선형 대수학에서 정의된 수학적 구조로, 벡터들의 집합과 그 위에서 정의된 연산(벡터 덧셈, 스칼라 곱셈)을 포함한다.
- 부분 공간(Subspace): 주어진 벡터 공간의 일부로, 특정 조건을 만족하는 벡터들의 집합이다. 부분 공간도 독립적인 벡터 공간의 성질을 가지며, 벡터 공간의 선형 구조를 유지한다.

벡터 공간의 정의

벡터 공간은 다음 성질을 만족하는 집합 V 와 연산(벡터 덧셈 및 스칼라 곱셈)으로 구성된다:

1. 벡터 덧셈의 닫힘성: $\mathbf{u}, \mathbf{v} \in V \implies \mathbf{u} + \mathbf{v} \in V$
2. 스칼라 곱셈의 닫힘성: $c \in \mathbb{R}, \mathbf{u} \in V \implies c \cdot \mathbf{u} \in V$
3. 덧셈 및 스칼라 곱셈의 결합 법칙, 분배 법칙 등 선형 대수의 기본 연산 성질을 만족.

부분 공간(Subspace)의 정의

V 의 부분 공간이 되기 위한 조건은 다음과 같다.

1. 0 벡터 포함: $\mathbf{0} \in W$ 항상 원점을 포함한다.
2. 덧셈에 대해 닫혀 있음: $\mathbf{u}, \mathbf{v} \in W \implies \mathbf{u} + \mathbf{v} \in W$
3. 스칼라 곱셈의 닫힘성: $c \in \mathbb{R}, \mathbf{u} \in V \implies c \cdot \mathbf{u} \in V$

부분 공간과 기저(basis)

부분 공간은 항상 특정 벡터 집합의 선형 결합으로 생성된다. 이를 생성(span)이라고 하며, 기저(basis)는 부분 공간을 생성하는 벡터들의 최소 집합을 말한다.

* Latent space

잠재공간(Latent Space)

- 잠재공간은 주로 기계 학습과 딥러닝에서 사용되는 개념으로, 고차원 데이터를 낮은 차원의 공간으로 변환한 결과를 의미한다.
- 이 공간은 원래 데이터의 중요한 정보를 압축하거나 표현한다.
- 예를 들어, 이미지 데이터의 주요 특징만을 추출해 표현한 벡터는 잠재공간에서의 표현이다.

특징

- 차원 축소: 고차원 데이터에서 의미 있는 패턴이나 특징을 추출하여 더 낮은 차원으로 표현.
- 해석 가능성: 잠재공간의 각 차원이 데이터의 특정 의미(예: 얼굴 이미지에서 웃음 정도, 조명 조건)를 나타낼 수 있음.
- 비선형 변환 가능: 딥러닝에서는 선형 변환뿐 아니라 비선형 방식으로 잠재공간을 생성.
- 수학적 예:
 - ✓ PCA를 통해 고차원 데이터를 주성분(Principal Components)으로 변환한 공간.
 - ✓ 오토인코더의 중간 레이어에서 데이터가 표현되는 공간.
- 사용 분야: 차원 축소, 데이터 압축, 생성 모델(GAN, VAE), 클러스터링, 추천 시스템.

구분	부분공간(Subspace)	잠재공간(Latent Space)
기원	선형 대수학	기계 학습 및 딥러닝
정의	벡터 공간의 선형적 부분 집합	데이터의 의미 있는 특징이 압축된 표현 공간
연산	선형 연산(덧셈, 스칼라 곱셈)에 대해 닫혀 있음	선형 및 비선형 연산 모두 가능
원점 포함 여부	항상 원점을 포함	원점을 포함할 필요 없음
응용	선형 대수학, 기하학	차원 축소, 데이터 압축, 생성 모델, 클러스터링
차원	명시적으로 정의됨	데이터에 따라 유연하게 설정 가능
구성 방식	벡터의 선형 결합으로 생성	딥러닝 또는 차원 축소 알고리즘을 통해 학습하여 생성

1차원 /1차원 벡터(Vector) 1D 벡터

1차원 (1D)

: 스칼라 값들이 모여서 구성된 1차원 배열이며, 수학적으로는 단순한 숫자 리스트라고 할 수 있다.

→ 데이터가 단순히 나열된 형태를 말한다.

- 1차원(1D) : 하나의 축(axis)만 가지는 공간이나 데이터를 나타냅니다. (점: 0차원 정지상태, 선: 1차원 한 방향)
- 컴퓨터 프로그래밍에서는 1차원 배열(array)이나 리스트(list)로 구현된다..
- [1, 2, 3, 4, 5]는 1차원 배열이며, 크기(Shape)는 (5,)이다.

1차원 벡터

: 방향과 크기를 가진 단일 축의 데이터를 나타낸다.

1차원 벡터의 특징

방향과 크기를 가진다. [3,4]는 크기가 $\sqrt{3^2 + 4^2}=5$ 이다.

1차원 벡터 활용 예시:

- 데이터 분석: 각 데이터 포인트의 특성(features)을 1차원 벡터로 표현할 수 있다. → 데이터 포인트의 특징 벡터
- 기계 학습: 입력 데이터는 보통 1차원 벡터로 모델에 전달된다. → CNN Flatten 출력
- 신호 처리: 시간에 따른 신호 데이터를 1차원 벡터로 표현할 수 있다.

특징	1차원 (1D)	1차원 벡터 (1D Vector)
정의	선형적으로 나열된 데이터	크기와 방향을 가진 벡터
형태	[v1, v2, ..., vn]	[x] 또는 $\mathbf{v} = [x]$
크기 (Shape)	(n,)	(1,)
방향	없음	있음
용도	데이터 분석, 배열	물리적 양, 수학적 벡터

2차원 / 2차원 벡터(Vector)

2차원 (2D)

정의: 2차원(2D)은 두 개의 축(axis), 즉 x축과 y축으로 이루어진 평면 공간을 나타낸다.

예시:

- **데이터 표(table):** 행(row)과 열(column)로 구성된 표.
- **이미지:** 가로와 세로 픽셀로 이루어진 2차원 배열. → **면적**
- **행렬(Matrix):** 수학에서의 행렬은 2차원 배열이다.

2차원 배열: $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$

크기 (Shape): (2, 3)

특징:

- 크기 (Shape)는 (행, 열) 형식이다.
- 데이터가 평면 상에 배치되며, 각 원소는 행과 열의 인덱스를 통해 접근할 수 있다.
- 물리적인 위치나 배열된 데이터를 나타낸다.
- 활용 예시: 이미지 데이터(예: 흑백 이미지의 픽셀값), 통계 데이터의 표 형식, NLP에서 단어 임베딩 행렬 등.
- 2차원 벡터는 데이터를 행과 열의 위치에 따라 구분하여 다룰 수 있어 이미지 데이터나 테이블 데이터를 표현하는 데 적합
- 크기 (Shape): (3, 3) — 3행과 3열을 가진 행렬

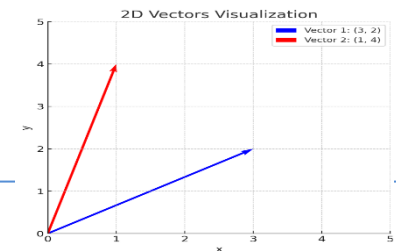
2차원 벡터 (2D Vector)

정의: 2차원 벡터는 크기(magnitude)와 방향(direction)을 가진 벡터로, x 성분과 y 성분으로 이루어진다.

• 이는 평면 상의 하나의 점이나 이동 방향을 나타낸다.

예시:

- **위치 벡터:** 평면 상의 특정 위치를 나타낸다.
- **속도 벡터:** 물체의 이동 방향과 속도를 나타낸다.



3차원 / 3차원 벡터(Vector)

3차원 (3D)

•정의: 3차원(3D)은 x, y, z라는 세 개의 축을 가지는 공간을 의미한다.

•예시:

- 3D 공간: **현실 세계의 공간**. 예를 들어, 우리가 사는 세상은 길이, 너비, 높이로 구성된 3차원 공간이다.
- 3차원 배열 (3D Array): 컴퓨터 과학에서 3차원 배열은 여러 개의 **2차원 행렬로 구성된 데이터 구조이다**. 예를 들어, 비디오 데이터는 여러 개의 이미지 프레임(2D 배열)으로 구성되어 3차원 배열이 된다.

구성 요소

: 3차원은 여러 개의 2차원 행렬(Height × Width)이 깊이(Depth)를 따라 쌓여 있는 형태

표현 방식: [depth, height, width]

코드 표현 : 크기 (Shape): (2, 3, 5) — 두 개의 3x5 행렬이다. Shape는 (깊이, 행, 열) 형식이다.

- 첫 번째 차원: 2 (행렬의 개수)
- 두 번째 차원: 3 (각 행렬의 행 개수)
- 세 번째 차원: 5 (각 행렬의 열 개수)

활용 예시

- 이미지 데이터: 컬러 이미지의 경우 각 픽셀에 대해 RGB 값이 필요하므로, 이미지 전체가 3차원 벡터로 표현
- CNN의 특징 맵: 합성곱 연산을 거친 CNN의 특징 맵은 깊이, 높이, 너비를 가진 3차원 배열이 된다.
- 동영상 데이터: 영상 프레임은 시간의 흐름에 따라 여러 개의 2D 이미지가 쌓여 있는 형태이므로 3차원 배열로 처리될 수 있다.

3차원 벡터 (3D Vector)

•정의: 3차원 벡터는 x, y, z 세 개의 성분으로 구성되며, 크기(magnitude)와 방향(direction)을 가지는 벡터입니다. 이는 3차원 공간에서 위치나 이동 방향을 나타낸다.

•예시:

- 위치 벡터: 3차원 공간에서의 특정 위치를 나타낸다.
- 힘 벡터: 물리학에서의 힘의 크기와 방향을 나타낸다.

* n차원

2차원 / 2차원 벡터(Vector) 요약 비교

특징	2차원 (2D)	2차원 벡터 (2D Vector)
정의	행과 열로 이루어진 2차원 배열	크기와 방향을 가진 벡터
형태	(행, 열) 형식의 배열	$[x, y]$ 또는 $\mathbf{v} = \begin{bmatrix} x \\ y \end{bmatrix}$
크기 (Shape)	(m, n) (예: (2, 3))	(2,)
방향	없음	방향이 정의됨
용도	데이터 표, 행렬, 이미지	위치, 힘, 속도 등 물리적 양
계산 방법	행렬 연산	벡터 연산 (내적, 외적, 길이 계산 등)

3차원 / 3차원 벡터(Vector) 요약 비교

특징	3차원 (3D)	3차원 벡터 (3D Vector)
정의	x, y, z 축의 3차원 공간	x, y, z 성분을 가진 벡터
형태	(깊이, 행, 열) 형식의 배열	$[x, y, z]$ 또는 $\mathbf{v} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$
크기 (Shape)	(n, m, k)	(3,)
방향	없음	방향이 정의됨
용도	데이터 구조, 이미지, 비디오	물리적 양, 위치, 속도
계산 방법	행렬 연산	벡터 연산 (내적, 외적, 길이 계산 등)

4차원 이상 (4D and Higher)

- 스칼라(scalar), 벡터(vector), 행렬(matrix)의 일반화된 개념이다.
- 텐서는 데이터를 다차원 공간에 표현하기 위한 구조이며, **다차원 배열**(N-dimensional array) 또는 **텐서(Tensor)**라고 불린다. → **tension**에서 유래
- 딥 러닝에서 사용되는 데이터는 종종 4차원 이상의 구조를 가진다.
- 예를 들어, 비디오 데이터는 **(batch_size, time_steps, height, width, channels)** 형식으로 표현된다.

텐서의 예시 (차원에 따른 구분)

- 스칼라 (0차원 텐서) → 단순한 숫자(크기만 존재).
예: $x=3$ 차원: 0 (랭크: 0)
- 벡터 (1차원 텐서) → 값들의 리스트(크기와 방향).
예: $v=[1,2,3]$ 차원: 1 (랭크: 1)
- 행렬 (2차원 텐서)
값들이 행(row)과 열(column)로 배열된 2차원 데이터.
예: $A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ 차원: 2 (랭크: 2)
- 3차원 텐서 (3D 텐서)** → 여러 개의 행렬(행렬의 배열)
예: RGB 이미지처럼 3개의 채널을 가지는 데이터. $T = \begin{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} & \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} \end{bmatrix}$
차원: 3 (랭크: 3)
- N차원 텐서 (N-D Tensor)** 더 높은 차원의 데이터.
예: 딥러닝 모델에서 사용하는 텐서.
차원이 4,5,...4, 5, \dots 4,5,... 일 수 있음.
예: 비디오 데이터 (batch size×time×height×width×channels)

텐서의 주요 속성

1. 랭크 (Rank)
텐서의 차원 수.
예: 스칼라의 랭크는 0, 벡터는 1, 행렬은 2.
2. 형상 (Shape)
각 차원의 크기.
예시)
 - 벡터 $[1,2,3]$ 의 Shape는 (3,) ,
 - 행렬 $\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ 의 Shape는 (2,2)
 - 3차원 텐서의 Shape는 (3,4,5) 일 수 있음.
3. 데이터 타입 (Data Type)
텐서에 저장된 데이터의 타입.
예: 정수(int), 부동소수점(float), 복소수(complex) 등.

차원의 중요성

1. 데이터 표현

: 차원이 많아질수록 데이터가 나타내는 특징이 복잡해진다.

2. 모델 학습

: 차원이 높으면 모델이 더 많은 정보를 학습할 수 있지만, 차원의 저주(Curse of Dimensionality)로 인해 학습이 어려워질 수도 있다.

차원 축소 방법

: 고차원 데이터를 다루기 어렵기 때문에 차원 축소 방법을 실행

1. 기계학습 기반 차원 축소 방법

1) **PCA(Principal Component Analysis) : 선형 변환**을 사용, 데이터의 분산을 최대화하는 방향을 보존, 비지도 학습 → 가장 널리 사용되는 선형 차원 축소 방법

2) **Autoencoder : 비선형 차원 축소 가능**. 데이터의 구조를 학습. 고차원 데이터에 적합

2. 수학적 변환 기반 차원 축소 방법

1) **선형 판별 분석 (LDA: Linear Discriminant Analysis)**

클래스 간 분산을 최대화하고, 클래스 내부 분산을 최소화하는 축을 찾아 차원을 축소하는 방법 → 분류 문제에서 주로 사용. 클래스 레이블이 필요함(**지도 학습 방식**). **선형 변환**.

2) **SVD (Singular Value Decomposition)**

: 데이터 행렬을 세 개의 행렬로 분해하여 차원을 줄이는 방법.

3. 피처 선택 기반 차원 축소 방법

: 변수 선택 (Feature Selection)을 통해 데이터를 변환하지 않고, 중요한 변수(특징)만 선택하는 방식.

1) **필터 방법(Filter Method)**: 통계적 기법을 사용하여 변수의 중요도 평가

- 분산 기준 (Variance Threshold): 분산이 낮은 피처(거의 변화하지 않는 피처)를 제거.
- 상관 계수 (Correlation Coefficient): 목표 변수와 각 피처 간의 상관 계수를 계산. → 상관 계수가 높은 피처만 선택(예: 피어슨 상관 계수).

2) **래퍼 방법(Wrapper Method)**: 모델의 성능을 기준으로 변수 선택

→ Forward Selection, Backward Elimination, Stepwise Selection, 순열 중요도(Permutation Importance).

3) **임베디드 방법(Embedded Method)**: 모델 학습 과정에서 피처 선택을 동시에 수행.

- L1 정규화 (Lasso Regression): 회귀 문제에 자주 사용.
- Elastic Net
- 트리 기반 모델 (Tree-Based Model)
- 결정 트리, 랜덤 포레스트, 그라디언트 부스팅 모델에서 피처 중요도를 평가. 피처의 분할 기준(불순도 감소량)을 기준으로 중요도 산출

차원 축소 방법

피처 선택 기반 차원 축소 방법 비교

방법	기준	장점	단점	활용예시
Filter Method	통계적 척도	빠르고 간단	피처 간의 상호작용을 고려하지 않음	- 데이터셋 크기가 크고 피처가 많을 때 - 사전 처리 단계로 사용.
Wrapper Method	모델 성능	피처 상호작용을 고려	계산 비용 높음, 과적합 위험	- 소규모 데이터셋에서 최적의 피처 집합 탐색. - 높은 정확도가 필요한 모델 학습
Embedded Method	모델 내부 평가	효율적이고 자동화	특정 알고리즘에 종속적	- 대규모 데이터 셋 - 트리기반 모델 - 머신러닝/딥러닝

Linear Algebra : Matrix

행렬 (Matrix)

- 행렬은 복수의 차원을 가지는 데이터 레코드가 다시 여러 개 있는 경우의 데이터를 합쳐서 표기한 것
- 행렬은 **행(row)과 열(column)**이라는 인덱스 : $m \times n$, (행 : m , 열 : n) $m \times m$
- 벡터가 공간에서 한 점을 된다면, 행렬은 여러점들로 표시 됨.
- 행렬 : 대문자로 표기, 스칼라 : 이태리 소문자로, 벡터 : 볼드체 소문자로 표기

행렬과 벡터와 스칼라

- 수학적으로 행렬은 벡터들을 나열한 것 !! → 행렬은 벡터를 계산하는데 사용된다.
- 스칼라와 벡터도 수학적으로는 행렬이다.
- 스칼라는 열과 행의 수가 각각 1인 행렬이다.

행렬 표기

- 보통 3차원까지의 벡터는 그림 등으로 시각적 표현이 가능하지만 그 이상의 벡터는 벡터의 각 구성요소를 괄호 안에 나열함으로 써 표기.
 - 선형화 혹은 선형 근사를 통해, 복잡한 비선형 방정식 문제를 간단한 선형 방정식 문제로 변환해 문제를 해결할 수 있기 때문
→ 행렬은 기본적으로 선형변환을 표현하는 도구이다.
- 특히, n -차원 벡터 공간에서 m 차원 벡터 공간으로의 선형변환은 항상 $m \times n$ 행렬로 표현될 수 있다.

텐서

같은 크기의 행렬이 여러 개 같이 묶여 있는 것

Linear Algebra : Matrix

행렬의 종류

전치행렬(transposed matrix) : A^T

행렬의 행과 열을 서로 맞바꾼 행렬

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad A^T = \begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix}$$

단위 행렬(unit matrix) 또는 항등 행렬(Identity matrix) : I

주대각선의 원소가 모두 1이며 나머지 원소는 모두 0인 정사각 행렬

$$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad I = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

역행렬(Inverse Matrix) : A^{-1}

$AB=BA=I$: B는 A의 역행렬

$\rightarrow AA^{-1}=A^{-1}A=I \rightarrow A^{-1}$ 는 A의 역행렬

행렬에는 나누기 연산이 없기 때문에 역행렬을 곱해야 함.

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \text{에 대한}$$

$$A^{-1} = \frac{1}{ad-bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$

(예시)

$$A = \begin{pmatrix} 1 & 2 \\ 2 & 6 \end{pmatrix} \rightarrow A^{-1} = \frac{1}{6-4} \begin{pmatrix} 6 & -2 \\ -2 & 1 \end{pmatrix}$$

$$A A^{-1} = \begin{pmatrix} 1 & 2 \\ 2 & 6 \end{pmatrix} \begin{pmatrix} 3 & -1 \\ -1 & \frac{1}{2} \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

가역성(nonsingular)

가역성은 행렬 A가 역행렬(inverse matrix)을 가질 수 있는 성질을 말한다.
A가 가역적(nonsingular)이라면 다음 조건을 만족한다:

$$A^{-1} \cdot A = I \quad (\text{단위행렬, } I, \text{반환})$$

정방행렬(square matrix)

행의 개수와 열의 개수가 같은 행렬 $m \times m$

영행렬(Null Matrix or zero Matrix)

모든 원소가 0 이다.

Linear Algebra : Matrix

* 행렬의 가역성 (Nonsingular Matrix)

- 주어진 정사각 행렬 A 가 역행렬 A^{-1} 를 가질 수 있는지를 판단하는 중요한 개념이다.
- 역행렬이 존재하는 행렬을 가역 행렬 (Invertible Matrix) 또는 비특이 행렬 (Nonsingular Matrix)이라고 한다.

1. 가역 행렬의 정의

정사각 행렬 A 에 대해, 만약 행렬 B 가 존재하여:

$A \cdot B = B \cdot A = I$ 를 만족한다면, A 는 가역적이며, B 를 A 의 역행렬이라고 한다.

여기서: I : 단위 행렬(identity matrix). $B = A^{-1}$ A 의 역행렬.

2. 가역 행렬의 조건 [행렬 A 가 가역적이기 위한 조건]

1. 행렬식이 0이 아니다 ($\det(A) \neq 0$)

: 행렬식 $\det(A) = 0$ 이면 A 는 특이 행렬(singular matrix)로, 역행렬이 존재하지 않는다.

2. 행의 독립성

행렬 A 의 모든 행 또는 열이 **선형적으로 독립적이어야 한다**. 즉, 하나의 행(열)을 다른 행(열)들의 선형 결합으로 표현할 수 없어야 한다.

3. 랭크가 행렬의 크기와 같다:

행렬 A 의 랭크(rank)가 행렬의 크기 n 과 같아야 한다. → 풀 랭크 (Full Rank) 즉, $\text{rank}(A) = n$

3. 가역 행렬의 계산 방법

행렬 A 의 역행렬을 구하는 가장 일반적인 방법은 **가우스-조던 소거법** (Gauss-Jordan Elimination)

역행렬 계산 공식 (2x2 행렬 예시)

주어진 행렬:

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \quad A^{-1} = \frac{1}{\det(A)} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix} \quad \text{단, } \det(A) = ad - bc \neq 0$$

행렬의 종류

직교행렬(Orthogonal Matrix)

직교행렬의 핵심적인 특징은 **벡터의 길이와 각도를 보존**한다는 점이다.

직교행렬의 정의

행렬 A 가 직교행렬이기 위해서는 다음 조건을 만족해야 한다.

$$A^T A = I$$

A^T : 행렬 A 의 전치행렬 (Transpose matrix).
 I : 단위 행렬 (Identity matrix).

- 이 식은 행렬 A 와 그 전치행렬을 곱했을 때 결과가 단위 행렬이 됨을 의미한다.
- 즉, 행렬 A 의 열 벡터들이 서로 직교(orthogonal)하고, 모두 단위 벡터(길이 1)임을 나타낸다.

행렬 A 가 직교행렬이라면, $A^T A$ 의 결과는 단위 행렬(I)이 된다.

직교행렬의 성질

1. 역행렬이 존재하며, 역행렬은 전치 행렬과 동일함: $A^{-1} = A^T$.

• 직교행렬의 역행렬은 그 전치행렬과 같다.

2. 벡터의 길이 보존:

직교행렬 A 에 벡터 x 를 곱하면, 그 결과 벡터의 길이(노름)가 변하지 않는다.

$$\|Ax\| = \|x\|$$

3. 각도 보존: 두 벡터 u 와 v 의 내적이 직교행렬에 의해 변형되더라도 그 값이 변하지 않는다. $(Au) \cdot (Av) = u \cdot v$.

직교행렬의 예

- 직교행렬의 가장 대표적인 예는 회전 행렬(Rotation Matrix)이다.
- 회전 행렬은 벡터를 원점에서 회전시키는 변환을 하며, 벡터의 길이와 각도를 보존한다.

Linear Algebra : Matrix

행렬의 연산

1. 행렬의 덧셈 (Matrix Addition)

행렬을 덧셈 하기 위해서는 먼저 연산 하고자 하는 **행렬끼리의 차원이 동일해야 한다.**

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix} + \begin{pmatrix} e & f \\ g & h \end{pmatrix} = \begin{pmatrix} a+e & b+f \\ c+g & d+h \end{pmatrix}$$

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} \quad A + B = \begin{bmatrix} 1+5 & 2+6 \\ 3+7 & 4+8 \end{bmatrix} = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}$$

2. 행렬의 뺄셈 (Matrix Subtraction)

행렬을 뺄셈하기 위해서는 먼저 연산 하고자 하는 **행렬끼리의 차원이 동일해야 한다.**

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix} - \begin{pmatrix} e & f \\ g & h \end{pmatrix} = \begin{pmatrix} a-e & b-f \\ c-g & d-h \end{pmatrix}$$

$$A = \begin{bmatrix} 7 & 8 \\ 5 & 6 \end{bmatrix}, \quad B = \begin{bmatrix} 3 & 2 \\ 1 & 4 \end{bmatrix} \quad A - B = \begin{bmatrix} 7-3 & 8-2 \\ 5-1 & 6-4 \end{bmatrix} = \begin{bmatrix} 4 & 6 \\ 4 & 2 \end{bmatrix}$$

행렬의 연산

3. 행렬의 곱셈 (Matrix Multiplication)

행렬끼리의 곱셈 : **적합성조건**(conformability condition)이 충족되어야 한다.

→ 두 행렬 중 앞 행렬의 **열**과 뒷 행렬의 **행**의 갯수가 동일해야 함.

→ 결과 만들어지는 행렬의 차원은 (앞 행렬의 행의 개수 x 뒷 행렬의 열의 개수)가 된다.

스칼라 곱셈

$$k \begin{pmatrix} a & b \\ c & d \end{pmatrix} = \begin{pmatrix} ka & kb \\ kc & kd \end{pmatrix}$$

→ 스칼라 곱셈은 어떤 차원의 행렬이든 가능

행렬의 연산

3. 행렬의 곱셈 사례

- 두 행렬 중 앞 행렬의 열과 뒷 행렬의 행의 갯수가 동일해야 함.
- $m \times n$ 행렬 곱셈은 $n \times k$ 행렬이어야 함.
- → 결과 만들어지는 행렬의 차원은 (앞 행렬의 행의 개수 $\times$ 뒷 행렬의 열의 개수)가 된다.
- $m \times n$ 행렬과 $n \times k$ 행렬 곱셈은 $m \times k$ 행렬이 됨.

$$[1 \ 2 \ 3]_{1 \times 3} \times \begin{pmatrix} 4 \\ 5 \\ 6 \end{pmatrix}_{3 \times 1} = 1 \times 4 + 2 \times 5 + 3 \times 6 = 32$$

→ 1×3 행렬과 3×1 행렬 곱셈 결과 스칼라

$$\begin{pmatrix} 1 \\ 2 \end{pmatrix}_{2 \times 1} \times (3 \ 4 \ 5)_{1 \times 3} = \begin{pmatrix} 1 \times 3 & 1 \times 4 & 1 \times 5 \\ 2 \times 3 & 2 \times 4 & 2 \times 5 \end{pmatrix} = \begin{bmatrix} 3 & 4 & 5 \\ 6 & 8 & 10 \end{bmatrix}$$

→ 2×1 행렬과 1×3 행렬 곱셈 결과 2×3 행렬

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}_{3 \times 2} \quad B = \begin{bmatrix} 2 \\ 3 \end{bmatrix}_{2 \times 1} \rightarrow AB = \begin{bmatrix} 1 \times 2 + 2 \times 3 \\ 3 \times 2 + 4 \times 3 \\ 5 \times 2 + 6 \times 3 \end{bmatrix} = \begin{bmatrix} 8 \\ 18 \\ 28 \end{bmatrix}_{3 \times 1}$$

→ 3×2 행렬과 2×1 행렬 곱셈 결과 3×1 행렬

Linear Algebra : Matrix

행렬의 연산

두 행렬

행렬 A : $m \times n$ 행렬

행렬 B : $n \times r$ 행렬

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}, B = \begin{pmatrix} b_{11} & b_{12} & \cdots & b_{1r} \\ b_{21} & b_{22} & \cdots & b_{2r} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{nr} \end{pmatrix}$$

행렬의 곱

AB : $m \times r$ 행렬

$$AB = \begin{pmatrix} \sum_k a_{1k} b_{k1} & \sum_k a_{1k} b_{k2} & \cdots & \sum_k a_{1k} b_{kr} \\ \sum_k a_{2k} b_{k1} & \sum_k a_{2k} b_{k2} & \cdots & \sum_k a_{2k} b_{kr} \\ \vdots & \vdots & \ddots & \vdots \\ \sum_k a_{mk} b_{k1} & \sum_k a_{mk} b_{k2} & \cdots & \sum_k a_{mk} b_{kr} \end{pmatrix}$$

단, $k=1, 2, 3, \dots, n$

행렬의 연산 법칙

행렬의 덧셈

- 교환법칙 성립 : $A + B = B + A$
- 결합법칙 성립 : $(A+B) + C = A + (B + C)$

행렬의 곱셈

- 교환 법칙이 성립하지 않는다 : $AB \neq BA$
 - 행렬의 곱셈에서 **두 행렬의 순서** 또한 중요한 것을 알 수 있다.
 - 단, 곱하려는 행렬 중 하나가 단위행렬이거나 영행렬인 경우는 교환법칙 성립
- 덧셈에 대한 분배법칙은 성립 : $A (B + C) = AB + AC$

<https://github.com/freejyb/KDT>

행렬과 사칙연산

행렬을 이용하여 연립방정식 사례

주어진 연립방정식:

$$2x + y = 5$$

$$x - y = 1$$

행렬로 표현하면: $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & -1 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 5 \\ 1 \end{bmatrix}$

행렬 방정식 형태로는: $\mathbf{Ax} = \mathbf{b}$

해: $\mathbf{x} = \begin{bmatrix} x \\ y \end{bmatrix} = \mathbf{A}^{-1}\mathbf{b}$

계산 결과: $x = 2.00, \quad y = 1.00$

따라서, 이 연립방정식의 해 $\rightarrow x=2, y=1$ 이다

<https://github.com/freejyb/Deep-learning>
딥러닝(수학1)

Linear Algebra : Matrix

행렬의 크기 (Norm)

표기 $\|A\|_p = \left(\sum_{i=1}^N \sum_{j=1}^M |a_{ij}|^p \right)^{1/p}$ **Vector L2 Norm → 행렬 Norm으로 확장**

p=2인 경우가 가장 일반적

$$\|A\| = \|A\|_2 = \|A\|_F = \sqrt{\sum_{i=1}^N \sum_{j=1}^M a_{ij}^2}$$

값 ≥ 0

벡터의 놈의 제곱이 벡터의 제곱합과 같다는 것

$$\|x\|^2 = \sum_{i=1}^N x_i^2 = x^T x$$

- Euclidean Norm → L2-Norm
- Frobenius norm : vector L2 Norm을 행렬로 확장한 버전

Frobenius

- 페르디난트 게오르크 프로베니우스(: Ferdinand Georg Frobenius, 1849년 10월 26일 ~ 1917년 8월 3일) 독일의 수학자
- 타원함수론, 미분방정식론 에 기여

Linear Algebra : Determinant (det)

1. 행렬식(det)의 정의

- 행렬식은 주어진 $n \times n$ 정사각행렬 A 에 대해 실수나 복소수 값으로 계산되는 함수로 정의된다.
- 행렬식은 보통 $\det(A)$ 또는 A 로 표시된다.

예를 들어, 2×2 행렬 A 가 다음과 같다면: 행렬식은 다음과 같이 계산된다:

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

$$\det(A) = ad - bc$$

$$A = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$$

$$\det(A) = a(ei - fh) - b(di - fg) + c(dh - eg)$$

2. 행렬식의 주요 의미

1) 면적 또는 부피:

2차원에서는 행렬식의 절댓값이 두 벡터로 정의되는 평행사변형의 면적을 나타낸다.

3차원에서는 행렬식의 절댓값이 벡터들이 정의하는 평행육면체의 부피를 나타낸다.

2) 선형 변환의 성질:

행렬은 선형 변환을 나타내며, 행렬식은 그 선형 변환이 공간에 미치는 크기 변화 비율(스케일링)을 나타낸다.

$\det(A) > 0$: 변환 후 방향이 유지됨.

$\det(A) < 0$: 변환 후 방향이 반전됨.

3) 역행렬 존재 조건:

$\det(A) = 0$ 이면 행렬 A 는 역행렬이 없으며, 이는 A 가 비가역적(singular)임을 의미한다.

$\det(A) \neq 0$ 이면 A 는 가역적(nonsingular)이며, 역행렬이 존재한다.

4) 선형 독립성:

$\det(A) = 0$ 일 경우, 행렬의 열벡터(또는 행벡터)들이 선형 종속임을 나타낸다.

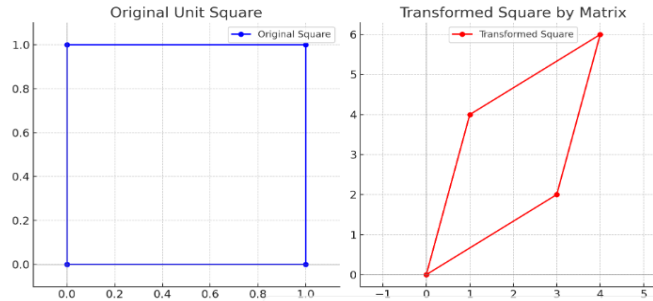
$\det(A) \neq 0$ 일 경우, 행렬의 열벡터(또는 행벡터)들이 선형 독립적임을 나타낸다.

Linear Algebra : Determinant (det)

행렬식 사례

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

$$\det(A) = ad - bc$$



$$\begin{bmatrix} 3 & 1 \\ 2 & 4 \end{bmatrix}$$

$$\det = 10$$

1) 행렬식(det) 값이 10이라는 것은 해당 행렬이 정의하는 선형 변환이 2차원 공간에서 면적을 10배로 확대한다는 것을 의미한다.

원래 면적: 변환 전 단위 정사각형의 면적은 1입니다.

변환 후 면적: 행렬식 값이 10이므로 변환 후 정사각형의 면적은 10이다.

즉, 행렬

$A = \begin{bmatrix} 3 & 1 \\ 2 & 4 \end{bmatrix}$ 은 공간에서 벡터를 변환하여 면적을 10배로 늘리는 선형 변환을 수행한다.

2) 행렬식이 양수($\det > 0$)이므로, 변환 후 도형의 방향(시계/반시계)은 유지되고 있다. → 행렬식이 $\neq 0$ 이므로, 이 행렬은 가역적(nonsingular)이다.

$$A = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$$

$$\det(A) = a(ei - fh) - b(di - fg) + c(dh - eg)$$

$$\begin{bmatrix} 2 & 3 & 1 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

$$\det = 9$$

<https://github.com/freejyb/Deep-learning>
딥러닝(수학1)

* 고유값(Eigenvalue)과 고유벡터(Eigenvector)

고유값 (Eigenvalue)과 고유벡터(Eigenvector)

고유벡터 (Eigenvector)

- 행렬 A 가 특정 방향의 벡터를 변화시키지 않고 크기만 변경시키는 경우, 그 벡터가 고유벡터이다. → 행렬 A 에 대해 고유벡터 v 는 우측 조건을 만족하는 벡터이다
- 즉, 행렬 A 가 고유벡터 v 를 크기만 λ 배로 변환시킬 때 v 를 고유벡터라고 한다.

행렬 방정식:

$$Av = \lambda v$$

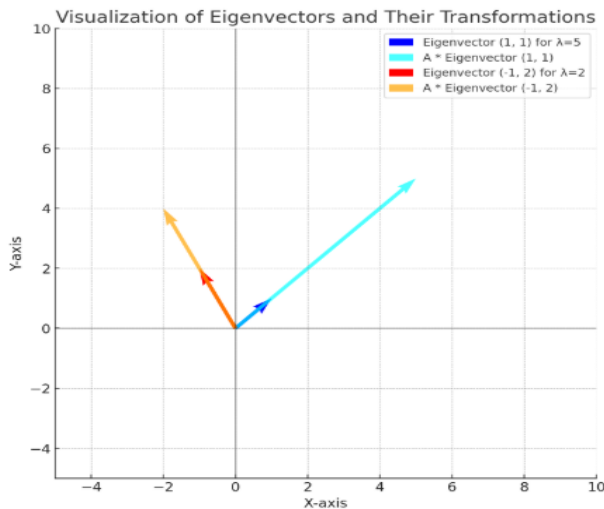
고유값 (Eigenvalue)

- 행렬 A 가 특정 방향의 벡터를 변화시키지 않고 크기만 변경시키는 경우, 그 벡터가 고유벡터이다..→ 이 때의 **스칼라 값 λ 는 고유값**
- 고유값은 행렬의 고유벡터에 대응하는 크기 조정 인자이다.

- A 는 행렬
- v 는 고유벡터
- λ 는 고유값

고유벡터는 행렬 변환 시 "고유의 방향"을 나타내고, 고유값은 그 방향에 대한 "스케일링(확대/축소) 정도"를 나타낸다.

$$A = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix}$$



고유벡터 (Eigenvectors):

- **파란색 벡터**는 첫 번째 고유벡터 (v_1) 로, 고유값 $\lambda_1=5$ 에 해당
- **빨간색 벡터**는 두 번째 고유벡터 (v_2)로, 고유값 $\lambda_2=2$ 에 해당

고유값
 $\lambda=5$ 와 $\lambda=2$
(뒷면 참조)

- 변환된 고유벡터 ($A * \text{Eigenvectors}$)
- 원래 고유벡터를 행렬 A 로 변환한 결과를 반투명 색상으로 표시했다.
- 변환된 벡터는 원래 고유벡터와 동일한 방향을 유지하면서 크기만 변경된 것을 볼 수 있다.
- 이 그래프를 통해 행렬 A 의 작용에 대해 고유벡터가 고유의 방향을 유지하며, 고유값에 따라 크기가 변하는 과정을 시각적으로 이해할 수 있다.

- 고유값 $\lambda_1=5$ 에 대한 고유벡터 → $\mathbf{v}_1 = k \begin{pmatrix} 1 \\ 1 \end{pmatrix}$
- 고유값 $\lambda_2=2$ 에 대한 고유벡터 → $\mathbf{v}_2 = k \begin{pmatrix} -1 \\ 1 \end{pmatrix}$

* 고유값(Eigenvalue)과 고유벡터(Eigenvector)

고유값과 고유벡터를 찾는 방법

1. 특성 방정식 (Characteristic Equation) 작성

: $\det(A - \lambda I) = 0$ I : 단위 행렬(identity matrix), $\det$: 행렬의 행렬식(determinant) 이 방정식을 풀어 고유값 λ 을 찾는다.

2. 고유값 λ 에 대한 고유벡터 v 찾기

: 고유값을 구한 후, $A - \lambda I$ 의 해 공간(null space)을 구해 고유벡터를 찾는다.

특성 방정식 (Characteristic Equation)

주어진 $n \times n$ 정사각 행렬 A 에 대해, 특성 방정식은 다음과 같이 정의된다.

: $\det(A - \lambda I) = 0$

• $\det$: 행렬의 행렬식(determinant)

• λ : 고유값(eigenvalue)

• I : 동일한 차원의 단위 행렬(identity matrix)

특성 방정식의 유도 과정

1. 행렬 A 와 단위 행렬 I 정의: 행렬 A 가 $n \times n$ 행렬, I : $n \times n$ 단위 행렬

2. 행렬 $A - \lambda I$ 계산: 각 성분에 대해 A 에서 λ 를 뺀다. 이 때, λI 는 행렬 A 와 같은 차원의 대각선 성분에 λ 가 들어간 행렬이다.

3. 행렬 $A - \lambda I$ 의 행렬식(det) 계산

: 이 행렬의 행렬식을 계산하여 특성 방정식을 얻는다. 이는 다항식의 형태로 나타나며, 그 다항식을 characteristic polynomial이라고 한다.

4. 특성 방정식: 이 다항식을 0으로 두고, 다음과 같은 방정식을 얻게 된다.

$\det(A - \lambda I) = 0$

→ 이 방정식을 풀어 λ 값을 찾으면, 그 값들이 행렬 A 의 고유값이 된다.

$$A - \lambda I = \begin{pmatrix} a_{11} - \lambda & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} - \lambda & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} - \lambda \end{pmatrix}$$

고유값 분해, 주성분 분석(PCA) 등 여러 응용 분야에서 중요한 역할을 한다.

* 고유값(Eigenvalue)과 고유벡터(Eigenvector)

예시

$$Av = \lambda v$$

행렬 $A = \begin{pmatrix} 4 & 2 \\ 1 & 3 \end{pmatrix}$ 인 경우

1. $A - \lambda I$ 계산

$$A - \lambda I = \begin{pmatrix} 4 & 2 \\ 1 & 3 \end{pmatrix} - \lambda \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 4 - \lambda & 2 \\ 1 & 3 - \lambda \end{pmatrix}$$

2. 이 행렬의 행렬식 계산

$$\det(A - \lambda I) = \det \begin{pmatrix} 4 - \lambda & 2 \\ 1 & 3 - \lambda \end{pmatrix} = (4 - \lambda)(3 - \lambda) - 2 \times 1 = \lambda^2 - 7\lambda + 10$$

3. 특성 방정식

$$\lambda^2 - 7\lambda + 10 = 0$$

4. 고유값 (Eigenvalue) 찾기

이 방정식을 풀면, 행렬 A 의 고유값(eigenvalue) $\rightarrow \lambda = 5, 2$

5. 고유벡터(Eigenvector)

▪ 고유값 $\lambda_1 = 5$ 에 대한 고유벡터

고유벡터 $\mathbf{v}_1$ 는 다음 방정식을 만족한다:

$$A - 5I = \begin{pmatrix} 4 & 2 \\ 1 & 3 \end{pmatrix} - 5 \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} -1 & 2 \\ 1 & -2 \end{pmatrix}$$

$$\begin{pmatrix} -1 & 2 \\ 2 & -2 \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \text{ 만족하는 } v_1, v_2 \text{ 를 찾는다.}$$

$$-1 \cdot v_1 + 2 \cdot v_2 = 0, \quad 2 \cdot v_1 - 2 \cdot v_2 = 0$$

위 두식을 만족하는 v_1, v_2 를 계산하면

$$v_1 = 1, \quad v_2 = 1 \text{ 이므로}$$

고유벡터(Eigenvector)는 $\mathbf{v}_1 = k \begin{pmatrix} 1 \\ 1 \end{pmatrix}$, k : 상수

▪ 고유값 $\lambda_2 = 2$ 에 대한 고유벡터

• 고유값 $\lambda_1 = 5$ 에 대한 고유벡터 $\rightarrow \mathbf{v}_1 = k \begin{pmatrix} 1 \\ 1 \end{pmatrix}$

• 고유값 $\lambda_2 = 2$ 에 대한 고유벡터 $\rightarrow \mathbf{v}_2 = k \begin{pmatrix} -1 \\ 1 \end{pmatrix}$

Determinant (det) vs Eigenvalue

행렬식과 고유값의 관계

1) 특성 방정식

고유값은 특성 방정식 $\det(A - \lambda I) = 0$ 의 해입니다.

A : 행렬, λ : 고유값, I : 단위행렬.

특성 방정식의 근을 풀면 행렬 A 의 고유값을 구할 수 있다.

(2) 행렬식은 고유값의 곱

행렬 A 의 행렬식 $\det(A)$ 는 행렬의 고유값 $\lambda_1, \lambda_2, \dots, \lambda_n$ 의 곱과 같다.

$$\det(A) = \prod_{i=1}^n \lambda_i$$

즉, 고유값 중 하나라도 $\lambda_i = 0$ 이면, $\det(A) = 0$ 이고, 행렬 A 는 비가역적(singular)이다.

고유값이 모두 0이 아니면, $\det(A) \neq 0$ 이고, 행렬 A 는 가역적(nonsingular)이다.

Matrix:

[[4 1]

[2 3]]

Eigenvalues(고유값) 5, 2

[5. 2.]

Eigenvectors(고유벡터) : v1, v2

[[0.70710678 -0.4472136]

[0.70710678 0.89442719]]

$$\mathbf{v}_1 = \begin{bmatrix} 0.7071 \\ 0.7071 \end{bmatrix}, \quad \mathbf{v}_2 = \begin{bmatrix} -0.4472 \\ 0.8944 \end{bmatrix}$$

행렬식 계산

$$\det(A) = (4 \cdot 3) - (1 \cdot 2) = 12 - 2 = 10$$

특성 방정식

$$\det(A - \lambda I) = \det \left(\begin{bmatrix} 4 - \lambda & 1 \\ 2 & 3 - \lambda \end{bmatrix} \right) = 0$$

$$\det = (4 - \lambda)(3 - \lambda) - (1 \cdot 2) = \lambda^2 - 7\lambda + 10 = 0$$

고유값

$$\lambda^2 - 7\lambda + 10 = 0 \implies (\lambda - 5)(\lambda - 2) = 0$$

$$\lambda_1 = 5, \lambda_2 = 2$$

고유값의 곱

$$\lambda_1 \cdot \lambda_2 = 5 \cdot 2 = 10 = \det(A)$$

<https://github.com/freejyb/Deep-learning>
딥러닝(수학1)

Determinant (det) vs Eigenvalue

행렬식과 고유값의 관계

Matrix:

$\begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix}$

$\begin{bmatrix} 2 & 3 \end{bmatrix}$

Eigenvalues(고유값) 5, 2

$\begin{bmatrix} 5 & 2 \end{bmatrix}$

Eigenvectors(고유벡터) : v_1, v_2

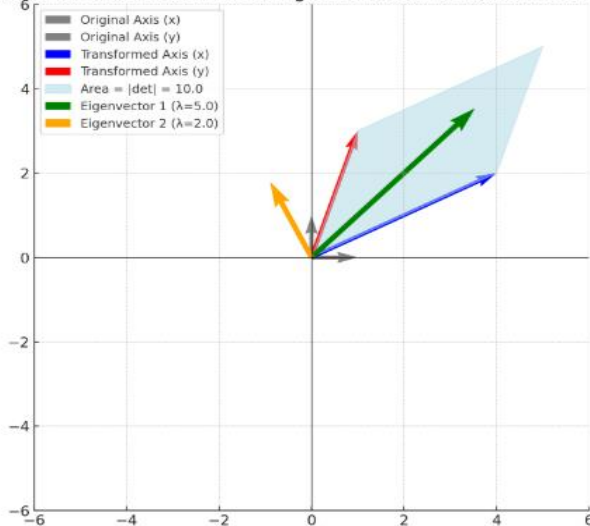
$\begin{bmatrix} 0.70710678 & -0.4472136 \end{bmatrix}$

$\begin{bmatrix} 0.70710678 & 0.89442719 \end{bmatrix}$

행렬식 계산

$$\det(A) = (4 \cdot 3) - (1 \cdot 2) = 12 - 2 = 10$$

Matrix Transformation: Axes, Eigenvectors, and Determinant Area



그래프 설명:

1. 회색 축: 원래의 x축과 y축을 나타낸다.

2. 변환된 축:

1. 파란색: 행렬 변환 후의 x축 방향.

2. 빨간색: 행렬 변환 후의 y축 방향.

3. 고유벡터:

■ 녹색: 첫 번째 고유벡터 (v_1)

■ 주황색: 두 번째 고유벡터 (v_2)

고유벡터는 변환 후에도 방향이 유지되는 벡터이다.

화살표의 길이는 대응되는 고유값 ($\lambda_1=5, \lambda_2=2$)에 따라 스케일링되었다.

4. 행렬식 영역:

밝은 파란색 영역: 변환된 축이 정의하는 평행사변형으로, 면적은 행렬식의 절댓값 $|\det| = 10$ 이다.

이 면적은 변환이 단위 정사각형의 면적을 어떻게 스케일링하는지를 나타낸다.

<https://github.com/freejyb/Deep-learning>
딥러닝(수학1)

고유값과 고유벡터를 통해 행렬의 변환 특성을 분석하며, 행렬식은 고유값들의 종합적 정보를 제공한다.

* 고유값(Eigenvalue)과 고유벡터(Eigenvector)

활용 사례

1. 주성분 분석 (Principal Component Analysis, PCA)

PCA는 데이터의 차원을 축소하는 데 사용되는 기법이다.
데이터의 분산이 최대인 방향을 찾기 위해 고유값과 고유벡터를 사용한다.

(주요 단계)

- 데이터 공분산 행렬을 계산한다.
- 공분산 행렬의 고유값과 고유벡터를 구한다.
- 가장 큰 고유값에 해당하는 고유벡터를 선택하여 새로운 축을 형성한다. 이를 통해 데이터의 주요 특징을 유지하면서 차원을 줄일 수 있다.

2. 진동 해석 (Vibration Analysis)

구조물이나 기계 시스템의 진동 모드를 분석하는 데 사용된다.
시스템의 질량 행렬과 강성 행렬을 통해 고유값 문제를 설정하고 고유값과 고유벡터를 구하면 각 진동 모드와 고유 주파수를 알 수 있다.
이는 구조물의 안정성을 평가하고 설계하는 데 중요한 역할을 한다.

3. 얼굴 인식 (Face Recognition)

얼굴 인식 시스템에서는 고유얼굴(eigenfaces)이라는 개념을 사용한다. 얼굴 이미지를 고유얼굴로 표현하여 차원을 축소하고, 새로운 얼굴 이미지를 비교하여 인식한다.
이를 통해 고유값과 고유벡터를 이용해 얼굴 데이터를 효율적으로 분석할 수 있다.

4. 구글 페이지랭크 (Google PageRank)

구글의 페이지랭크 알고리즘은 웹 페이지의 중요도를 평가하는 방법이다. 웹 그래프의 행렬 표현을 통해 고유값 문제를 설정하고, 가장 큰 고유값에 해당하는 고유벡터를 구하면 각 페이지의 순위를 결정할 수 있다.

5. 양자역학 (Quantum Mechanics)

양자역학에서는 고유값과 고유벡터를 이용해 시스템의 에너지 상태를 분석한다. 슈뢰딩거 방정식의 해를 구하면 시스템의 에너지 고유값과 이에 해당하는 파동 함수(고유벡터)를 얻을 수 있다.

벡터와 행렬 : 머신러닝 활용

- 벡터(Vector)와 행렬(Matrix)은 머신러닝과 딥러닝의 핵심 수학적 도구로, *데이터 표현, 모델 학습, 최적화 과정 등에서* 광범위하게 사용된다.
- 이들은 데이터와 가중치, 연산을 효율적으로 처리하기 위한 수학적 구조로, 모델의 성능과 속도를 크게 좌우한다.

1. 데이터 표현 (Data Representation)

벡터와 행렬의 연산을 이용하면 대량의 데이터에 대한 계산을 간단한 수식으로 표시

벡터

- 특징 벡터(feature vector)를 나타내는 데 사용
- 각 특징 벡터는 입력 데이터를 숫자로 변환하여 모델에 전달하는 구조
- 예를 들어, 이미지의 각 픽셀 값이나 문장의 각 단어를 수치 벡터로 변환

행렬

여러 개의 데이터 포인트를 동시에 표현할 때 사용

예를 들어, 데이터셋의 각 행이 데이터 포인트, 각 열이 해당 데이터 포인트의 특징을 나타낸다.

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1m} \\ x_{21} & x_{22} & \cdots & x_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nm} \end{bmatrix}$$

X 는 $n \times m$ 크기의 데이터 행렬로, n 은 샘플의 개수, m 은 피처의 개수

• **벡터화 연산**: 데이터를 행렬로 표현하면 반복문 없이 벡터화 연산으로 병렬 처리가 가능하다. 이는 **NumPy**와 같은 라이브러리에서 빠른 계산을 가능하게 한다.

벡터와 행렬 : 머신러닝 활용

2. 선형 회귀 (Linear Regression)

벡터와 행렬은 선형 회귀(Linear Regression) 모델에서 입력 데이터를 가중치 벡터와 곱하는 데 사용된다.

예측값 $\hat{y}$ 는 입력 데이터 x 와 가중치 벡터 $\mathbf{w}$ 의 행렬 곱으로 계산된다

$$\hat{\mathbf{y}} = X\mathbf{w}$$

$X: n \times m$ 행렬 (입력 데이터)

$\mathbf{w}: m \times 1$ 벡터 (가중치)

$\hat{\mathbf{y}}: n \times 1$ 벡터 (예측값)

손실 함수 계산

손실 함수(예: 평균 제곱 오차, MSE)는 행렬 연산으로 간단히 계산할 수 있다

$$\text{MSE} = \frac{1}{n}(\hat{\mathbf{y}} - \mathbf{y})^T(\hat{\mathbf{y}} - \mathbf{y})$$

3. 신경망과 딥러닝 (Neural Networks and Deep Learning)

뉴럴 네트워크 (신경망)

- 뉴럴 네트워크는 각 계층의 노드들이 벡터와 행렬로 연결된다.
- 입력층에서 전달된 데이터를 가중치 행렬과 곱하여 은닉층 또는 출력층으로 전달한다.

순환 신경망 (RNN)

RNN에서는 입력 벡터가 시간에 따라 변화하는 데이터(시계열 데이터)를 처리하며, 가중치 행렬을 사용하여 이전 상태에서 받은 정보를 현재 상태로 전달한다.

역전파 (Backpropagation):

역전파 알고리즘에서 **체인 룰**과 **행렬 미분**을 사용해 가중치의 그라디언트를 계산한다. 이 그라디언트는 경사 하강법을 통해 가중치를 업데이트하는 데 사용된다.

$$\mathbf{z}^{(l)} = W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

$W^{(l)}$: 가중치 행렬 (layer l)

$\mathbf{a}^{(l-1)}$: 이전 층의 활성화 값 (벡터)

$\mathbf{b}^{(l)}$: 편향(bias) 벡터

$\mathbf{z}^{(l)}$: 선형 변환 결과
이전 상태에

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla L(\mathbf{w})$$

4. 차원 축소 (Dimensionality Reduction)

주성분 분석 (PCA, Principal Component Analysis)

- PCA는 데이터의 분산을 최대화하는 방향으로 차원을 축소한다.
- 데이터 행렬 X 를 공분산 행렬 C 로 변환하고, 고유값 분해(Eigen decomposition) 또는 특이값 분해(SVD)를 통해 주성분을 찾는다.

$$C = \frac{1}{n} X^T X$$

X 의 주성분은 공분산 행렬 C 의 고유벡터(eigenvector)로 나타난다.

선형 판별 분석 (LDA)

LDA는 클래스 간의 분산을 최대화하고, 클래스 내의 분산을 최소화하는 차원 축소 기법이다. 행렬 연산으로 클래스 간의 분산 행렬과 클래스 내의 분산 행렬을 계산한다.

*Singular Value Decomposition, SVD

- **특이값 분해(Singular Value Decomposition, SVD)** : 행렬을 세 개의 행렬(U, Σ, V^T)로 분해하여 원래 행렬의 특성을 분석하는 방법
- SVD는 선형대수에서 중요한 역할을 하며, 특히 데이터 압축, 차원 축소, 잡음 제거 등 다양한 분야에 널리 사용

SVD의 정의

특이값 분해는 다음과 같은 형태로 주어진 임의의 행렬 A 를 세 개의 행렬로 분해한다. → 각각은 원래 행렬의 구조와 방향성을 분리하여 데이터의 특성을 효과적으로 분석할 수 있게 해준다..

$$A = U\Sigma V^T$$

- A : $m \times n$ 행렬로, 임의의 데이터.
- U : $m \times m$ 직교 행렬(orthogonal matrix) 이 행렬의 열벡터들은 왼쪽 특이 벡터(left singular vectors)라고 불린다.
- V : $n \times n$ 직교 행렬,, 이 행렬의 열벡터들은 오른쪽 특이 벡터(right singular vectors)라고 한다.
- Σ : $m \times n$ 대각 행렬(diagonal matrix), 대각 성분은 특이값(singular values)라고 하며, 이를 크기 순으로 나열한다.

이 분해를 통해 행렬 A 를 이해하기 쉽게 여러 부분으로 나눌 수 있으며, 각각의 행렬이 특정한 의미를 가진다.

SVD의 구성 요소

1) 왼쪽 특이 벡터 (Columns of U)

- 행렬 U 의 열은 원래 행렬의 행 공간(row space)과 관련된 정보를 포함한다.
- 각 특이값에 대응하는 성분이 주 데이터에 어느 정도 영향을 미치는지를 나타낸다.

2) 특이 값 (Singular Values) Σ

- 행렬 Σ 의 대각선 요소들은 특이값이다. 특이값들은 행렬의 크기나 구조에 관한 중요한 정보를 포함하고 있으며, 이를 통해 행렬의 랭크(rank)와 같은 특성을 파악할 수 있다.
- 특이값이 0이 아닌 값의 개수는 행렬의 유효 차원을 결정하는 데 도움을 준다.

3) 오른쪽 특이 벡터 (Columns of V^T)

- 행렬 V 의 열은 열 공간(column space)에 관한 정보를 제공한다.
- 이는 데이터의 기저(base)를 형성하며, 데이터의 중요한 특성을 나타낸다.

*Singular Value Decomposition, SVD

SVD의 직관적 이해

$$A = U\Sigma V^T$$

- 특이값 분해를 사용하면 행렬을 더 단순한 형태로 나누어 각각의 특성을 분석할 수 있다.
(직관적으로, 행렬 A 해석)
- 행렬 U : 이 스케일된 데이터를 최종 좌표계에 맞춰 회전시킨다. → 데이터가 새로운 좌표계에서 **행 공간에서** 어떻게 회전되고 분포되는지 설명한다.
- 행렬 Σ : 데이터의 스케일을 조정한다 (즉, 각각의 방향에 대한 영향력의 크기를 나타낸다). → 큰 특이값은 해당 방향이 데이터의 설명에서 중요한 축임을 나타낸다.
- 행렬 V^T : 원래 데이터의 방향을 정의한다. → 데이터의 **열 공간에서** 벡터들이 어떻게 배치되고 회전되는지를 나타냅니다.

SVD의 주요 응용

- **차원 축소 (Dimensionality Reduction)**
SVD는 차원 축소 기법인 주성분 분석(PCA, Principal Component Analysis)의 기본이 된다.
특이값의 크기를 기준으로 작은 값을 무시하면 데이터의 중요한 정보만 남기고 차원을 줄일 수 있다.
- **잡음 제거 (Noise Reduction)**
이미지나 신호에서 잡음을 제거할 때, 특이값 중 작은 값을 무시하면 원본의 중요한 구조를 유지하면서 잡음 성분을 제거할 수 있다.
- **추천 시스템 (Recommendation System)**
추천 시스템에서 행렬 인수분해(Matrix Factorization) 기법으로 사용되어 사용자-아이템 평점 행렬을 근사하는 데 도움을 준다.

SVD와 기저 벡터의 관계

- U 와 V 의 열 벡터들은 각각 기저 벡터 역할을 한다.
- 행렬 U 의 열 벡터들은 왼쪽 기저 벡터로서, 행렬의 행 공간(row space)을 나타낸다.
- 행렬 V 의 열 벡터들은 오른쪽 기저 벡터로서, 행렬의 열 공간(column space)을 나타낸다.
- 이러한 기저 벡터들은 행렬이 표현하는 데이터를 새로운 공간에서 재해석할 수 있는 기초를 제공한다.

*Singular Value Decomposition, SVD

행렬 A의 SVD 형태 : 사례

1. 원래 행렬 A

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$$

$$A = U\Sigma V^T$$

A 행렬은 3x2 크기의 행렬
→ m x n의 경우

A: 3 × 2

U: 3 × 3 (왼쪽 특이 벡터 행렬)

Σ: 3 × 2 (대각 특이값 행렬)

V^T: 2 × 2 (오른쪽 특이 벡터의 전치 행렬)

2. 왼쪽 특이 벡터 (U)

U는 왼쪽 특이 벡터들을 포함하는 행렬로, 원래 행렬 A의 행 공간(row space)과 관련된 정보를 가지고 있다.

• U의 크기는 3x3이다. → U는 원래 행렬의 행과 같은 수를 가지는 정사각 행렬이다. (m x m 행렬)

• U는 직교 행렬(orthogonal matrix)로, 이 행렬의 열 벡터들은 서로 직교하고 길이가 1이다.

• 이 벡터들은 원래 행렬 A를 새로운 좌표계로 변환하는 역할을 한다.

예를 들어, U의 열들은 원래 데이터가 회전 및 변환된 공간에서 어떻게 나타나는지를 보여준다.

$$U = \begin{bmatrix} -0.2298 & 0.8835 & 0.4082 \\ -0.5247 & 0.2408 & -0.8165 \\ -0.8196 & -0.4019 & 0.4082 \end{bmatrix}$$

$$\Sigma = \begin{bmatrix} 9.5255 & 0 \\ 0 & 0.5143 \\ 0 & 0 \end{bmatrix}$$

3. 특이값 (Σ)

Σ는 특이값(singular values)으로 이루어진 대각 요소로 특이값을 가지며, 나머지 요소는 0으로 채워진 대각 행렬이다.

실제 Python의 np.linalg.svd()는 이를 벡터로 반환한다: [9.5255, 0.5143]. → m x n 행렬

특이값은 행렬 A가 각 축 방향으로 얼마나 "늘어나거나 줄어드는지"를 나타낸다.

▪ 첫 번째 특이값 9.5255 → 9.5255는 가장 큰 특이값으로, 이는 원래 행렬의 주요 정보(에너지)가 이 방향으로 많이 분포되어 있음을 나타낸다.

▪ 두 번째 특이값 0.5143 → 0.5143는 첫 번째 특이값에 비해 상대적으로 작으며, 이는 해당 축에서 데이터의 기여가 작다는 것을 의미한다. : 특이값이 작을수록 그 축의 정보가 덜 중요하다는 의미이다. → 따라서 작은 특이값은 무시할 수 있으며, 이를 통해 차원 축소를 할 수 있다.

3. 오른쪽 특이 벡터 (V^T)

▪ 행렬 V^T는 오른쪽 특이 벡터(right singular vectors)로 이루어진 전치행렬이며, 행렬 A의 열 공간(column space)과 관련된 정보를 제공합니다. → 원래 행렬의 열과 같은 수를 가진다. n x n 행렬

▪ 행렬 V^T는 A의 열 벡터들이 정의하는 새로운 좌표계를 나타냅니다.

▪ 이 행렬의 행 벡터들은 원래 행렬 A를 표현하기 위해 중요한 기저 벡터들이다.

▪ V^T의 벡터들은 데이터가 각 축을 기준으로 회전 및 변환되는 방식을 정의하며, 열 벡터들이 새로운 좌표계에서 어떻게 표현될지를 나타낸다.

$$V^T = \begin{bmatrix} -0.6196 & -0.7849 \\ -0.7849 & 0.6196 \end{bmatrix}$$

Linear Algebra : Matrix

행렬의 미적분학

미분과 적분은 행렬 자체는 아니지만, **행렬**과 관련된 개념을 포함할 수 있으며, 특히 **선형 대수학**과 **미적분학**의 접점에서 중요한 역할을 한다.

https://en.wikipedia.org/wiki/Matrix_calculus

Types of matrix derivative

Types	Scalar	Vector	Matrix
Scalar	$\frac{\partial y}{\partial x}$	$\frac{\partial \mathbf{y}}{\partial x}$	$\frac{\partial \mathbf{Y}}{\partial x}$
Vector	$\frac{\partial y}{\partial \mathbf{x}}$	$\frac{\partial \mathbf{y}}{\partial \mathbf{x}}$	
Matrix	$\frac{\partial y}{\partial \mathbf{X}}$		

- **스칼라 함수**: 그 결과값이 1차원의 값인 함수
- **벡터함수**: 결과값이 다차원인 함수

1. 벡터 미분과 행렬 (Vector Calculus)

- 1) 스칼라 함수를 벡터 변수로 미분할 때, **그레디언트 벡터**가 사용된다.
 • 예시: $f(\mathbf{x})$ 가 벡터 $\mathbf{x}$ 에 대한 스칼라 함수일 때, $\nabla f(\mathbf{x})$ 는 그레디언트 벡터이다.

그레디언트 (Gradient): 스칼라 함수 $f(\mathbf{x})$ 가 벡터 $\mathbf{x}$ 의 함수일 때, 이 함수의 편미분을 모은 벡터를 **그레디언트**라고 한다.

- 2) 벡터 함수를 벡터로 미분할 때, 결과는 야코비안 행렬 (Jacobian Matrix)이 된다.
 예시: $\mathbf{f}(\mathbf{x})$ 가 벡터 함수일 때, 그 미분은 야코비안 행렬 $J = \partial \mathbf{x} / \partial \mathbf{f}$ 이다.

$$\frac{\partial y}{\partial \mathbf{x}} = \left[\frac{\partial y}{\partial x_1} \quad \frac{\partial y}{\partial x_2} \quad \dots \quad \frac{\partial y}{\partial x_n} \right]$$

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \dots & \frac{\partial y_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_m}{\partial x_1} & \dots & \frac{\partial y_m}{\partial x_n} \end{bmatrix}$$

델 연산자 (∇)

Linear Algebra : Matrix

행렬의 미적분학

미분과 적분은 행렬 자체는 아니지만, **행렬**과 관련된 개념을 포함할 수 있으며, 특히 **선형 대수학과 미적분학**의 접점에서 중요한 역할을 한다.

https://en.wikipedia.org/wiki/Matrix_calculus

2. 행렬 미분 (Matrix Calculus)

- 행렬이나 벡터를 변수로 하는 함수의 미분을 행렬 미분 (Matrix Calculus)라고 한다.
 - 예시: $f(\mathbf{x})$ 가 행렬 $\mathbf{x}$ 를 변수로 가지는 함수일 때, 미분 결과는 그라디언트 또는 헤시안 행렬이 된다.
 - **헤시안 행렬 (Hessian Matrix)**: 2차 미분의 결과로, 함수의 곡률 정보를 담고 있다.

3. 적분과 행렬 (Matrix Integration)

- 적분에서도 **선형 대수학과 미적분학**의 관계가 존재한다.
 - **행렬 함수의 적분**은 주로 확률과 통계에서 사용되며, 다변량 정규분포의 확률 밀도 함수를 적분할 때 행렬 연산이 포함된다.
 - **확률론**에서, 행렬 형태의 적분은 베이저안 추론과 같은 분야에서 자주 나타난다.

4. 미분 방정식과 행렬 (Differential Equations and Matrices)

- **선형 미분 방정식**에서는 미분 연산이 행렬로 표현되기도 한다.
 - 예시: 시스템의 상태 벡터 $\mathbf{x}$ 에 대해 $\mathbf{x}' = \mathbf{A}\mathbf{x}$ 는 행렬 $\mathbf{A}$ 를 사용한 미분 방정식이다.

5. 선형 대수학과 적분 변환 (Linear Algebra and Integral Transforms)

- **적분 변환** (예: 라플라스 변환, **푸리에 변환**)에서는 행렬 연산이 필수적이다.
 - 예시: **푸리에 변환**에서 행렬 연산은 디지털 신호 처리 (DSP)에서 필터링과 같은 작업에 활용된다.

Linear Algebra : Matrix

행렬의 미적분학

야코비안 행렬 (Jacobian Matrix)

야코비안 행렬은 **벡터 값 함수**의 모든 1차 편미분을 포함한 행렬이다.

벡터 함수 $\mathbf{f}(\mathbf{x})=[f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_m(\mathbf{x})]$ 가 n -차원 벡터 $\mathbf{x}=[x_1, x_2, \dots, x_n]$ 의 함수일 때, 야코비안 행렬은 다음과 같이 정의된다.

$$J(\mathbf{x}) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \dots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \dots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \dots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

•크기: $m \times n$ 행렬

•용도: 기계 학습에서 역전파(backpropagation), 최적화 문제 등에 사용된다.

해시안 행렬 (Hessian Matrix)

해시안 행렬은 **스칼라 값 함수**의 2차 편미분으로 구성된 **대칭 행렬**이다.

스칼라 값 함수 $f(\mathbf{x})$ 가 n -차원 벡터 $\mathbf{x}=[x_1, x_2, \dots, x_n]$ 의 함수일 때, 해시안 행렬은 다음과 같이 정의한다.

$$H(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \dots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \dots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \dots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

•크기: $n \times n$ 대칭 행렬

•용도: 함수의 곡률 분석, 최적화 문제에서의 극값 판별 등에서 사용된다.

야코비안 행렬과 해시안 행렬의 차이점

특징	야코비안 행렬	해시안 행렬
정의	1차 편미분	2차 편미분
적용 대상	벡터 값 함수	스칼라 값 함수
크기	$m \times n$	$n \times n$ 대칭 행렬
용도	함수의 선형 근사, 역전파	곡률 분석, 최적화

* Covariance

같이 학습하기!!

공분산(Covariance)

- 두 확률변수 X, Y 사이의 선형관계가 어떤 **방향**으로 이루어지는지를 표현하는 수치
- 공분산이 양수이면 X 가 증가할 때 Y 도 증가하는 + 선형관계
- 음수이면 X 가 증가할 때 Y 는 감소하는 - 선형관계,
- 0이면 선형관계가 없다는 것을 의미
- 공분산 값의 범위 ; 무한대

표본집단
공분산

$$S_{XY} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{n-1}$$

표본 분산

$$S^2 = \frac{\sum_{i=1}^n (X - \bar{X})(X - \bar{X})}{n-1}$$

공분산은 제곱이 없고 (부호 +, -), 표본분산은 제곱이라는 점 (부호 +)

모집단
공분산

$$\sigma_{XY} = \frac{\sum_{i=1}^n (x_i - \mu_x)(y_i - \mu_y)}{N}$$

공분산은 x, y 의 척도단위에 민감하게 영향을 받는다. 즉, 상관관계 강도와 상관없이 단위가 크면 큰 값이 나온다는 것이다.

→ 데이터 표준화가 필요 !!!

- 공분산이 0라고 해서 두 확률변수가 아무 관계가 없다(상호 독립이다)는 의미가 아니라는 것이다.
- 공분산이 0인 것은 변수간에 **선형의 관계만 없다는 것을 의미**
- → 두 변수가 상호 독립이면 공분산은 0이 되지만 공분산이 0이라고 해서 두 변수가 상호 독립인 것은 아니다.

* Covariance Matrix

공분산 행렬(covariance matrix)

- 여러 변수 간의 공분산을 요소로 하는 행렬
- 이 행렬은 데이터 세트의 변수들 사이의 선형 관계를 이해하는 데 사용
- 각 요소는 두 변수 사이의 공분산을 나타내며, 대각선 요소는 각 변수의 분산을 나타낸다.

공분산 행렬의 정의

n개의 변수를 가진 데이터 세트가 있다고 가정할 때, 공분산 행렬 C 정의

$$C = \begin{bmatrix} \text{Var}(X_1) & \text{Cov}(X_1, X_2) & \cdots & \text{Cov}(X_1, X_n) \\ \text{Cov}(X_2, X_1) & \text{Var}(X_2) & \cdots & \text{Cov}(X_2, X_n) \\ \vdots & \vdots & \ddots & \vdots \\ \text{Cov}(X_n, X_1) & \text{Cov}(X_n, X_2) & \cdots & \text{Var}(X_n) \end{bmatrix}$$

공분산 행렬의 성질

- 대칭성:** 공분산 행렬은 대칭 행렬이므로, $C = C^T$
- 대각선 요소:** 대각선 상의 요소는 **각 변수의 분산을 나타낸다.**
- 크기와 단위:** 공분산의 크기와 단위는 원본 데이터의 단위에 의존적이다. → 변수들이 서로 다른 단위를 가지고 있을 때 해석하기 어려울 수 있다. 이런 경우, 상관 행렬을 사용하는 것이 더 유용할 수 있다.

활용

- 공분산 행렬은 다변량 데이터의 특성을 분석하고, 주성분 분석(PCA) 같은 차원 축소 기법, 다변량 분석, 금융에서 포트폴리오의 위험 평가 등 다양한 분야에서 활용
- 데이터의 분포를 이해하고, 변수 간의 관계를 파악하는 데 중요한 도구

Text similarity** NLP 유사도**

$$\vec{a} \cdot \vec{b} = \|\vec{a}\| \|\vec{b}\| \cos\theta$$

1. 벡터 유사도 : cosine Similarity

$$= \cos \theta = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \|\vec{b}\|}$$

→ 각도기반 유사도 → 0과 1사이의 값

→ 해석: 유사도가 1에 가까울수록 (클수록) 두 벡터는 비슷한 방향이다.

: 두 벡터의 방향이 같을수록 유사하다고 판단

2. 벡터 유사도 : Euclidean metric

→ 거리 기반 유사도, 두 벡터간 유클리드 거리가 작을수록 유사하다.

$$\vec{a} = (a_1, a_2), \vec{b} = (b_1, b_2)$$

$$\text{dist} = (\vec{a}, \vec{b}) = \|\vec{a} - \vec{b}\| = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2}$$

NLP : 문서 유사도 분석에서 어느 유사도를 사용해야 할까?

스케일의 차이가 크기에 따라 판단 !!!

- 스케일 차이가 크지 않다 → 거리 기반 유사도 (유클리디안 유사도)
- 스케일 차이가 클 경우 → 각도 기반 유사도 (코사인 유사도)

*Text similarity

1. NLP 유사도 : 코사인 유사도

$$\vec{a} \cdot \vec{b} = \|\vec{a}\| \|\vec{b}\| \cos \theta$$

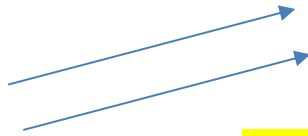
$$\text{벡터 유사도 : cosine metric} = \cos \theta = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \|\vec{b}\|}$$



(1) $\theta = 90^\circ$: 내적 값은 0

$$\vec{a} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad \vec{b} = \begin{pmatrix} 0 \\ 3 \end{pmatrix}$$

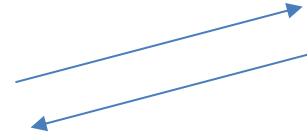
$$\vec{a} \cdot \vec{b} = 0$$



(2) $\theta = 0^\circ$: 내적 값은 1

$$\vec{a} = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \quad \vec{b} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

$$\vec{a} \cdot \vec{b} = 1$$



(3) $\theta = 180^\circ$: 내적 값은 -1

$$\vec{a} = \begin{pmatrix} 1 \\ -1 \end{pmatrix} \quad \vec{b} = \begin{pmatrix} -1 \\ 1 \end{pmatrix}$$

$$\vec{a} \cdot \vec{b} = -1$$

α°	$\alpha \text{ rad}$	$\sin \alpha$	$\cos \alpha$	$\tan \alpha$	$\cot \alpha$	$\sec \alpha$	$\operatorname{cosec} \alpha$
0	0	0	1	0	∞	1	∞
30	$\frac{\pi}{6}$	$\frac{1}{2}$	$\frac{\sqrt{3}}{2}$	$\frac{1}{\sqrt{3}}$	$\sqrt{3}$	$\frac{2}{\sqrt{3}}$	2
45	$\frac{\pi}{4}$	$\frac{\sqrt{2}}{2}$	$\frac{\sqrt{2}}{2}$	1	1	$\sqrt{2}$	$\sqrt{2}$
60	$\frac{\pi}{3}$	$\frac{\sqrt{3}}{2}$	$\frac{1}{2}$	$\sqrt{3}$	$\frac{1}{\sqrt{3}}$	2	$\frac{2}{\sqrt{3}}$
90	$\frac{\pi}{2}$	1	0	∞	0	∞	1
120	$\frac{2\pi}{3}$	$\frac{\sqrt{3}}{2}$	$-\frac{1}{2}$	$-\sqrt{3}$	$-\frac{1}{\sqrt{3}}$	-2	$\frac{2}{\sqrt{3}}$
180	π	0	-1	0	∞	-1	∞
270	$\frac{3\pi}{2}$	-1	0	∞	0	∞	-1
360	2π	0	1	0	∞	1	∞

*Text similarity

1. NLP 유사도 : 코사인 유사도

코사인 유사도(Cosine Similarity)

(의의)

- 두 벡터 사이의 각도를 계산하여 두 벡터가 얼마나 유사한지 측정하는 척도
- DTM, TF-IDF, Word2Vec 등과 같이 단어를 수치화하여 표현할 수 있다면 코사인 유사도를 활용하여 문서 간 유사도를 비교하는 게 가능

(해석)

코사인 유사도는 1에 가까울수록 두 벡터가 유사하다고 해석

(장점)

문서의 길이가 다른 경우에도 비교적 공정하게 비교할 수 있다는 장점

*Text similarity

1. NLP 유사도 : 코사인 유사도

문서1 나는 일본여행보다 베트남여행이 좋다
 문서2 운동은 아침보다 저녁이 좋다
 문서3 나는 일본여행보다 저녁이 좋다

띄어쓰기를 기준으로 토큰화를 진행 가정
 문서-단어 행렬(Document Term Matrix, DTM)을 표현

	나는	일본여행보다	베트남여행이	좋다	운동은	아침보다	저녁이
문서1	1	1	1	1	0	0	0
문서2	0	0	0	1	1	1	1
문서3	1	1	0	1	0	0	1

Python 결과

1. 문서1-문서2 간 유사도: 0.25

2. 문서1-문서3 간 유사도: 0.75

3. 문서2-문서3 간 유사도: 0.5

*Text similarity

2. NLP 유사도 : 유클리드 유사도

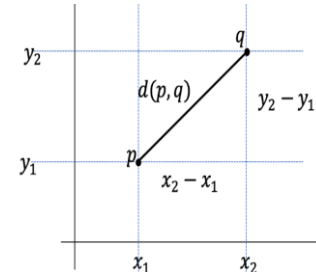
$$\vec{a} \cdot \vec{b} = \|\vec{a}\| \|\vec{b}\| \cos\theta$$

벡터 유사도 : Euclidean metric 두 벡터 사이의 거리를 측정하는데 이용

유클리드 거리(Euclidean Distance): 두 점 사이의 거리를 계산하는 기법

$$\vec{a} = (x_1, y_1), \vec{b} = (x_2, y_2)$$

$$\text{dist} = (\vec{a}, \vec{b}) = \|\vec{a} - \vec{b}\| = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$



(활용)

유클리드 거리를 활용하여 문서 간 유사도를 계산

문서 간 유클리드 거리가 가까울수록 해당 문서들이 서로 유사하다고 할 수 있다.

코사인 유사도(Cosine Similarity)와 달리, 유클리드 거리는 비교하는 문서 간 길이에 영향을 받는다는 한계가 있다.

*Text similarity

2. NLP 유사도 : 유클리드 유사도

$$\vec{a} \cdot \vec{b} = \|\vec{a}\| \|\vec{b}\| \cos\theta$$

벡터 유사도 : **Euclidean metric** 두 벡터 사이의 거리를 측정하는데 이용

유클리드 거리가 가까울수록 문서 간 유사하다고 해석

문서1	나는 일본여행보다 베트남여행이 좋다
문서2	운동은 아침보다 저녁이 좋다
문서3	나는 일본여행보다 저녁이 좋다

	나는	일본여행보다	베트남여행이	좋다	운동은	아침보다	저녁이
문서1	1	1	1	1	0	0	0
문서2	0	0	0	1	1	1	1
문서3	1	1	0	1	0	0	1

1. 문서1-문서2 간 유사도: 2.449489742783178

2. 문서1-문서3 간 유사도: 1.4142135623730951

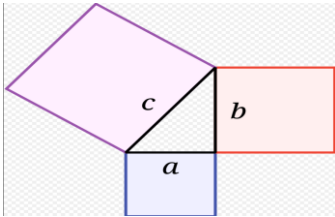
3. 문서2-문서3 간 유사도: 2.0

<https://github.com/freejyb/Deep-learning>
딥러닝(수학2)

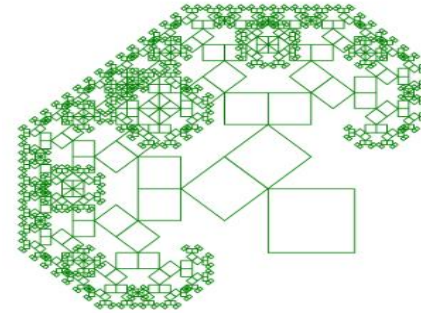
* Pythagoras' theorem

피타고라스의 정리 Pythagoras' theorem

직각삼각형에서 빗변 길이의 제곱은 다른 두 변의 길이의 제곱의 합과 같다



<https://youtu.be/PLMAULYCNeo?si=GzrXbpwhqb9DLz88>



<https://github.com/freejyb/Deep-learning/>
딥러닝(수학3)

유클리드 거리(Euclidean distance)

- 두 점 사이의 거리를 계산할 때 흔히 쓰는 방법
- 이 거리를 사용하여 유클리드 공간을 정의할 수 있으며, 이 거리에 대응하는 노름을 유클리드 노름(Euclidean norm)이라고 부른다.

직교 좌표계의 두 점 (x_1, y_1) , (x_2, y_2)

$$\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Linear vs Non-linear

선형성이 만족되는 시스템이나 함수는

다음 두 가지 중요한 특성인 **중첩성(superposition)**과 **동차성(homogeneity)**을 만족한다.

1. 중첩성 (Superposition)

- **중첩성**은 두 개 이상의 입력이 주어졌을 때의 **출력의 합**이 각 입력에 대한 **출력의 합**과 같다는 것을 의미한다.
- 즉, 시스템이 선형성을 만족하면 입력을 합쳤을 때의 출력은 각 입력에 독립적으로 적용된 출력들의 합과 동일하다.

$$f(x_1 + x_2) = f(x_1) + f(x_2)$$

두 입력을 합친 후 시스템에 적용한 결과가 각 입력을 따로 적용한 후 더한 것과 같음을 의미한다.

2. 동차성 (Homogeneity)

- **동차성**(또는 **스칼라 곱의 선형성**)은 입력에 어떤 **스칼라(scalar)** 값을 곱하면, 출력도 동일한 스칼라로 곱해진다는 것을 의미한다.
- 즉, 선형 시스템은 입력이 스칼라 값에 의해 변할 때 출력도 같은 비율로 변한다.

$$f(\alpha x) = \alpha f(x)$$

입력에 α 를 곱했을 때, 출력에도 동일한 α 가 곱해짐을 의미한다.

선형 시스템의 정의

선형성을 만족하는 시스템은 위의 중첩성과 동차성 두 가지 성질을 모두 만족해야 한다.

이 두 가지 성질은 선형 시스템이나 선형 함수가 **입력과 출력 사이의 비례 관계**를 유지하게 만든다.

• **중첩성**: 여러 입력의 합에 대한 출력이 각각의 입력에 대한 출력의 합과 동일하다.

• **동차성**: 입력이 어떤 스칼라 배로 증가하면 출력도 동일한 스칼라 배로 증가한다.

이러한 성질 때문에 선형 시스템은 계산이 간단하고 예측이 쉬운 특성을 가지게 된다.

Linear vs Non-linear

예시: 선형 회귀

선형 회귀 모델 $y = w_1 x_1 + w_2 x_2 + b$ 는 **선형성**을 만족한다.

중첩성과 동차성 모두 만족한다.

예시: 비선형 시스템

비선형 함수 $y = x^2$ 는 선형성이 **성립하지 않는다**.

중첩성 실패: $f(x_1 + x_2) = (x_1 + x_2)^2$ 는 일반적으로 $x_1^2 + x_2^2$ 와 일치하지 않기 때문에 중첩성이 성립하지 않는다.

동차성 실패: $f(ax) = (ax)^2 = a^2 x^2$ 이므로 $af(x) = ax^2$ 과 다르다. 결국 동차성이 성립하지 않는다.

- **선형 시스템**은 **중첩성과 동차성** 두 가지 성질을 만족하는 시스템으로, 입력과 출력의 관계가 단순하고 예측하기 쉽다.
- 이는 많은 수학적 계산에서 유용하며, 선형 회귀와 같은 머신러닝 알고리즘에서 이러한 성질이 활용된다.
- **비선형 시스템**은 이러한 성질을 만족하지 않으며, 더 복잡하고 다양한 동작을 나타내는 경우가 많다.

- 머신러닝에서 "**선형**" 모델은 반드시 **2차원 공간의 직선**만을 의미하는 것이 아니라, **여러 입력 변수들의 선형 결합**을 의미한다. 이는 고차원 공간에서 **평면**이나 **초평면**을 포함한다.
- 머신러닝에서 **선형**이라는 용어는 반드시 **직선**을 의미하지는 않는다. 보다 정확히는, **선형 모델**은 **입력 변수들의 선형 결합**으로 표현된다는 의미이다. 이는 **직선뿐만 아니라 고차원 공간에서의 평면이나 초평면**을 포함하는 개념이다. 따라서, 선형 모델의 결과는 2차원에서는 직선일 수 있지만, 더 높은 차원의 공간에서는 평면이나 초평면으로 나타난다.

Linear Model / Non-linear Model

Linear Model

- 선형 모델은 **계수간의 관계가 선형 결합으로 표현된다는 의미**
- 입력 변수를 비선형 변환한 후에 선형 모델을 적용하면 곡선 형태의 관계도 나타낼 수 있다.
- 따라서, **선형 모델은 반드시 직선만 나타내는 것은 아니며, 입력 변환을 통해 다양한 형태의 곡선을 나타낼 수 있다.**

선형회귀모델: 회귀계수(파라미터)를 선형결합으로 표현할 수 있는 모델 → 주어지는 입력변수(x)가 1차식이나, 2차식이나 상관 없음

비선형 모델 (Non-linear Model)

개념

- 비선형 모델은 입력 변수와 출력 변수 간의 관계가 직선이 아닌 곡선형이라고 가정한다.
- 입력 변수의 비선형 조합을 사용하여 복잡한 패턴을 학습할 수 있다. → 데이터를 변형하더라도 계수를 선형결합식으로 표현 불가능한 모델 → 비선형 모델은 계수 간에 곱셈, 제곱, 지수 등 비선형 연산이 포함된 모델

특징

복잡한 관계 표현 가능: 비선형 모델은 곡선 형태의 데이터도 잘 학습할 수 있다.

높은 표현력: 다양한 패턴과 복잡한 관계를 모델링할 수 있다.

과적합 위험: 비선형 모델은 복잡한 관계를 학습할 수 있지만, 데이터에 너무 잘 맞추려고 할 때 과적합(overfitting)이 발생할 수 있다.

해석이 어려움: 모델이 복잡해질수록 해석이 어려워진다.

비선형 모델의 예시

- 다항 회귀(Polynomial Regression): 입력 변수의 제곱, 세제곱 등을 사용하여 비선형 관계를 학습한다.
- 신경망(Neural Network): 활성화 함수(ReLU, Sigmoid, Tanh 등)를 사용하여 비선형성을 추가한다.
- 서포트 벡터 머신(SVM) - 커널 트릭: 비선형 커널(RBF, Polynomial)을 사용하여 입력 변수를 고차원 공간으로 변환한다.

특징	선형 모델	비선형 모델
선형성	계수 간의 선형 결합	계수 간 비선형 연산 포함 가능
관계 형태	직선 또는 곡선 (비선형 변환 가능)	곡선 형태
복잡도	낮음	높음
해석 가능성	높음	낮음
예시	선형 회귀, 다항 회귀, 로지스틱 회귀	결정 트리, 신경망, 비선형 SVM

Linear Model / Non-linear Model

비선형 모델에서의 비선형 연산

- 비선형 모델(Non-linear Model)은 입력 변수와 출력 변수 간의 관계가 선형적이지 않은 모델
- 여기서 "비선형"이라는 의미는 모델의 계수(coefficient) 간에 비선형 연산이 포함될 수 있다는 것을 뜻한다. 즉, 곱셈, 제곱, 지수 함수, 삼각 함수 등의 비선형 연산이 적용될 수 있다.

1. 선형 모델 vs 비선형 모델에서의 계수

1) 선형 모델에서의 계수 연산

선형 모델에서는 계수 w_i 와 입력 변수 x_i 가 곱셈 연산으로 결합되며, 이들은 선형적으로 합산된다.

$$y = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

여기서 중요한 점은 w_i 와 x_i 간에는 곱셈만 있고, 계수 간에는 비선형 연산이 없다.

2) 비선형 모델에서의 계수 연산

비선형 모델에서는 계수 간에 비선형 연산이 포함될 수 있다.

이는 곱셈, 제곱, 지수 함수 등 다양한 연산을 포함한다.

$$y = w_1x_1^2 + w_2 \sin(x_2) + e^{w_3x_3} + b$$

- 입력변수 x_1 에 대해 제곱 연산이 적용
- 입력 변수 x_2 에 대해 삼각 함수가 적용
- 입력 변수 x_3 에 대해 지수 함수가 적용

이러한 연산들은 비선형적이기 때문에, 출력 값 y 는 입력 변수들과의 비선형 관계를 나타낸다.

Linear Combination

선형 결합 (Linear Combination)

정의

- 선형 결합은 주어진 벡터들의 가중합(weighted sum)을 의미한다.
- 여러 벡터에 스칼라 계수를 곱하고, 이들을 모두 합한 결과이다.
- **선형 결합의 핵심은 입력 변수들이 각각 가중치와 곱해져 더해진다는 것이다.** 이 과정에서 입력 변수와 가중치의 곱셈 및 합산은 선형적이다.

$$y = a_1x_1 + a_2x_2 + \dots + a_nx_n$$

y : 선형 결합의 결과 벡터

x_i : 주어진 벡터들

a_i : 스칼라 계수 (가중치)

특징

선형 결합은 벡터 공간 생성(span)과 관련이 있다.

여러 벡터의 선형 결합으로 표현할 수 있는 벡터들의 집합을 그 벡터들의 생성 공간이라 한다.

예를 들어, 2차원 평면에서 두 벡터가 독립적이라면, 이 벡터들의 선형 결합은 평면 전체를 생성할 수 있다.

•**독립성과 종속성**: 만약 어떤 벡터가 다른 벡터들의 선형결합으로 정확히 표현될 수 있다면, 그 벡터들은 선형종속(Linearly Dependent)이다.

•반대로, 어떤 벡터도 다른 벡터들의 선형결합으로 표현될 수 없다면, 그 벡터들은 선형독립(Linearly Independent)이다.

선형 결합의 특징

- 가중합(Weighted Sum) : 선형 결합은 입력 변수들의 가중합
→ 각 입력 변수에 가중치(계수)를 곱하고, 그 결과를 모두 더한 값이다.
- 선형성: 선형 결합에서는 곱셈, 나눗셈, 제곱 등의 비선형 연산이 없습니다.
 w_i 와 x_i 의 관계는 선형적입니다. 즉, x_i 의 변화에 따라 출력 값 y 도 비례적으로 변한다.
- 해석 가능성:
각 가중치 w_i 는 해당 입력 변수 x_i 의 중요도나 영향력을 나타낸다.
✓ $w_i > 0$ 이면, x_i 가 증가할 때 y 도 증가한다.
✓ $w_i < 0$ 이면, x_i 가 증가할 때 y 는 감소한다.

<https://github.com/freejyb/KDT>

데이터전처리/ Linear Combination

Linear Combination

선형 결합 (Linear Combination)

선형 결합의 중요성 해석 가능 모델:

선형 회귀, 로지스틱 회귀와 같은 모델들은 선형 결합을 사용하기 때문에 해석이 쉽고, 각 입력 변수의 영향력을 직관적으로 이해할 수 있다.

기초 개념:

- 많은 머신러닝 모델들은 선형 결합을 기초로 하여 더 복잡한 구조를 만든다.
- 예를 들어, 인공 신경망(Neural Network)에서도 선형 결합 후 비선형 활성화 함수를 적용한다.

확장 가능성:

- 선형 결합의 기본 개념을 사용해 입력 변수들을 변환하거나 확장하여 비선형 모델을 만들 수 있다.
- 예를 들어, 다항 회귀(Polynomial Regression)는 입력 변수를 제곱이나 세제곱으로 변환하여 선형 회귀를 적용한다.

결론

선형 결합은 여러 변수와 가중치가 곱해진 후 합산된 값입니다.

선형 모델에서의 선형 결합은 계수 간 선형 관계를 의미하며, 반드시 "직선"만 나타내는 것은 아니다.

선형 결합은 많은 머신러닝 모델의 기초 개념으로 사용되며, 데이터의 특성을 이해하고 모델을 해석하는 데 중요한 역할을 한다.

Linear Transformation

선형 변환(Linear Transformation)

두 벡터 공간 사이의 관계를 설명하는 함수로, 선형성을 만족하는 변환

- 선형변환은 벡터 공간의 구조를 보존하면서 한 벡터 공간의 원소를 다른 벡터 공간의 원소로 매핑한다.
- 벡터 공간에서의 함수로, 벡터를 다른 벡터로 매핑한다.
- 행렬을 사용하여 표현

선형변환의 정의

- 선형변환은 두 가지 주요 성질을 만족해야 한다.
- 벡터의 덧셈과 스칼라 곱셈에 대한 연산이 변환 후에도 유지된다는 것
→ 즉, 선형변환에 대해 두 벡터 u 와 v , 그리고 스칼라 c 에 대해 다음을 만족

1. 가산성(Additivity) (superposition)

$$1. T(u + v) = T(u) + T(v)$$

2. 동차성(Homogeneity)

$$2. T(c \cdot u) = c \cdot T(u)$$

이 두 조건이 성립하면, T 는 선형 변환이다

선형 변환의 수학적 표현

선형 변환은 주로 행렬을 통해 표현된다. 어떤 선형 변환 T 는 행렬 A 에 의해 정의될 수 있으며, 이는 벡터 x 에 행렬 곱셈을 수행하여 새로운 벡터를 생성하는 과정이다.

$$T(x) = Ax$$

- x : 입력 벡터
- A : 선형 변환을 정의하는 행렬
- 결과인 Ax 는 변환된 벡터

Linear Transformation

선형 변환 종류

1. 회전 (Rotation):

벡터를 원점에 대해 일정 각도로 회전시키는 변환이다.
2차원에서 회전 행렬은 다음과 같다.

$$A = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

이 행렬을 벡터 $\mathbf{x} = [x_1, x_2]^T$ 에 곱하면, 벡터는 각도 θ 만큼 회전된다.

2. 확대 및 축소 (Scaling):

• 벡터의 크기를 일정 비율로 늘리거나 줄이는 변환이다.

예를 들어, 행렬 $A = \begin{bmatrix} k & 0 \\ 0 & k \end{bmatrix}$ 를 곱하면 벡터의 크기가 k 배가 된다.

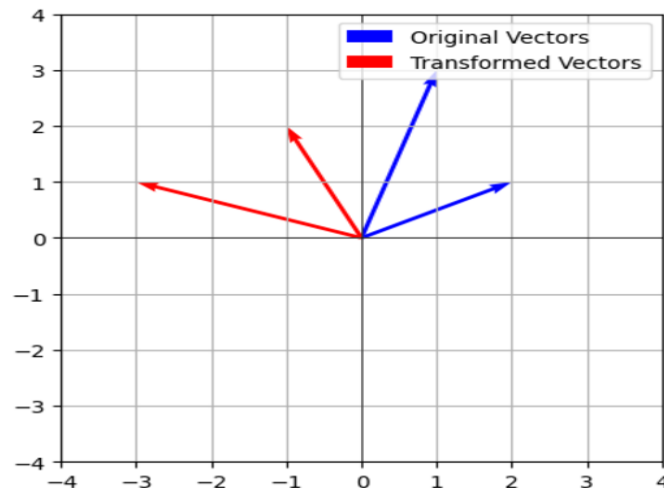
3. 반사 (Reflection):

• 벡터를 특정 축에 대해 대칭 이동시키는 변환이다.

예를 들어, 행렬 $A = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$ 은 x축에 대해 반사하는 변환을 나타낸다.

선형 변환의 성질

- 원점을 유지: 선형 변환은 원점을 다른 곳으로 이동시키지 않는다. 즉, 모든 선형 변환에서 원점은 고정된 점이다.
- 덧셈과 스칼라 곱의 보존: 앞서 정의한 것처럼, 선형 변환은 벡터 간의 덧셈과 스칼라 곱을 보존한다.
- 행렬 표현 가능: 모든 선형 변환은 행렬을 사용하여 표현할 수 있으며, 이를 통해 벡터 공간의 차원을 늘리거나 줄일 수 있다.



그래프 설명

- 파란색 벡터: 변환 전의 원본 벡터들
- 빨간색 벡터: 회전(Rotation) 선형 변환

원래 벡터들

```
vectors = np.array([[2, 1], [1, 3]])
```

변환 행렬 (90도 회전 행렬)

```
rotation_matrix = np.array([[0, -1], [1, 0]])
```

<https://github.com/freejyb/KDT>

데이터전처리/ Linear Combination

Linear Transformation

선형 변환 활용

선형 변환(Linear Transformation)은 다양한 분야에서 벡터 공간의 변형을 위해 사용되며, 많은 수학적, 과학적, 공학적 문제를 해결하는 데 중요한 역할을 한다. 선형 변환의 기본적인 용도는 벡터 공간을 변환하여 특정한 특성을 강조하거나, 좌표계를 변경하고, 기하학적 연산을 수행하는 것이다.

1. 기계 학습 및 데이터 분석

- 선형 변환은 머신러닝과 데이터 분석에서 특성 공간(feature space)을 변환하거나 차원 축소(Dimensionality Reduction)하는 데 사용된다.
- PCA(주성분 분석, Principal Component Analysis)는 고차원 데이터의 차원을 줄여 주요 정보를 추출하는 데 사용되며, 이는 선형 변환을 통해 각 주성분으로 데이터를 투영하는 방식이다.
- 선형 회귀(Linear Regression), 로지스틱 회귀(Logistic Regression) 등의 선형 모델들은 데이터의 입력 공간을 선형 변환하여 예측을 수행한다.
- 선형 변환을 통해 특성 간의 상관관계를 제거하거나 데이터를 새로운 좌표계로 변환함으로써 학습 성능을 향상시킬 수 있다.

2. 통계 및 확률

- 선형 변환은 확률 밀도 함수를 변환하거나 데이터의 공분산 행렬(Covariance Matrix)을 구하는 데 사용된다.
- PCA를 비롯한 다양한 차원 축소 방법들은 데이터의 공분산을 기반으로 선형 변환을 적용하여 고차원 데이터를 보다 간단하게 분석할 수 있는 형태로 변환한다.

3. 신경망과 딥러닝

- 신경망에서는 각 층의 뉴런들이 선형 변환을 통해 서로 연결된다. 예를 들어, 가중치 행렬(Weight Matrix)을 사용해 입력 벡터를 변환한 후, 활성화 함수로 비선형성을 추가한다.
- 신경망의 각 층에서 선형 변환을 적용한 후 비선형 활성화 함수를 사용함으로써 모델이 더 복잡한 패턴을 학습할 수 있다.

4. 컴퓨터 비전

- 컴퓨터 비전에서 이미지나 물체를 특정한 각도로 회전, 크기 조절, 전단(Shear) 등의 변환을 수행하는 데 선형 변환이 사용된다.
- 예를 들어, 카메라 렌즈의 왜곡을 보정하거나 이미지의 특정 부분을 확대하여 볼 때, 선형 변환을 통해 필요한 변형을 적용한다.
- 이미지 처리에서도 변형(Transformation), 왜곡(Warping) 등 다양한 효과를 주기 위해 선형 변환을 사용한다. 예를 들어, 이미지의 크기를 조정하거나 회전시키는 경우에 행렬 곱을 사용한 선형 변환을 적용한다.

5. 선형 대수와 벡터 공간의 변환

- 벡터 공간 변환: 벡터 공간에서 특정 벡터를 다른 벡터로 변환하는 역할을 하며, 이는 좌표계를 변환하거나 데이터의 표현 방식을 바꾸는 데 사용된다.
- 좌표계 변경(Coordinate Transformation): 선형 변환은 데이터를 특정 기준 좌표계에서 다른 좌표계로 변환할 수 있다. 이는 좌표계를 이동하거나 회전할 때 많이 사용된다. 예를 들어, 원래 좌표계에서 새로운 기준으로 데이터를 표현하고 싶을 때 행렬 곱을 사용해 좌표 변환을 수행할 수 있다.

Linear Transformation

선형 변환은 만족하는 사례

- **항등 변환(Identity Transformation):** 모든 벡터를 자기 자신으로 매핑하는 변환
- **회전 변환(Rotation Transformation):** 평면상의 모든 점을 일정한 각도로 회전시키는 변환
- **스케일링 변환(Scaling Transformation):** 모든 벡터의 길이를 일정한 비율로 늘리거나 줄이는 변환
- **투영 변환(Projection Transformation):** 모든 점을 특정 평면이나 직선 위로 투영시키는 변환
- **전단 변환(Shear Transformation):** 한 축 방향으로 객체를 기울게 하는 변환

비교: 선형 결합 vs 선형 변환

특징	선형 결합 (Linear Combination)	선형 변환 (Linear Transformation)
정의	벡터들의 가중합	벡터를 다른 벡터로 변환하는 함수
수식	$\mathbf{y} = a_1\mathbf{x}_1 + a_2\mathbf{x}_2$	$T(\mathbf{x}) = \mathbf{A}\mathbf{x}$
목적	새로운 벡터 생성 (가중합)	벡터의 변환 (확대, 축소, 회전 등)
연산 대상	벡터들의 합	벡터와 행렬의 곱
예시	$2\mathbf{x}_1 + 3\mathbf{x}_2$	$T(\mathbf{x}) = \mathbf{A}\mathbf{x}$

관계

- 선형 변환은 선형 결합을 사용하여 벡터를 변환한다.
- 즉, 선형 변환에서 출력 벡터는 입력 벡터의 선형 결합으로 나타난다.
- 예를 들어, 행렬 A 는 열 벡터들의 집합으로 표현될 수 있으며, 이 열 벡터들의 선형 결합을 통해 변환된 벡터가 만들어진다.

Linear Transformation vs Matrix

행렬과 선형변환의 관계

- 행렬은 기본적으로 선형변환을 표현하는 도구이다.
- 특히, n -차원 벡터 공간에서 m -차원 벡터 공간으로의 선형변환은 항상 $m \times n$ 행렬로 표현될 수 있다.

행렬을 통한 선형변환의 본질

행렬을 이용한 선형변환은 벡터의 크기와 방향을 변환하는 것을 의미한다.

예를 들어, 행렬은 다음을 수행할 수 있다:

1. 회전: 벡터를 특정 각도만큼 회전시킴.
2. 확대/축소: 벡터의 크기를 스케일링.
3. 전단(Shear): 벡터를 비스듬히 변형.
4. 반사(Reflection): 축을 기준으로 벡터를 반사.

왜 행렬(곱셈)이 선형변환인가?

행렬 곱셈 Ax 은 항상 다음 두 조건을 만족한다:

- 덧셈에 대한 선형성: $A(u+v) = Au + Av$
- 스칼라 곱에 대한 선형성: $A(cu) = c(Au)$

따라서 행렬 곱셈은 선형변환의 정의를 만족한다.

행렬은 선형변환의 구체적인 표현이며, 행렬을 사용하면 선형변환을 간결하게 계산하고 나타낼 수 있다.

따라서 "행렬(의 곱)은 선형변환이다"라는 말은, 선형변환의 본질이 행렬로 표현된다는 뜻이다.

* Probability distribution

* 통계분석 자료 참조

Stochastic Process

확률 과정(Stochastic Process)란 ? 확정적

→시간의 진행에 대해 확률적인 변화를 가지는 구조를 의미

Markov Process, Wiener's process, Brownian motion process, Ito process, poisson process

Random walk

- 임의 방향으로 향하는 연속적인 걸음을 나타내는 수학적 개념
- 1905년 칼 피어슨이 소개
- 생태학, 수학, 컴퓨터 과학, 물리학, 화학 등의 분야에서 광범위하게 사용
- 대표적인 예 : 브라운 운동

- 절편 = 0 , 시계열의 평균은 일정 , 분산은 시간에 비례하여 증가 → 앞뒤로 움직일 확률이 동일하다고 해도 시간이 흐름에 따라 평균에서 점차 벗어나는 경향
- 절편 $\neq 0$, 시계열의 평균과 분산이 모두 시간에 비례하여 증가

Brownian motion

- 1827년 스코틀랜드 식물학자 Robert Brown이 발견
- 액체나 기체 속에서 미소입자들이 불규칙하게 운동하는 현상

Stochastic Process

랜덤 과정 / 확률 과정 (Random Process, Stochastic Process, Probabilistic Process)

Random / Stochastic

- 시간적으로 미리(사전에) 결과에 대해 정확히 예측, 정의할 수 없다는 의미
- 단, 어떤 확률적 분포를 가질 수 있다는 통계적 규칙성은 있음 → 랜덤성
- > 여기서, 'Random(무작위)' 및 'Stochastic(추계적)'를 같은 의미로 씀

Process (과정)

- 시간을 고려한 상태를 말할 때는 주로 '과정'
- 시간을 고려하지 않는 상태는 주로 '사건'이라고 함

확률 과정(Stochastic Process)

연속적 확률과정 : 시간 T 가 어떠한 연속적인 범위를 두고 있는 경우

- 정상성(stationary) : 연속형 확률과정 $X(t)$ 가 같은 시간의 주기를 가지면 같은 확률분포를 가진다는 의미
- 독립성 : 연속형 확률과정 $X(t)$ 가 서로 겹치지 않는 구간들에 대해서 전부 독립적인 확률 변수라는 의미

이산적 확률과정

- Markov Process
- Wiener process (Standard Brownian Motion)
- Ito process
- poisson process

Stochastic Process

Markov stochastic process

- 1906년 러시아의 수학자 안드레이 마르코프가 도입
- T+1시점은 과거의 값에는 전혀 영향을 받지 않고 오직 오늘의 값(T시점)에만 영향을 받는 확률과정
- 현재에 대한 조건부로 과거와 미래가 서로 독립인 확률 과정

$$E(S_{t+1} | S_t, S_{t-1}, \dots, S_0) = E(S_{t+1} | S_t)$$

브라운 운동(Brownian motion)

- 1827년 스코틀랜드 식물학자 로버트 브라운(Robert Brown)이 발견
- 액체나 기체 속에서 미소입자들이 불규칙(무작위)하게 운동하는 현상

https://ko.wikipedia.org/wiki/%EB%B8%8C%EB%9D%BC%EC%9A%B4_%EC%9A%B4%EB%8F%99

Wieners process

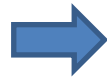
- Norbert Wiener(1894년 11월 26일 ~ 1964년 3월 18일)에 의해 브라운운동을 수학적으로 정립
 - 이러한 수학적 모델을 **Wieners process** 라고 부른다.
- **Wieners process** 를 통해 무작위성에 대한 수학적 추상화(abstraction) 를 달성

Stochastic Process

Wiener's process, → 표준 브라운 운동(Standard Brownian Motion, SBM)

- 마르코프 과정 중 변화량의 평균 0 → 향후 예상되어지는 변화가 평균적으로 0 + 연간 분산은 1을 따르는 확률과정
- Wiener's process를 따르는 확률변수 z 의 특징 $\Delta Z = \varepsilon \sqrt{\Delta t}$
- z 의 변화량은 시간의 변화에 영향을 받고, z 는 마르코프 과정을 따르기 때문에, 각각의 시간에 따른 z 값은 다른 시간 간격의 영향을 받지 않는다.
- z 의 변화량 : 평균은 0, 분산은 t 의 변화량을 값으로 갖는 정규분포를 따른다. T 시점의 분산은 T 만큼의 값을 갖는다.
→ 시간의 변화량이 커질수록, z 의 변동성도 함께 커지게 된다.
- (위너 과정 특징)
→ 위너과정의 평균과 분산이 시간이 지남에 따라 커지는 선형함수로 표현된다는 것

$$\Delta Z = \varepsilon \sqrt{\Delta t}$$



$$dX = \varepsilon \sqrt{dt}$$

- ε : 정규분포를 통해 뽑아낸 난수
- $\sqrt{dt}$: 시간 프레임에 따른 스케일링 팩터
- 위험자산의 통계적 성질을 모델에 담고 있지 못하는 한계
 - 1) 시간의 흐름에 따른 장기적 추세 (drift) - 평균
 - 2) 단기적으로 추세성보다 큰 변동성 - 표준편차
- 수익률의 평균과 표준편차는 주가(확률변수)의 통계적 구조를 형성

Stochastic Process

Wieners process, → 표준 브라운 운동(Standard Brownian Motion, SBM)

(특징)

- 1) 시작점 : 표준 브라운 운동은 보통 시간 $t=0$ 에서 0의 값에서 시작
- 2) 독립적 증분 : 브라운 운동의 증분은 서로 독립적 : 과거의 경로가 미래의 경로에 영향을 주지 않는다.
- 3) 정규분포 증분: 브라운 운동의 증분은 정규 분포를 따릅니다. 구체적으로, 어떤 시간 간격 Δt 에 대해, 그 간격 동안의 운동은 평균이 0이고 분산이 Δt 인 정규 분포를 따른다..
→ $B(t+\Delta t)-B(t)$ 는 평균이 0이고 분산이 Δt 인 정규 분포를 따른다. 즉, 이 증분은 $N(0, \Delta t)$ 으로 모델링된다.
- 3) 연속성: 브라운 운동의 경로는 연속적 : 급격한 점프나 불연속적인 변화가 없다.
→ 금융 연구에서 주가의 움직임이 위너과정을 따른다고 전제

Stochastic Process

Arithmetic Brownian motion ABM

평균변화율이 0이 아닌 확률 과정 : Generalized Wiener process , Arithmetic Brownian motion

$$dS(t) = \mu dt + \sigma dW$$

- **S(t) : Stochastic Process**
- μ : S의 변화율의 평균, drift parameter - 장기적 추세
- σ : 표준편차를 나타내는 상수항 - 단기적 변동성
- W : 위너 과정 - 예측 불가능성

추가 Drift 현상

주가가 상승하든지 하락하든지 관계없이 매 기마다 무위험자 산수익률, 인플레이션 영향으로 Drift 만큼은 오르게 된다.

정의: 산술 브라운 운동은 표준 브라운 운동에 선형 추세(drift)를 추가한 것

이는 시간에 따라 일정한 속도로 증가하거나 감소하는 경향이 있는 확률 과정을 모델링한다.

특징: drift 항 때문에, 산술 브라운 운동 (ABM) 은 시간이 지남에 따라 평균값이 변화한다.

사용 : 경제 지표나 이자율과 같이 시간에 따라 선형적으로 증가하거나 감소하는 현상을 모델링하는 데 적합

Standard Brownian Motion(SBM) 과 Arithmetic Brownian motion(ABM)

구분	SBM	ABM
추세(DRIFT)의 유무(분포)	drift 항 없음 (정규분포)	drift 항 포함 (정규분포)
모델링 대상	순수한 무작위 움직임을 모델링	시간에 따라 일정한 추세를 가지는 현상을 모델링
평균값의 변화	평균값은 시간에 따라 변하지 않는다.	평균값은 시간에 따라 선형적으로 변한다.
사용	보다 일반적인 무작위 움직임을 이해하는 데 사용	경제학과 금융에서 이자율, 환율, 경제 지표 등의 시간에 따른 변화를 모델링하는 데 유용

Stochastic Process

Geometric Brownian motion (GBM)

- **정의:** 표준 브라운 운동을 기반으로 하지만, 로그 정규분포를 따르는 비율 변화를 모델링한다.
→ 이는 주로 금융에서 자산 가격의 시간에 따른 변화를 모델링하는 데 사용
- **특징:** 로그 정규 분포를 따르며, 이는 자산 가격이 음수가 되지 않고, 비율 변화가 중요한 경우에 적합

$$\frac{dS(t)}{S(t)} = \mu dt + \sigma dW$$

- → **Stochastic Process의 변화량을 변화율로 바꿔**, Stochastic Process의 값이 음수(-)가 나오게 하지 않는 것
- → 별도 참조 (뒷면)

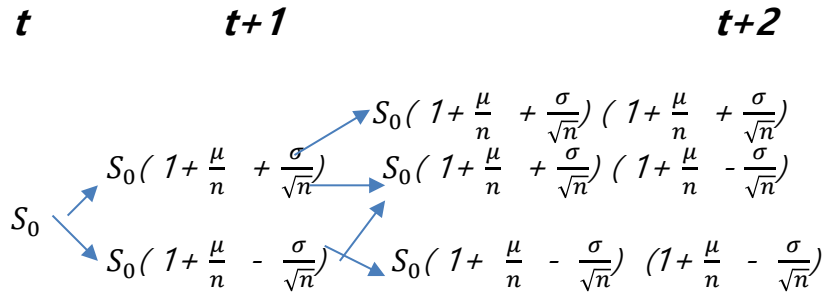
구분	SBM	GBM
분포 전제	정규 분포	로그 정규 분포
값의 범위	양수 또는 음수	항상 양수 값 → 이는 자산 가격이 음수가 될 수 없다는 현실을 반영
수학적 복잡성		상대적으로 복잡성 → GBM이 자산 가격의 비율 변화를 모델링하기 때문
적용 분야	일반적인 확률 과정을 모델링하는 데 사용	주로 금융에서 자산 가격의 변동을 모델링하는 데 사용

사례) 블랙-숄즈 모형 , Geometric Brownian motion

- 주식의 가격을 브라운 운동으로 모델링
 - 이러한 모델은 가격이 불규칙적(무작위적)으로 움직인다는 가정을 기반
1. 주가의 변화는 연속적이다.
 2. 주가는 로그정규분포를 따른다.
 3. 주가수익률은 정규분포를 따른다.
 4. 주식의 기대수익률과 수익률의 불확실성은 보유기간에 비례한다.

* Geometric Brownian Motion (GBM)

현재 주가(S_0) 변화 (Drift 효과 고려 $(\frac{\mu}{n})$ 매 기에 적용되는 주가 수익률 표준편차 $\frac{\sigma}{\sqrt{n}}$ 는 동일한 것으로 가정)



테일러 급수 ($a=0$)를 2차 항 까지만 적용

$$\ln\left(1 + \frac{\mu}{n} + \frac{\sigma}{\sqrt{n}}\right) \approx \left(\frac{\mu}{n} + \frac{\sigma}{\sqrt{n}}\right) - \frac{1}{2} \left(\frac{\mu}{n} + \frac{\sigma}{\sqrt{n}}\right)^2$$

$$= \frac{\mu}{n} + \frac{\sigma}{\sqrt{n}} - \frac{\mu^2}{2n^2} - \frac{\mu\sigma}{n\sqrt{n}} - \frac{\sigma^2}{2n}$$

$$\ln\left(1 + \frac{\mu}{n} - \frac{\sigma}{\sqrt{n}}\right) \approx \left(\frac{\mu}{n} - \frac{\sigma}{\sqrt{n}}\right) - \frac{1}{2} \left(\frac{\mu}{n} - \frac{\sigma}{\sqrt{n}}\right)^2$$

$$= \frac{\mu}{n} - \frac{\sigma}{\sqrt{n}} - \frac{\mu^2}{2n^2} + \frac{\mu\sigma}{n\sqrt{n}} - \frac{\sigma^2}{2n}$$

주가의 모형 일반화

$$S_t = S_0 \left(1 + \frac{\mu}{n} + \frac{\sigma}{\sqrt{n}}\right)^u \left(1 + \frac{\mu}{n} - \frac{\sigma}{\sqrt{n}}\right)^d$$

- 총 시행 횟수 (nt) = u (상승 횟수) + d (하락 횟수)
 - 랜덤워크 시행 결과 (M_{nt}) = $u - d$
 $-u = \frac{1}{2}(nt + M_{nt})$, $d = \frac{1}{2}(nt - M_{nt})$
 $\rightarrow S_t = S_0 \left(1 + \frac{\mu}{n} + \frac{\sigma}{\sqrt{n}}\right)^{\frac{1}{2}(nt + M_{nt})} \left(1 + \frac{\mu}{n} - \frac{\sigma}{\sqrt{n}}\right)^{\frac{1}{2}(nt - M_{nt})}$

$$\ln S_t = \ln S_0 + ut - \frac{\sigma^2}{2}t + \frac{M_{nt}}{\sqrt{n}}\sigma - \frac{\mu^2 t}{2n} - \frac{\mu t}{n} \frac{M_{nt}}{\sqrt{n}}$$

여기서 $n \rightarrow \infty$ (무한 반복 수행)

$$: \frac{M_{nt}}{\sqrt{n}} = W_t, \quad \frac{\mu^2 t}{2n} = 0, \quad \frac{\mu t}{n} = 0$$

$$\ln S_t = \ln S_0 + ut - \frac{\sigma^2}{2}t + \sigma W_t$$

$$\text{결국 } S_t = S_0 e^{\sigma W_t + \left(u - \frac{\sigma^2}{2}\right)t}$$

자연로그

$$\ln S_t = \ln S_0 + \frac{1}{2}(nt + M_{nt}) \ln \left(1 + \frac{\mu}{n} + \frac{\sigma}{\sqrt{n}}\right) + \frac{1}{2}(nt - M_{nt}) \ln \left(1 + \frac{\mu}{n} - \frac{\sigma}{\sqrt{n}}\right)$$

$$\text{GBM : } S_t = S_0 e^{\sigma W_t + \left(u - \frac{\sigma^2}{2}\right)t}$$

Stochastic Process

Ito Process

- Stochastic Process의 일반화
- Brownian Motion을 기초로 하는 어떤 임의의 Stochastic Process의 Stochastic Differential Equation은
- 일반화 한 것

$$dx(t) = \alpha(x(t), t)dt + \sigma(x(t), t)dW$$

- $\alpha(x(t), t)$: *drift Term* , 결정론적 파트
- $\sigma(x(t), t)$: Diffusion Term , 확률론적 파트

ARIMA (AutoRegressive Integrated Moving Average) 모형

확률모형을 기반으로 한 시계열 분석 기법

- 정상확률과정(Stationary Process)을 가정
- 만약 시계열 데이터가 정상성(stationarity) 가정을 만족하지 않을 경우 데이터 전처리를 통해서 정상시계열(stationary time series)로 변환해 준 후에 ARIMA 모형 적용 가능

- 백색잡음과정 (White Noise Process)
- 확률보행과정 (Random Walk Process)
- 정상확률과정 (Stationary Process)

백색잡음과정 (White Noise Process)

- 신호처리 분야 : 다른 주파수에서 동일한 강도를 가지고, 항상 일정한 스펙트럼 밀도를 발생시키는 무작위한 신호 → 물리학, 음향공학, 통신, 통계 예측 등 다양한 과학과 기술 분야에서 사용.
- 백색잡음과정(white noise process) : 평균이 0, 분산은 유한한 상수(σ^2)인 확률분포(random variables)로 부터 서로 상관되지 않게(uncorrelated) 무작위로 샘플을 추출한 이산 신호(discrete signal)

특히, 평균이 0 인 동일한 정규분포로부터 서로 독립적으로 샘플을 추출한 백색잡음을 가법 백색 가우시언 잡음(additive white Gaussian noise) 라고 함.

Stochastic Process

확률보행과정 (Random Walk Process)

시계열 과정 (python 참조)

정상확률과정 (Stationary Process)

[정상시계열의 조건 (conditions for stationary time series)]

- (a) 평균이 일정
- (b) 분산이 존재하며 상수
- (c) 두 시점 사이의 자기공분산은 시차(time lag)에만 의존

→ ARIMA 시계열 통계모형은 시계열 데이터의 정상성(stationarity) 조건을 만족하는 경우에 사용 가능

→ 만약 추세(trend)가 있거나 시간이 갈 수록 분산이 커지거나 작아지는 시계열 데이터라면 정상확률과정 시계열이 아니다.

마팅게일(Martingale)

- 마팅게일은 **확률론적 과정** 중 하나로, 시간에 따른 기대값이 변하지 않는 무작위 과정
- 현재 시점까지 주어진 모든 정보를 기반으로 미래 값을 예측했을 때, 기대값이 현재 값과 동일하면 그 과정은 마팅게일이다.
- 미래 값의 조건부 기대값이 현재 값과 같다는 것을 의미**

$$\mathbb{E}[X_{t+1}|\mathcal{F}_t] = X_t, \quad \forall t \geq 0$$

X_t : t 시점의 확률 변수

$\mathcal{F}_t$: 시점 t 까지의 정보(시그마 대수)

E : 기대값

특징

- 무기억성**

마팅게일 과정에서는 과거의 값이 미래의 변화에 영향을 미치지 않는다.

현재 값이 미래 값을 예측하는 데 있어서 유일한 정보이다.

- 공정한 게임 모델**

마팅게일은 카지노의 공정한 게임(**예: 동전 던지기**)처럼, 시간이 지나도 평균적인 이익이나 손해가 없다는 것을 모델링한다.

마팅게일의 응용

- 금융**

금융공학에서 마팅게일은 리스크 중립 확률 측도에서 옵션 가격 결정 모델에 활용
블랙-숄즈 모델(Black-Scholes Model)에서 주가가 마팅게일 특성을 가진다고 가정한다.

- 확률론 및 통계**

통계적 검정, 시뮬레이션, 그리고 최적화 문제에서 사용된다.

마팅게일(Martingale)

➤ 서브마팅게일 (Submartingale)

$$\mathbb{E}[X_{t+1}|\mathcal{F}_t] \geq X_t$$

→ 미래 기대값이 현재 값보다 크거나 같을 때.

➤ 슈퍼마팅게일 (Supermartingale)

$$\mathbb{E}[X_{t+1}|\mathcal{F}_t] \leq X_t$$

→ 미래 기대값이 현재 값보다 작거나 같을 때.

➤ 마팅게일 변환 (Doob's Martingale Transformation)

마팅게일 X_t 와 가중치 M_t 에 대해 변환된 과정:

$$Y_t = \sum_{i=1}^t M_i(X_i - X_{i-1})$$

이 과정 Y_t 도 마팅게일이 된다

*위험중립확률(Risk-Neutral Probability)

옵션 가격 결정에서 **위험중립확률(Risk-Neutral Probability)**은 옵션을 평가하는 데 중요한 개념으로, 위험 회피 성향이나 시장 기대 수익률과 무관하게 옵션의 공정 가치를 계산하기 위해 도입된 가상의 확률 체계이다.

위험중립확률의 정의

위험중립확률은 다음과 같은 조건을 만족하는 확률 측도를 의미한다.

- 1) 투자자가 위험을 회피하지 않는다고 가정한다. 즉, **모든 투자자가 무위험 금리로 자산을 할인한다.**
- 2) 기대 수익률이 무위험 금리(r)와 같도록 확률 분포를 조정한다.

$$P = \mathbb{E}^Q \left[\frac{\text{미래 현금 흐름}}{(1+r)^T} \right]$$

P : 자산의 현재 가격
 $\mathbb{E}^Q$: 위험중립확률 하에서의 기대값
 r : 무위험 이자율
 T : 만기 시간

위험중립확률의 도출

$$q = \frac{(1+r) - (1-d)}{(1+u) - (1-d)}$$

u : 상승 비율
 d : 하락 비율
 r : 무위험 금리

위험중립확률의 역할

- 옵션 가격의 계산 블랙-숄즈 옵션 가격 결정 모델(Black-Scholes Model)에서는 자산의 미래 가격이 위험중립확률 아래에서 로그 정규 분포를 따른다고 가정한다. 이를 통해 옵션의 이론적 가치를 계산한다.
- 시장 효율성 가정 위험중립확률은 시장이 효율적이며 무차익(no-arbitrage) 조건이 성립한다는 가정하에 작동한다.
- 무차익 조건 위험중립확률을 사용하면 옵션 가격이 무차익 기회가 없는 수준으로 결정된다. 즉, 동일한 조건의 다른 금융 상품으로부터 얻을 수 있는 이익과 동일한 가치를 가지도록 가격이 결정된다.

위험중립확률의 장점

- 모델 단순화: 위험중립확률을 사용하면 **투자자의 위험 선호도를 고려하지 않아도 되므로 옵션 가격 계산이 단순화된다.**
- 무차익 가격: 위험중립확률은 무차익 조건을 보장하며, 모든 금융 상품의 가격이 일관되게 유지된다.

Optimization

최적화 문제

- arguments of min : argmin
- arg min $f(x)$
- 함수 $f(x)$ 를 최솟값으로 만들기 위한 x 값을 구한다
- arguments of max : argmax
- arg max $f(x)$
- $f(x)$ 를 최댓값으로 만들기 위한 x 값을 구한다.
- $f(x)$: 목적함수(objective function), 비용함수(cost function), 손실함수(loss function)

1) grid search 방법

가능한 x 의 값을 여러 개 넣어 보고 그중 가장 작은 값을 선택

2) 수치적 최적화(numerical optimization)

- 단일 변수 함수인 경우: 미분값이 0
- 다변수 함수인 경우: 모든 변수에 대한 편미분값이 0

$$\frac{df(x)}{dx} = 0$$

$$\frac{\partial f(x_1, x_2, \dots, x_N)}{\partial x_1} = 0$$

$$\frac{\partial f(x_1, x_2, \dots, x_N)}{\partial x_2} = 0$$

$$\vdots$$

$$\frac{\partial f(x_1, x_2, \dots, x_N)}{\partial x_N} = 0$$

Optimization

딥러닝 손실함수

- 손실 함수(loss function)는 모델이 얼마나 잘못 예측했는지를 측정하는 함수이다.
- 주로 모델을 학습시키는 과정에서 손실을 최소화하기 위해 사용된다.

1. MSE (Mean Squared Error, 평균 제곱 오차)

- 주로 회귀(regression) 문제에서 사용
- 두 벡터(예측값과 실제값)의 차이를 제곱해 평균화한 값으로 계산

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

2. 크로스 엔트로피 (Cross-Entropy Loss)

- 주로 분류(classification) 문제에서 사용
- 실제값과 예측확률 간의 차이를 측정하며, 확률 분포 사이의 차이를 계산

$$\text{Cross-Entropy Loss} = -\frac{1}{N} \sum_{i=1}^N (y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i))$$

최적화 과정

기울기(gradient)* (뒷면참조) 를 계산한 후, 이를 이용해 경사 하강법(Gradient Descent)으로 파라미터를 업데이트 한다.

일반적인 업데이트 규칙:

$$w := w - \eta \cdot \nabla L$$

w: 모델 파라미터
 η : 학습률 (learning rate)
 ∇L : 손실 함수의 기울기

<https://github.com/freejyb/Deep-learning>

Optimization

딥러닝 손실함수

MSE (Mean Squared Error)의 기울기 전개

$$L = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

미분 과정:

목표: y^i 에 대해 $\partial L / \partial y^i$ 를 계산한다.

각 데이터 포인트 i 에 대해 손실의 한 항목을 미분:

$$\frac{\partial L}{\partial \hat{y}_i} = \frac{-2}{N} \sum_{i=1}^N (y_i - \hat{y}_i) \quad \frac{\partial L}{\partial \hat{y}_i} = -\frac{2}{N} (y_i - \hat{y}_i)$$

MSE 기울기는 예측값과 실제값의 **차이**를 기반으로 단순히 선형적으로 계산된다.

크로스 엔트로피 (Cross-Entropy Loss)의 기울기 전개

$$L = -\frac{1}{N} \sum_{i=1}^N (y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i))$$

첫 번째 항목에 대해 미분

두 번째 항목에 대한 미분

$$\frac{\partial}{\partial \hat{y}_i} y_i \log(\hat{y}_i) = y_i \cdot \frac{1}{\hat{y}_i}$$

$$\frac{\partial}{\partial \hat{y}_i} (1 - y_i) \log(1 - \hat{y}_i) = (1 - y_i) \cdot \frac{-1}{1 - \hat{y}_i}$$

L 전체에 대해 미분:

크로스 엔트로피는 확률 값을 기반으로 작동하기 때문에 y^i 가 0 또는 1에 가까워질 때 **기울기 폭발**(gradient explosion)을 방지하도록 클리핑(clipping)을 적용하는 것이 일반적이다.

$$\frac{\partial L}{\partial \hat{y}_i} = -\frac{1}{N} \left(\frac{y_i}{\hat{y}_i} - \frac{1 - y_i}{1 - \hat{y}_i} \right)$$

클리핑 (Clipping)

클리핑은 손실 함수나 기울기(gradient) 계산 과정에서 값이 너무 커지거나 작아져서 발생하는 문제를 방지하기 위해 값의 범위를 제한하는 방법

제한조건 있는 최적화(constrained optimization)

라그랑주 승수법 (Lagrange Multiplier Method) 을 사용하여 최적화

라그랑주 승수법 (Lagrange Multiplier Method)

- 프랑스의 수학자 조셉 루이 라그랑주 (Joseph-Louis Lagrange)가 **제약 조건이 있는** 최적화 문제를 풀기 위해 고안한 방법
- 어떠한 문제의 최적점을 찾는 것이 아니라, 최적점이 되기 위한 조건을 찾는 방법
→ 최적해의 필요조건을 찾는 방법

제한조건 등식에 $\lambda (\neq 0)$ 라는 새로운 변수를 곱해서 더한 함수

$$\begin{aligned} h(x, \lambda) &= h(x_1, x_2, \dots, x_N, \lambda_1, \dots, \lambda_M) \\ &= f(x) + \sum_{j=1}^M \lambda_j g_j(x) \end{aligned}$$

<https://github.com/freejyb/KDT/>
재무/최적화문제

Linear Programming

선형계획법 (LP : Linear Programming) 문제 ?

방정식, 부등식 제한 조건을 가지는 선형 모형(linear model)의 값을 최소화하는 문제

- 최소화하려는 목적함수 설정 (아래 c 관련)
- 제한조건 표현 : 등식 제한조건, 부등식 제한조건

1. scipy.optimize 패키지의 linprog() 명령을 사용하면 선형계획법 문제

linprog(c, A, b)

c: 목적함수의 계수 벡터

A: 등식 제한조건의 계수 행렬

b: 등식 제한조건의 상수 벡터

2. CVXPY 또는 PuLP

파이썬 패키지를 사용하면 선형계획법 문제의 계수 행렬 A, b, c 를 직접 숫자로 정의하지 않고 심볼로 정의하여 더 직관적인 파이썬 코드 가능

PuLP : 파이썬에서 선형 프로그래밍 문제를 풀기 위한 라이브러리로, 특히 선형 최적화 문제를 해결하는 데 사용

<https://github.com/freejyb/KDT/>
재무/최적화문제

Quadratic Programming

이차계획법(Quadratic Programming) 문제란?

방정식, 부등식 제한 조건을 가지는 일반화된 이차형식(Quadratic Form)의 값을 최소화하는 문제

- 이차 형식은 함수의 꼴이 이차 다항식인 것

$$f(\mathbf{x}) = \mathbf{x}^T \mathbf{Q} \mathbf{x}$$

$$= q_{11}x_1^2 + q_{22}x_2^2 + \cdots + q_{nn}x_n^2 + q_{12}x_1x_2 + \cdots + q_{n,n-1}x_nx_{n-1}$$

- Q는 nxn 크기의 대칭 행렬(symmetric matrix)이고, 이차 형식의 행렬(the matrix of the quadratic form)이라고 함.
- 대칭 행렬 : P와 D로 분해하면 P의 column이 A의 eigen vector로 이루어져 있고, 각 vector는 서로 직교한다는 것

`from cvxopt import matrix, solvers`

* Python : Library

CVXPY

- 파이썬을 위한 강력한 볼록 최적화 라이브러리
- 다양한 최적화 문제를 해결하기 위해 설계되었으며, 금융, 엔지니어링, 기계 학습 등 여러 분야에서 활용

1. 포트폴리오 최적화

투자자가 주어진 자산 집합에서 기대수익률을 최대화하면서 위험(변동성)을 최소화하고자 할 때, 포트폴리오 최적화가 필요

2. 기계 학습 및 데이터 과학

기계 학습에서는 모델의 복잡성을 제어하고 과적합을 방지하기 위해 정규화 기법을 사용

CVXPY는 로지스틱 회귀, 서포트 벡터 머신(SVM), 릿지 회귀, 라쏘 회귀 등 다양한 기계 학습 모델의 최적화 문제를 해결하는 데 사용

3. 자원 할당

엔지니어링, 제조, 프로젝트 관리 등에서 제한된 자원을 여러 프로젝트나 작업에 효율적으로 할당하는 것이 중요 → CVXPY를 사용하여 최적의 자원 할당 계획을 수립 가능

CVXPY

최소화

$$f(x) = 3x_1 + 4x_2$$

제약조건

$$x_1 + 2x_2 \geq 1, \quad x_1, x_2 \geq 0$$

최소화

$$f(x) = x_1^2 + x_2^2$$

제약조건

$$x_1 + x_2 \geq 1, \quad x_1, x_2 \geq 0$$

<https://github.com/freejyb/KDT/>

재무/최적화문제

* Python : Library

CVXPY

Portfolio 최적화

주어진 자산들 사이의 **최적의 가중치**를 찾아, 특정 목표(예: 최소 위험, 최대 수익)를 달성하는 문제

4개의 자산 (A,B,C,D)으로 구성된 포트폴리오의 최소 분산 구성을 찾는 문제

expected_returns : 각 자산의 예상 수익률

cov_matrix : 자산 간의 공분산 행렬

목적 함수: **포트폴리오의 분산을 최소화하는 것** - `cp.quad_form(weights, cov_matrix)`를 사용하여 계산

제약 조건 : 포트폴리오의 가중치 합이 1, 모든 가중치가 0 이상이어야 한다.

주식	기대수익률	위험(분산)
A	12%	0.04
B	10%	0.025
C	7%	0.01
D	3%	0.005

최소 분산 포트폴리오의 가중치

: [0.02687776 0.03350515 0.27724595 0.66237113]

<https://github.com/freejyb/KDT/>

재무/최적화문제

* Python : Library

CVXPY

Portfolio 최적화

3개의 자산 (A,B,C)으로 구성된 포트폴리오의 위험을 일정미만으로 제약을 두는 경우

제약조건: **기대 수익률을 10% 이상으로 하고 위험을 20% 미만**

#. 목적 함수는 최소화할 특정 목표가 아니라, 제약 조건을 만족하는 가중치를 찾는 것→ 따라서, 목적 함수를 설정할 필요가 없으며, 대신 기대 수익률과 위험에 대한 제약 조건을 명시

주식	기대수익률	위험(분산)
A	12%	0.005
B	20%	0.04
C	15%	0.023

최적 포트폴리오의 가중치: [0.44267178 0.24808369 0.30924453]

* Python : Library

CVXPY

Portfolio 최적화

3개의 자산 (A,B,C) + 무위험자산(기대수익률 3%)

제약조건: 기대 수익률을 10% 이상으로 하고 위험을 20% 미만

#. 목적 함수는 최소화할 특정 목표가 아니라, 제약 조건을 만족하는 가중치를 찾는 것→ 따라서, 목적 함수를 설정할 필요가 없으며, 대신 기대 수익률과 위험에 대한 제약 조건을 명시

주식	기대수익률	위험(분산)
A	12%	0.005
B	20%	0.04
C	15%	0.023

최적 포트폴리오의 가중치: [0.28982991 0.19198905 0.22022891 0.29795214]

무위험자산 비중 : 0.29795214

* Python 실습 & Mathematics 도구

파이썬 실습 : <https://github.com/freejyb/deeplearning>
딥러닝(수학1)

계산기 Microsoft Math Solver
(이전 **Microsoft Mathematics** 및 **Microsoft Math**)



<https://mathsolver.microsoft.com/en/algebra-calculator>



<https://www.desmos.com/calculator?lang=ko>

geogebra

<https://www.geogebra.org/?lang=ko>

수학을 공부하거나 가르칠 때 사용하는 지오지브라

클래스 활동, 그래프, 기하, 공동 화이트보드 등을 위한 무료 디지털 학습 도구

계산기 시작하기

커뮤니티 자료



Learn GeoGebra Classic

<https://www.geogebra.org/m/XUv5mXTm>

* Greek letter symbol

수학 기호 : https://ko.wikipedia.org/wiki/%EC%88%98%ED%95%99_%EA%B8%B0%ED%98%B8

가설검정 : 유의수준

제2종 오류

감마분포형태, 각도

미분 derivative

- ✓ Δ 와 d ??
- ✓ ∂ : round
- ✓ ∇ : del, nabla

모수

단위벡터

감마분포의 비율모수
역급수, 파장

모평균

$A \alpha$	Alpha (알파)	$N \nu$	Nu (뉴)
$B \beta$	Beta (베타)	$\Xi \xi$	Xi (크시)
$\Gamma \gamma$	Gamma (감마)	$O o$	Omicron (오미크론)
$\Delta \delta$	Delta (델타)	$\Pi \pi$	Pi (파이)
$E \epsilon$	Epsilon (엡실론)	$P \rho$	Rho (로우)
$Z \zeta$	Zeta (제타)	$\Sigma \sigma$	Sigma (시그마)
$H \eta$	Eta (에타)	$T \tau$	Tau (타우)
$\Theta \theta$	Theta (세타)	$Y \upsilon$	Upsilon (업실론)
$I \iota$	Iota (이오타)	$\Phi \phi$	Phi (파이)
$K \kappa$	Kappa (카파)	$X \chi$	Chi (카이)
$\Lambda \lambda$	Lambda (람다)	$\Psi \psi$	Psi (프사이)
$M \mu$	Mu (뮤)	$\Omega \omega$	Omega (오메가)

자유도 degree of freedom

좌표

누적곱 / 원주율

상관계수, 좌표

합계, 표준편차

시간

누적정규분포함수
평면의 각도

카이제곱분포

좌표, 각도

표본공간
각도, 저항

Σ , $\int$?